

全国大学生数学建模竞赛论文模板

摘要

摘要

对于问题一,

对于问题二,

对于问题三,

对于问题四,

最后,

关键字: 关键词 关键词 关键词 关键词 关键词

一、问题重述

1.1 问题背景

中药材干燥是影响成品品质的重要工序，热风烘干是常用方式之一。该过程通常分为预热平衡与恒温干燥两个阶段，通过调节烘房温湿环境实现药材脱水。烘干时，工艺参数若选择不合理，会造成干燥效率低、能耗高、品质波动等问题；传统试验优化又存在成本高、周期长等局限。为了解决这些问题，本文将采用数理分析与数值仿真方法，研究中药材烘干过程中的变化规律。

1.2 问题要求

问题 1 在热风烘干预热平衡阶段，考虑一形状近似为圆柱体、长度和半径已知的中药材。我们需要利用给出的药材初始温度、初始水分浓度、烘房温度与水分浓度随时间的变化情况，以及相关物性参数和水分浓度扩散系数经验公式，建立预热平衡阶段药材内部温度和水分浓度随时间、空间变化的数学模型，并基于该模型计算给出的 7 个时刻下，距药材中心 0, 0.5, 1, 1.5, 2cm 共 5 个位置处的药材温度和水分浓度，在论文中给出相应表格；同时计算 1800s 内每隔 1s、距药材中心每隔 0.1cm 的完整结果，所有结果保留四位小数。

问题 2 我们需要利用给出的相关经验公式，在预热平衡阶段与恒温干燥阶段的参数有所不同的前提下，建立整个烘干过程中药材温度和水分浓度变化规律的数学模型，并基于该模型计算 3h 内每隔 0.5h、距药材中心 0, 0.5, 1, 1.5, 2cm 处的药材温度和水分浓度，按表 3 和表 4 的格式在论文中给出；同时计算每隔 1s、距药材中心每隔 0.1cm 的完整结果，所有结果保留四位小数。

问题 3 我们需要利用给出的水分浓度限值，确定药材烘干所需的时间（单位：h），并计算每隔 6h、距药材中心每隔 0.5cm 的水分浓度，按表 5 的格式在论文中给出；同时计算药材内部水分浓度每隔 60s、距药材中心每隔 0.1cm 的完整结果，所有结果保留四位小数。

问题 4 在实际烘干过程中，药材会因水分流失发生尺寸变化。我们需要利用给出的药材尺寸变化数据和相关经验公式，确定药材的烘干时长，并计算每隔 6h、距药材中心每隔 0.5cm 的水分浓度，按表 6 的格式在论文中给出；同时计算药材内部水分浓度每隔 60s、距药材中心每隔 0.1cm 的完整结果，所有结果保留四位小数。

二、问题分析

2.1 问题一分析

对于问题一，要求在预热平衡阶段，建立圆柱形中药材内部温度与水分浓度时空演化的数学模型，本质上是一个给定外部热湿环境边界后，求解药材内部非稳态传热传质耦合偏微分方程的初边值问题。

从物理机理看，中药材内部结构复杂，鉴于题目提供的密度、比热容、导热系数与有效扩散系数均为宏观等效系数，本文采取宏观连续介质假设，将药材划分为微元控制体，建立宏观连续介质模型。同时，针对圆柱形及初始分布均匀的特点，忽略轴向与圆周方向的变化，简化为**一维径向模型**。基于傅里叶导热定律与有效 Fick 定律建立控制方程。由于水分扩散系数 D 是水分浓度 C 与温度 T 的非线性函数，模型呈现强非线性特征。边界条件方面，药材中心满足对称条件，表面则发生对流换热与对流传质，属于第三类边界条件。

在模型求解上，由于难以获得解析解，本文采用**显式有限差分法（FDM）**进行数值求解。求解主要面临两大难点：一是环境数据离散，且显式差分对边界条件的一阶导数极为敏感，常规线性插值易引发数值震荡，故引入 **PCHIP 插值**，保证插值曲线一阶导数连续且单调，避免非物理的过冲现象；二是需妥善处理圆柱中心的坐标奇点及表面通量平衡，为此本文利用洛必达法则处理中心节点，并基于有限体积法思想，保障数值计算的稳定性与物理守恒性。最后，从全局能量守恒与水分质量守恒两个维度对算法进行验证，确保结果可靠并满足精度要求。

2.2 问题二分析

对于问题二，在问题一基础上，需将密度、比热容、导热系数与扩散系数视为随局部温度与水分浓度变化的函数，并考虑烘房环境由预热平衡向恒温干燥的过渡，模型因此变为强非线性双向耦合问题。

针对该问题，本文仍采用显式有限差分法进行数值求解：在每一时间步内，根据当前温度与水分浓度更新物性参数，再推进节点更新；环境边界值采用 **PCHIP 插值** 获得，以保证边界导数连续、避免数值震荡；求解完成后，从能量守恒与水分质量守恒两个维度对算法进行验证，确保整个烘干过程数值结果的可靠性。

2.3 问题三分析

对于问题三，要求基于问题二的变物性模型，从初始时刻 $t = 0$ 出发，全过程计算直至药材各处水分浓度均低于 0.15 kg/kg ，从而确定烘干所需时间。与问题二不同，问题三不再划分预热与恒温两个独立阶段，而是使用拼接后的完整环境边界，一次求解至结束。主要难点在于：烘干过程长达数十小时，若沿用问题一的小时间步长将导致计

算量过大；同时，烘干结束时刻并非恰好落在网格点上，需通过插值获得精确的结束时间。为此，本文沿用问题二的系数滞后后向 Euler 半隐式格式，并适当增大时间步长至 60 s，在保证稳定性的前提下提高计算效率。每一步求解完成后，计算全域最大水分浓度 $C_{\max}(t_n)$ ，当首次满足 $C_{\max}(t_n) \leq 0.15$ 时，在相邻两个时间点之间线性插值得到烘干结束时间 t_d ，并据此输出完整结果。

2.4 问题四分析

对于问题四，在问题三的基础上进一步考虑药材因水分流失而发生的尺寸收缩。此时药材半径 $R(t)$ 随时间变化，计算区域从固定域变为移动边界域，直接求解会导致网格不断移动，增加数值处理难度。为此，本文引入材料坐标 $\xi = r/R(t)$ ，将移动边界问题转化为固定域 $0 \leq \xi \leq 1$ 上的问题，使网格固定不变。物性参数采用附录 4 的经验关系，控制方程中相应出现尺度因子 $1/R(t)^2$ ，表面边界条件也需除以 $R(t)$ 以保持通量一致。求解仍采用系数滞后的后向 Euler 半隐式格式，时间步长 60 s，每个时间步先更新物性参数与界面系数，再依次求解温度与水分三对角方程。输出时将材料坐标映射回物理位置 $r_i(t_n) = \xi_i R(t_n)$ ，从而得到收缩后药材内部各物理位置的水分浓度分布，并确定烘干结束时间。

符号	说明	单位
t	时间	s
L	药材长度	m
r	到药材中心的距离	m
R	药材半径	m
T_0	药材初始温度	°C
$T(r, t)$	药材内部温度	°C 或 K
T_s	药材表面温度	°C
$T_\infty(t)$	烘房温度	°C
C_0	药材初始水分浓度	kg/kg
$C(r, t)$	药材水分浓度	kg/kg
C_s	药材初始水分浓度	kg/kg
$C_\infty(t)$	烘房空气水分浓度	kg/kg
ρ	药材密度	kg/m ³
c_p	药材比热容	J/(kg · K)
k	药材热传导系数	W/(m · K)
α	热扩散系数 ($\alpha = k/(\rho c_p)$)	m ² /s
h	对流换热系数	W/(m ² · K)
h_m	对流传质系数	m/s
D	水分浓度扩散系数	m ² /s
Q_v	单位体积热量累积率	W/m ³
q	热流密度矢量	W/m ²
J_c	水分扩散通量矢量	kg/(m ² · s)
N_r	空间径向节点总数	—
N_t	时间节点总数	—

三、统一建模框架

3.1 模型选取

在建立中药材热风干燥数学模型前，需明确描述传热传质过程的数值模拟方法。查阅文献可知，农产品热风干燥的数值模拟方法主要可分为三类：干燥动力学模型、连续介质假设模型和孔道网络模型 [?]。

考虑到题目给出的密度、比热容、导热系数与有效扩散系数均为宏观等效系数，且问题需要获取药材内部温度与水分浓度的时空变化规律及烘干时长，这些特性更符合连

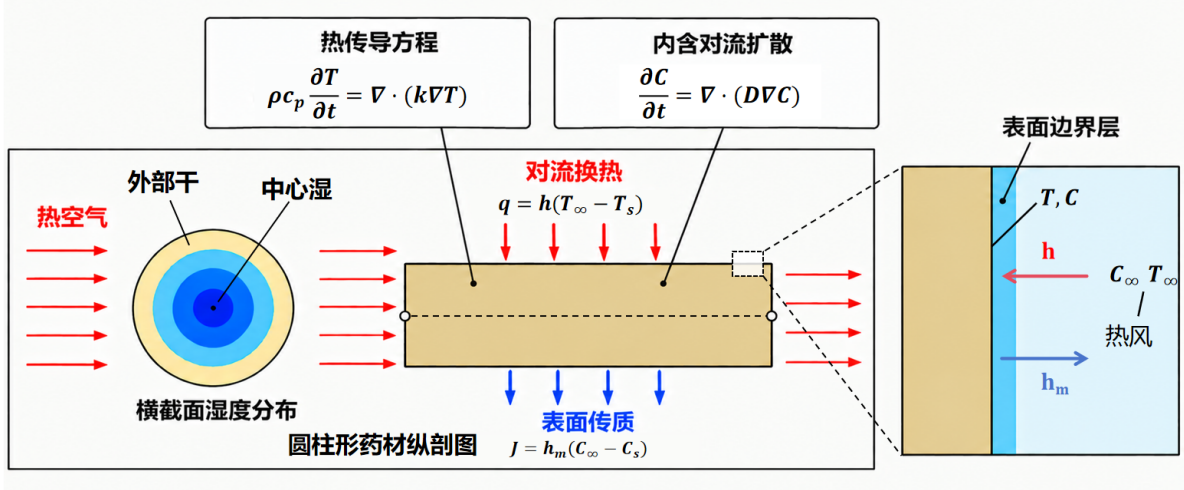


图 1 圆柱形药材热风烘干的传热传质机理示意图

续介质假设模型的应用条件。本文选择连续介质假设模型进行模拟。即将药材等效为均匀连续的多孔介质，建立描述其内部水分浓度与温度时空演化的偏微分方程组，结合给定的初始条件与边界条件，使用数值方法求解，最终得到温度和水分浓度分布。

3.2 研究对象

将药材视为等效均匀连续介质。药材近似为长度 $L = 0.25 \text{ m}$ 、初始半径 $R_0 = 0.02 \text{ m}$ 的圆柱体。记 r 为距圆柱中心的径向距离， $T(r, t)$ 为药材温度， $C(r, t)$ 为干基水分浓度， t 为时间。温度数值均以摄氏度表示；经验公式中使用开尔文温度

$$T_K = T + 273.15. \quad (1)$$

药材初始状态为

$$T(r, 0) = T_0 = 28^\circ\text{C}, \quad C(r, 0) = C_0 = 2.55 \text{ kg/kg}. \quad (2)$$

3.3 基本假设

- 药材内部初始温度和含水率在径向上均匀分布；
- 忽略轴向和圆周方向的传递，仅考虑半径方向的变化；
- 长度在径向控制方程中约去，守恒量按单位轴向长度计算；
- 问题一中的 ρ, c_p, k 为常数， D 仅依赖 C ；问题二和问题三使用附录 3 的变物性关系；问题四使用附录 4 关系并进一步考虑半径变化。

3.4 统一的热质传递方程

3.4.1 热量守恒方程

取药材内部一微元控制体，由能量守恒定律可知：单位时间内控制体内热量的累积等于净流入控制体的热量与内热源生成热量之和。本文忽略内热源，则单位体积内热量的累积率可表示为

$$Q_v = \rho c_p \frac{\partial T}{\partial t} \quad (3)$$

式中， Q_v 为单位体积热量累积率， ρ 为药材密度， c_p 为药材比热容， T 为药材内部温度， t 为时间。

药材内部热量主要以导热方式传递。根据傅里叶定律，热流密度与温度梯度成正比、方向相反，即

$$q = -k \nabla T \quad (4)$$

式中， q 为热流密度矢量， k 为药材热传导系数， ∇ 表示梯度算子， $\nabla \cdot$ 表示散度算子。

由能量守恒定律，控制体内热量的累积率等于热流密度的散度，即

$$\rho c_p \frac{\partial T}{\partial t} = -\nabla \cdot q \quad (5)$$

将式 (2) 代入式 (3)，得到药材内部温度场的控制方程

$$\rho c_p \frac{\partial T}{\partial t} = \nabla \cdot (k \nabla T) \quad (6)$$

3.4.2 水分扩散方程

药材内部水分的迁移以扩散为主。根据有效 Fick 定律，水分扩散通量与水分浓度梯度成正比、方向相反，即

$$J_c = -D \nabla C \quad (7)$$

式中， J_c 为水分扩散通量矢量， D 为水分浓度扩散系数， C 为药材水分浓度。

由水分质量守恒定律，控制体内水分浓度的变化率等于水分扩散通量的散度，即

$$\frac{\partial C}{\partial t} = -\nabla \cdot J_c \quad (8)$$

将式 (5) 代入式 (6)，得到药材内部水分浓度场的控制方程

$$\frac{\partial C}{\partial t} = \nabla \cdot (D \nabla C) \quad (9)$$

需要指出的是，题目附录中给出的扩散系数 D 并非常数，而是与水分浓度 C 及温度 T 相关的函数。因此，式 (7) 应进一步写为

$$\frac{\partial C}{\partial t} = \nabla \cdot (D(C, T) \nabla C) \quad (10)$$

3.4.3 一维径向模型

考虑到药材初始温度与含水率分布均匀，且药材几何形状为圆柱体，可作如下简化：忽略沿轴向的变化，忽略沿圆周方向的变化，仅考虑从药材中心到表面的径向变化。由此，三维问题可退化为沿半径方向的一维问题。

在柱坐标系下，对任一标量场 u 和传输系数 q ，有恒等式

$$\nabla \cdot (q \nabla u) = \frac{1}{r} \frac{\partial}{\partial r} \left(r q \frac{\partial u}{\partial r} \right) \quad (11)$$

式中， r 为到药材中心的径向距离。

将式 (9) 分别代入温度控制方程 (4) 和水分浓度控制方程 (8)，可得一维径向模型下的控制方程组：

$$\rho(C) c_p(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right) \quad (12)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C, T) \frac{\partial C}{\partial r} \right) \quad (13)$$

3.5 初始条件与边界条件

3.5.1 初始条件

烘干开始时，药材内部温度和水分浓度分布均匀，由题意可得

$$T(r, 0) = 28^\circ\text{C}, \quad C(r, 0) = 2.55 \text{ kg/kg}, \quad 0 \leq r \leq R \quad (14)$$

式中， R 为药材半径。根据题目给定尺寸，半径 $R = 0.02 \text{ m}$ 。

3.5.2 中心对称条件

在圆柱中心 $r = 0$ 处，由对称性可知热流密度与水分通量均为零，即

$$\left. \frac{\partial T}{\partial r} \right|_{r=0} = 0, \quad \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0 \quad (15)$$

3.5.3 表面对流换热条件

烘房空气通过对流将热量传递给药材，药材表面温度不一定等于空气温度，药材内部导热与外部对流共同决定其升温过程。由于题目未直接指定药材表面温度，采用 Newton 对流边界条件：

$$-k \left. \frac{\partial T}{\partial r} \right|_{r=R} = h [T(R, t) - T_\infty(t)] \quad (16)$$

式中， h 为对流换热系数， $T_f(t)$ 为烘房温度，其随时间的变化由附件 1 给出。

3.5.4 表面对流传质条件

药材表面含水率与空气水分浓度之间存在差异，水分先从药材内部扩散至表面，再通过气膜传递至空气中。因此，药材表面满足对流传质边界条件

$$-D \frac{\partial C}{\partial r} \Big|_{r=R} = h_m [C(R, t) - C_\infty(t)] \quad (17)$$

式中， h_m 为对流传质系数， $C_\infty(t)$ 为烘房空气水分浓度，其随时间的变化由附件 1 给出。

3.6 环境数据处理与两阶段约定

根据给出的离散时刻的烘房温度和空气水分浓度。问题一先用 PCHIP（分段三次 Hermite 保形插值）将原始数据插值到 1s 网格，输出相应结果。问题二至问题四调用边界时采用离散数据的分段线性插值；当查询时间超过数据末端时，取末尾 1000 个数据点的均值作常值延拓。

3.7 稳定性判断与边界切换

由附件 1 的稳定性判断得到约 5439s 的稳定起点。为了使阶段初值、边界切换和输出时间原点统一，程序取

$$t_s = 5500 \text{ s}. \quad (18)$$

因此，所有全过程计算均按

$$B(t) = \begin{cases} (T_1(t), C_1(t)), & 0 \leq t \leq t_s, \\ (T_2(t), C_2(t)), & t > t_s, \end{cases} \quad (19)$$

调用边界。其中第一段为附件 1 的变化边界，第二段为稳定阶段边界。问题二的结果表以 t_s 为新的时间原点，表中 0 10800s 表示绝对时间 5500 16300s；问题三和问题四从原始 $t = 0$ 开始计时。

3.8 数值保护约定

程序在每一步求解后检查数组是否有限，并对经验公式的输入作物理范围保护：问题二、三中评价 $D(C, T)$ 时将 C 限制在 $[0.05, 10]$ ， T_K 限制在 $[273.15, 373.15] \text{ K}$ ，并将 D 限制在 $[10^{-14}, 10^{-3}] \text{ m}^2/\text{s}$ ；问题四采用同样的温度和扩散系数保护。状态量的保护范围为 $-50 \leq T \leq 250^\circ\text{C}$ ， $0 \leq C \leq 10$ 。验证结果表明正式计算未触及这些保护边界，因此我们进行的保护操作不改变本题数值解。

四、问题一：常物性径向模型的建立和求解

4.1 模型建立与求解

本文建立的一维径向传热传质控制方程属于非线性偏微分方程组，难以获得解析解。为此，本文采用显式有限差分法（FDM）对空间域和时间域进行离散，通过时间推进循环求解药材内部温度和水分浓度的时空分布。

4.1.1 时空离散与网格划分

在空间域上，将药材半径 R 离散为 $N_r = 21$ 个节点，空间步长 $\Delta r = 0.001 \text{ m}$ ，节点位置为 $r_i = i\Delta r$ ($i = 0, 1, \dots, N_r - 1$)。在时间域上，取时间步长 $\Delta t = 1 \text{ s}$ ，记 $t_n = n\Delta t$ ($n = 0, 1, \dots, N_t - 1$)，本文问题一的总计算区间为 $0 \leq t_n \leq 1800 \text{ s}$ 。

令系数

$$\alpha = \frac{k}{\rho c_p}, \quad \lambda_i^+ = 1 + \frac{\Delta r}{2r_i}, \quad \lambda_i^- = 1 - \frac{\Delta r}{2r_i}. \quad (20)$$

4.1.2 边界条件的数据预处理

在算法的时间推进过程中，表面节点及内部节点的更新均高度依赖于烘房环境的实时边界值 $T_\infty(t)$ 和 $C_\infty(t)$ 。由于仅提供了有限个离散时间点的环境参数数据，必须通过插值获取任意时间步 $n + 1$ 对应的环境值 T_∞^{n+1} 和 C_∞^{n+1} 。

由于显式有限差分法对边界条件的一阶导数十分敏感，若采用常规线性插值，其在节点处导数不连续（折点）的非光滑特性极易引起数值震荡，进而导致求解发散。因此，本文选取 PCHIP（分段三次 Hermite 插值多项式）对烘房环境参数进行插值。PCHIP 不仅保证了插值曲线的一阶导数连续，符合物理量平滑变化的规律，同时保持数据的单调性，避免非物理的过冲现象。这使得插值得到的水分浓度不会出现负值，为后续偏微分方程的数值求解提供了平滑且稳定的边界输入，保障了计算收敛性。

4.1.3 内部节点更新

对内部节点 $i = 1, 2, \dots, N_r - 2$ ，温度采用显式柱坐标中心差分格式：

$$T_i^{n+1} = T_i^n + \alpha \frac{\Delta t}{\Delta r^2} [\lambda_i^+ T_{i+1}^n - 2T_i^n + \lambda_i^- T_{i-1}^n]. \quad (21)$$

水分方程则采用守恒型界面通量离散，而不再直接使用节点扩散系数。定义界面扩散系数为左右节点浓度的算术平均：

$$D_{i+1/2}^n = D \left(\frac{C_i^n + C_{i+1}^n}{2} \right), \quad D_{i-1/2}^n = D \left(\frac{C_{i-1}^n + C_i^n}{2} \right). \quad (22)$$

则内部节点的水分更新式为

$$C_i^{n+1} = C_i^n + \frac{\Delta t}{\Delta r^2} \left[\lambda_i^+ D_{i+1/2}^n (C_{i+1}^n - C_i^n) + \lambda_i^- D_{i-1/2}^n (C_{i-1}^n - C_i^n) \right]. \quad (23)$$

同一界面系数由相邻两个控制体共用，因此内部界面通量在全局求和时成对抵消，从而在数值上保证质量守恒。

4.1.4 中心节点更新

在圆柱中心 $r = 0$ 处，由对称性可知热流与水分通量均为零。利用洛必达法则对柱坐标方程取中心极限，得到中心节点的更新公式：

$$T_0^{n+1} = T_0^n + 4\alpha \frac{\Delta t}{\Delta r^2} (T_1^n - T_0^n), \quad (24)$$

$$C_0^{n+1} = C_0^n + 4D_{1/2}^n \frac{\Delta t}{\Delta r^2} (C_1^n - C_0^n), \quad (25)$$

其中 $D_{1/2}^n = D((C_0^n + C_1^n)/2)$ 。

4.1.5 表面节点更新

记 $i_s = N_r - 1$ ，表面节点控制体的单位长度体积为

$$V_s = \pi \left[R_0^2 - \left(R_0 - \frac{\Delta r}{2} \right)^2 \right] = \pi \Delta r^2 \left(i_s - \frac{1}{4} \right). \quad (26)$$

温度表面更新式由内表面导热、外表面对流以及控制体热量累积三部分构成：

$$T_{i_s}^{n+1} = T_{i_s}^n + \Delta t \left[\frac{2(i_s - \frac{1}{2})k}{\rho c_p (i_s - \frac{1}{4}) \Delta r^2} (T_{i_s-1}^n - T_{i_s}^n) + \frac{2i_s h}{\rho c_p (i_s - \frac{1}{4}) \Delta r} (T_\infty^{n+1} - T_{i_s}^n) \right]. \quad (27)$$

令表面处界面扩散系数为 $D_{s-1/2}^n = D\left(\frac{C_{i_s-1}^n + C_{i_s}^n}{2}\right)$ ，水分表面更新式为

$$C_{i_s}^{n+1} = C_{i_s}^n + \Delta t \left[\frac{2(i_s - \frac{1}{2})D_{s-1/2}^n}{(i_s - \frac{1}{4}) \Delta r^2} (C_{i_s-1}^n - C_{i_s}^n) + \frac{2i_s h_m}{(i_s - \frac{1}{4}) \Delta r} (C_\infty^{n+1} - C_{i_s}^n) \right]. \quad (28)$$

式中， h 和 h_m 分别为对流换热系数和对流传质系数。此处环境边界值 T_∞^{n+1} 和 C_∞^{n+1} 使用下一时刻的插值取值，而表面状态依赖当前时刻值，这正是本文程序显式实现中的边界处理方式。

4.1.6 求解流程与结果输出

上述显式差分的求解流程如图 2 所示。算法首先初始化温度与水分浓度矩阵，并计算热扩散系数；随后进入时间推进循环（ $n = 0$ 到 $N_t - 2$ ），在每一时间步内，先利用 PCHIP 插值获取下一时刻的环境值 T_∞^{n+1} 和 C_∞^{n+1} ，接着计算当前时刻各节点的界面扩散系数 $D_{i+1/2}^n$ ，并依次更新内部节点、中心节点及表面节点的温度与水分浓度；循环结束后，输出完整的温度矩阵 T_i^n 和水分矩阵 C_i^n ，并按要求提取特定时刻与径向位置的切片数据。

4.1.7 稳定性条件

显式差分格式需满足 CFL 稳定性条件。在本题参数下，

$$\frac{\alpha \Delta t}{\Delta r^2} = 0.1689, \quad 4 \frac{\alpha \Delta t}{\Delta r^2} = 0.6754 < 1. \quad (29)$$

初始含水率下中心水分更新系数约为 0.0198，满足显式推进的稳定性要求。程序输出 $0 \sim 1800$ s 内每隔 1 s、径向每隔 0.1 cm 的完整温度和水分结果。论文用表在 $t \in \{100, 300, 600, 900, 1200, 1500, 1800\}$ s 以及 $r \in \{0, 0.5, 1, 1.5, 2\}$ cm 处提取。

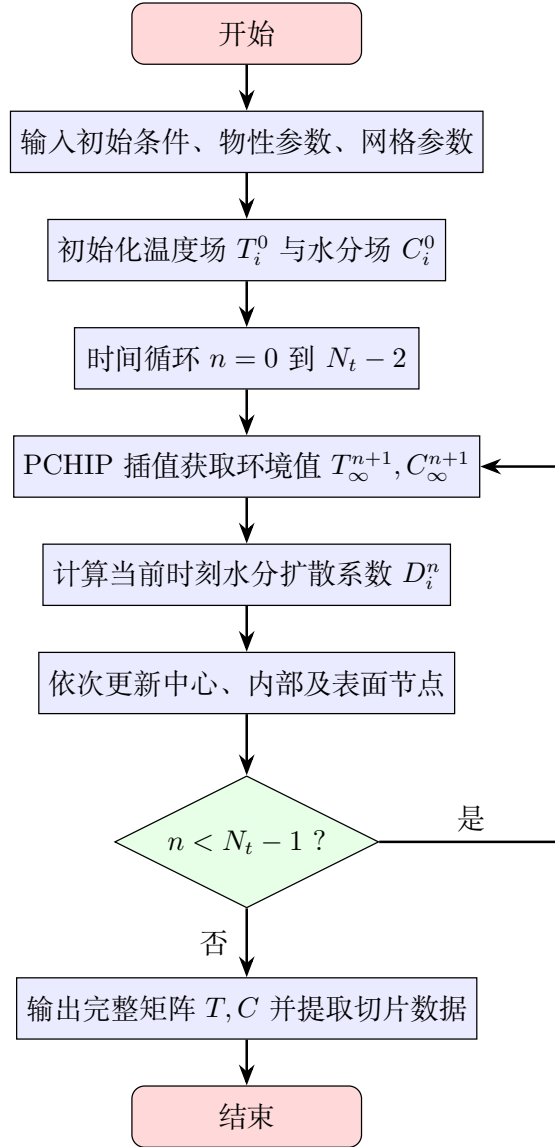


图 2 问题一算法流程图

4.1.8 求解结果

结果表明，总能量变化曲线与边界热流累积曲线在整个模拟时段内几乎完全重合；总水分变化曲线与边界质流累积曲线同样几乎完全重合。两项守恒误差均远小于 5% 的工程容许阈值，充分证明了本文所建立的显式有限差分算法具有良好的守恒性与数值稳定性，能够为后续中药材烘干过程的温度场与水分场求解提供可靠的数值保障。

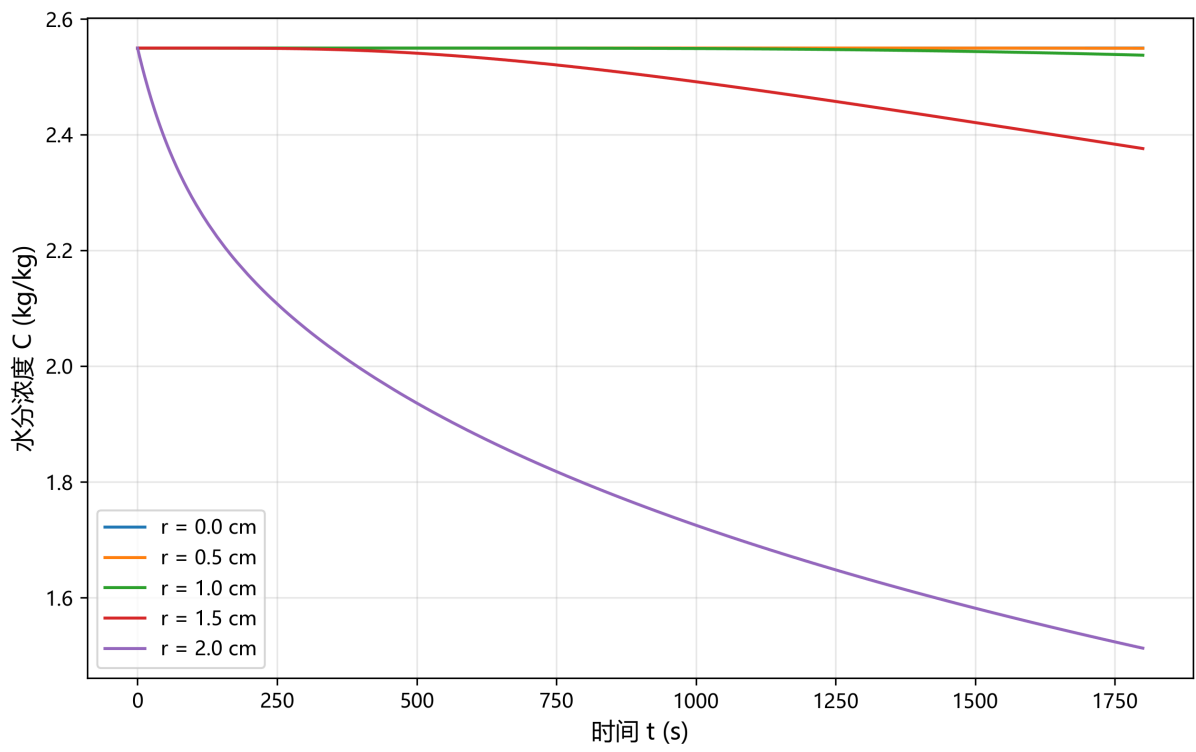


图 3 问题一水分浓度随时间的变化

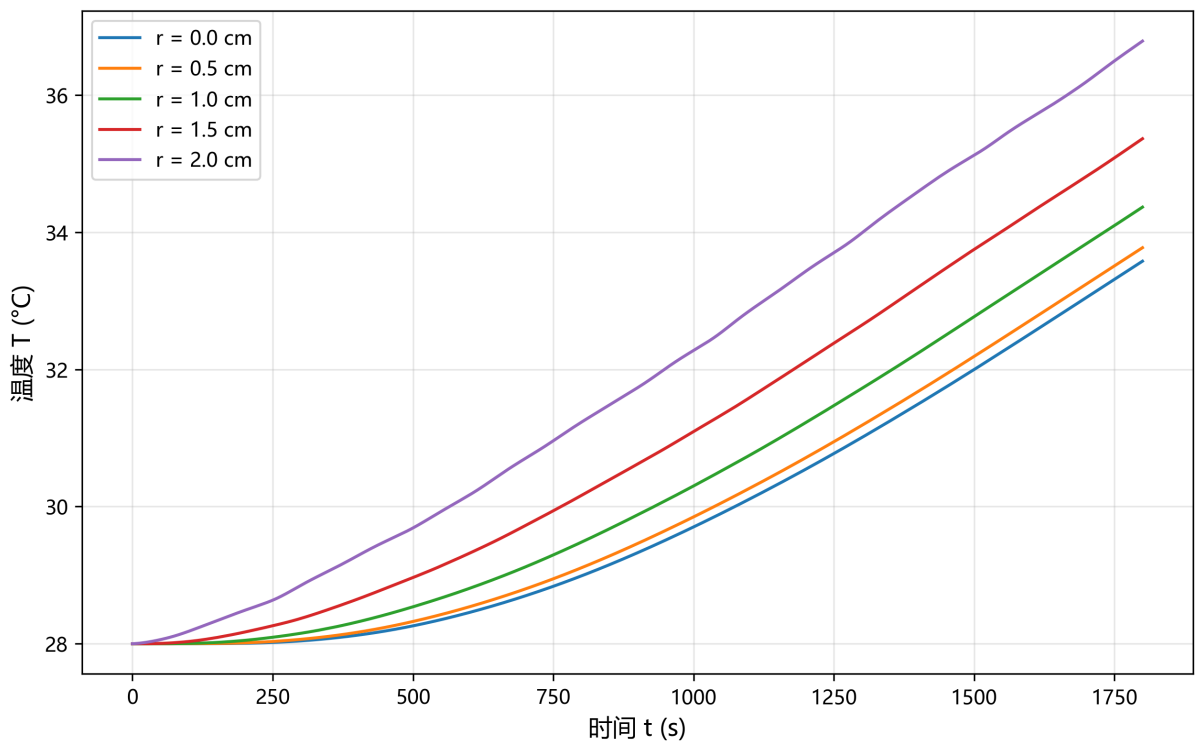


图 4 问题一温度随时间的变化

表 1 30 分钟内药材的温度

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	28.0000	28.0003	28.0039	28.032	28.1803
300	28.0408	28.0636	28.1517	28.3681	28.8504
600	28.4543	28.5372	28.8053	29.3175	30.1667
900	29.3262	29.4604	29.8781	30.6192	31.7329
1200	30.5456	30.7128	31.2256	32.1166	33.4336
1500	31.9994	32.1906	32.7705	33.7517	35.1255
1800	33.5798	33.7764	34.3684	35.3666	36.7905

表 2 30 分钟内药材的水分浓度

时间/s	到药材中心距离/cm				
	0	0.5	1	1.5	2
100	2.55	2.55	2.55	2.55	2.2875
300	2.55	2.55	2.55	2.5486	2.066
600	2.55	2.55	2.55	2.5342	1.8847
900	2.55	2.55	2.5495	2.5041	1.7598
1200	2.55	2.55	2.5478	2.4647	1.6623
1500	2.55	2.5499	2.544	2.4211	1.5816
1800	2.55	2.5496	2.5376	2.3762	1.5125

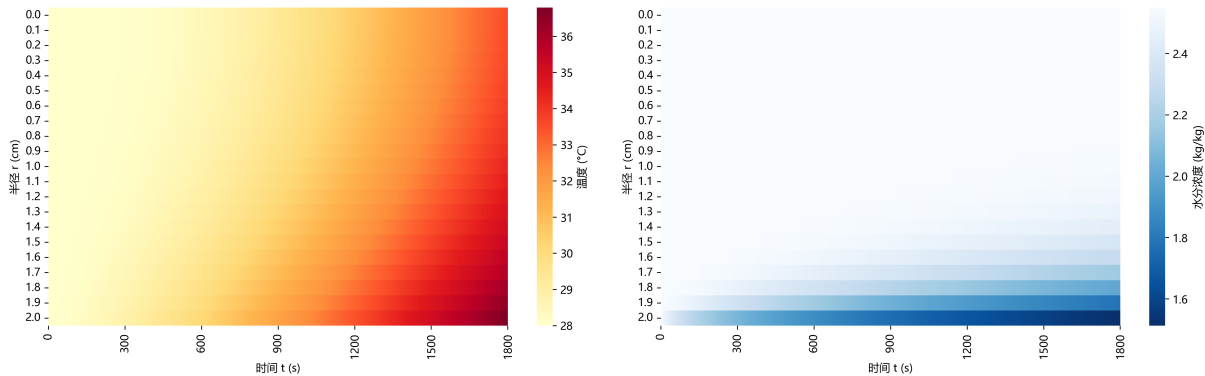


图 5 问题一温度 and 水分浓度热力图

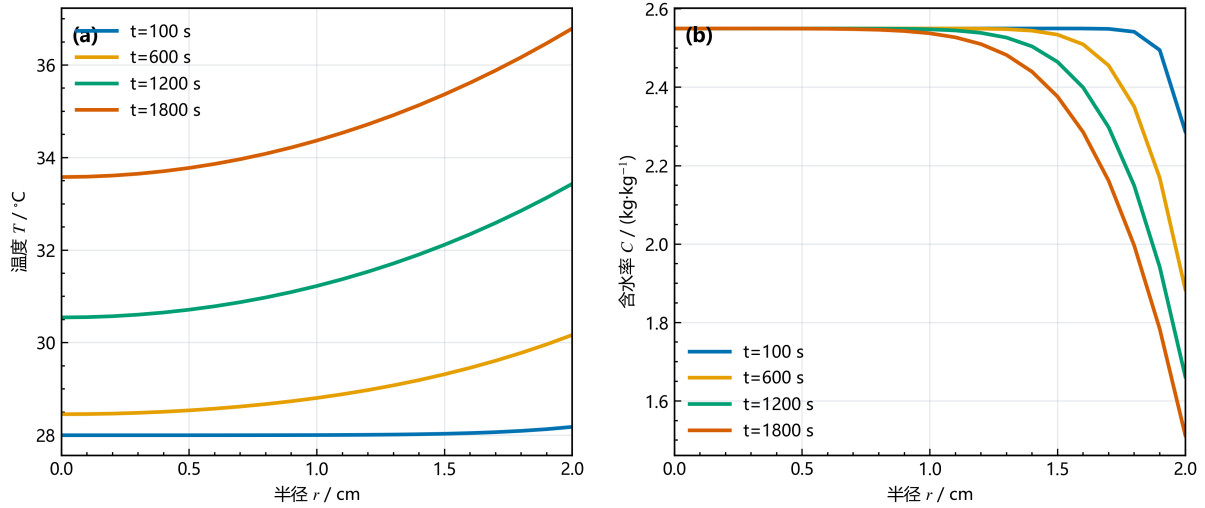


图 6 问题一径向温度和含水率分布

五、问题二：变物性两阶段模型的建立和求解

5.1 模型建立

问题一将密度 ρ 、比热容 c_p 、热传导系数 k 视为常数，水分扩散系数 D 仅依赖 C 。然而整个烘干过程持续 2 ~ 3 天，药材内部水分浓度发生显著变化，物性参数也随之改变，且烘房环境由预热平衡阶段过渡到恒温干燥阶段。因此，问题二在问题一的基础上，将物性参数改为随局部温度与水分浓度变化的函数，并考虑热质传递的双向耦合效应。

5.1.1 变物性关系

问题二使用给出的经验关系：

$$\rho(C) = 650 + 128C, \quad (30)$$

$$c_p(C) = 1450 + 2736 \frac{C}{C+1}, \quad (31)$$

$$k(C) = 0.21 + 0.38 \frac{C}{C+1}, \quad (32)$$

$$D(C, T) = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left(-\frac{3850}{T_K}\right), \quad (33)$$

其中 $T_K = T + 273.15$ 为开尔文温度。

需要强调的是：预热平衡与恒温干燥这两个阶段使用的是同一套附录 3 物性公式，两阶段的差异体现在外部环境边界与时间分段上，而非重新替换物性参数。

5.1.2 双向耦合关系

在变物性条件下，温度场与水分浓度场之间形成双向耦合，耦合路径为：

$$\begin{aligned} C &\rightarrow \rho, c_p, k \rightarrow T, \\ (C, T) &\rightarrow D \rightarrow C. \end{aligned}$$

即水分浓度的变化通过改变密度、比热容与导热系数进而影响温度场；同时温度与水分浓度又共同决定水分扩散系数，进而反过来影响水分浓度场。这与问题一中温度场不依赖水分场的准线性情形形成本质区别。

5.2 阶段切换与恒温段初值

整个烘干过程按统一建模框架中的约定划分为两段：预热平衡段与恒温干燥段，切换时刻取 $t_s = 5500$ s。预热阶段由 $T = 28^\circ\text{C}$, $C = 2.55$ kg/kg 的均匀初态开始，使用绝对时间 $0 \sim 5500$ s 的附件 1 边界进行求解，得到 5500 s 时刻的温度与水分剖面并保存为输出文件。

恒温段以该剖面为初值，再计算 10800 s，生成问题二的最终结果。因此，恒温段求解器内部的时间记为 τ ，而边界查询使用绝对时间 $t = t_s + \tau$ ：

$$T_\infty^{n+1} = T_2(t_s + \tau_{n+1}), \quad C_\infty^{n+1} = C_2(t_s + \tau_{n+1}), \quad (34)$$

其中 T_2, C_2 为统一建模框架中定义的稳定阶段边界函数。结果文件中的时间列为 $\tau = 0, 1, \dots, 10800$ s。

5.3 系数滞后的后向 Euler 半隐式格式

由于 ρ, c_p, k, D 均为 C 与 T 的函数，直接采用显式格式会受严格的稳定性条件限制。为此，本文采用**系数滞后的后向 Euler 半隐式格式**：每一时间步内，先由旧时刻的 C^n, T^n 计算物性参数与界面系数，再对新时刻的未知量组装三对角方程组并求解。该格式兼顾了后向 Euler 的无条件稳定性与显式物性计算的简洁性。

空间网格取 $r_i = i\Delta r$, $\Delta r = 0.001$ m, $N_r = 21$ ；时间步长 $\Delta t = 1$ s。界面处的物性系数取左右节点算术平均：

$$k_{i+1/2}^n = k \left(\frac{C_i^n + C_{i+1}^n}{2} \right), \quad (35)$$

$$D_{i+1/2}^n = D \left(\frac{C_i^n + C_{i+1}^n}{2}, \frac{T_i^n + T_{i+1}^n}{2} \right). \quad (36)$$

5.3.1 内部节点的隐式方程

对内部节点 $i = 1, 2, \dots, N_r - 2$, 定义

$$a_i^T = \frac{\Delta t}{\rho(C_i^n) c_p(C_i^n) \Delta r^2}, \quad \lambda_i^\pm = 1 \pm \frac{\Delta r}{2r_i}, \quad (37)$$

则温度三对角方程为

$$\begin{aligned} & -a_i^T \lambda_i^- k_{i-1/2}^n T_{i-1}^{n+1} + \left[1 + a_i^T \left(\lambda_i^- k_{i-1/2}^n + \lambda_i^+ k_{i+1/2}^n \right) \right] T_i^{n+1} \\ & - a_i^T \lambda_i^+ k_{i+1/2}^n T_{i+1}^{n+1} = T_i^n. \end{aligned} \quad (38)$$

水分内部方程具有相同的守恒结构。令 $a^C = \Delta t / \Delta r^2$, 则

$$\begin{aligned} & -a^C \lambda_i^- D_{i-1/2}^n C_{i-1}^{n+1} + \left[1 + a^C \left(\lambda_i^- D_{i-1/2}^n + \lambda_i^+ D_{i+1/2}^n \right) \right] C_i^{n+1} \\ & - a^C \lambda_i^+ D_{i+1/2}^n C_{i+1}^{n+1} = C_i^n. \end{aligned} \quad (39)$$

5.3.2 中心节点的隐式方程

令 $k_{1/2}^n = k((C_0^n + C_1^n)/2)$, $\beta_T = \Delta t / [\rho(C_0^n) c_p(C_0^n) \Delta r^2]$, 则中心温度满足

$$\left(1 + 4\beta_T k_{1/2}^n \right) T_0^{n+1} - 4\beta_T k_{1/2}^n T_1^{n+1} = T_0^n. \quad (40)$$

相应地, 令

$$D_{1/2}^n = D \left(\frac{C_0^n + C_1^n}{2}, \frac{T_0^n + T_1^n}{2} \right), \quad (41)$$

中心水分方程为

$$\left(1 + 4a^C D_{1/2}^n \right) C_0^{n+1} - 4a^C D_{1/2}^n C_1^{n+1} = C_0^n. \quad (42)$$

5.3.3 表面节点的隐式方程

记 $i_s = N_r - 1$, $V_f = i_s - 1/4$, 令

$$k_s^n = k \left(\frac{C_{i_s-1}^n + C_{i_s}^n}{2} \right), \quad (43)$$

$$\beta_s^T = \frac{\Delta t}{\rho(C_{i_s}^n) c_p(C_{i_s}^n) V_f \Delta r^2}, \quad A_T = 2 \left(i_s - \frac{1}{2} \right) k_s^n, \quad B_T = 2i_s h \Delta r, \quad (44)$$

则表面温度方程为

$$-\beta_s^T A_T T_{i_s-1}^{n+1} + [1 + \beta_s^T (A_T + B_T)] T_{i_s}^{n+1} = T_{i_s}^n + \beta_s^T B_T T_\infty^{n+1}. \quad (45)$$

对于表面水分方程, 令

$$D_s^n = D \left(\frac{C_{i_s-1}^n + C_{i_s}^n}{2}, \frac{T_{i_s-1}^n + T_{i_s}^n}{2} \right), \quad (46)$$

$$G_1 = \frac{2\Delta t (i_s - 1/2) D_s^n}{V_f \Delta r^2}, \quad G_2 = \frac{2\Delta t i_s h_m}{V_f \Delta r}, \quad (47)$$

则表面水分方程为

$$-G_1 C_{i_s-1}^{n+1} + (1 + G_1 + G_2) C_{i_s}^{n+1} = C_{i_s}^n + G_2 C_\infty^{n+1}. \quad (48)$$

5.4 求解顺序与输出

每个时间步内的计算顺序为：先用旧时刻的 C^n, T^n 计算所有物性参数与界面系数；然后求解温度三对角方程；再用同一旧时刻的界面扩散系数求解水分三对角方程。整个时间步内不进行非线性迭代，两组线性方程组均使用追赶法（Thomas 算法）求解，效率较高。

程序输出 10800 s 内每隔 1 s、径向每隔 0.1 cm 的完整结果；论文用表按 0.5 h、0.5 cm 抽取。

5.5 求解结果

表 3 3 小时内药材的温度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	48.5403	48.5761	48.6808	48.8471	49.0497
1.0	49.4961	49.5077	49.5442	49.6138	49.7045
1.5	49.8659	49.8729	49.8947	49.9328	49.9776
2.0	49.9494	49.9495	49.952	49.9606	49.9619
2.5	49.9975	49.9962	49.9928	49.9897	49.9902
3.0	49.9965	49.9966	49.9967	49.997	49.9973

表 4 3 小时内药材的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	2.1583	2.0962	1.9125	1.6167	1.2243
1.0	1.9454	1.8897	1.7261	1.4637	1.1102
1.5	1.7563	1.707	1.5615	1.3258	1.002
2.0	1.591	1.5468	1.4159	1.2015	0.9015
2.5	1.446	1.4059	1.2867	1.0899	0.8103
3.0	1.3177	1.2811	1.1718	0.9902	0.7281

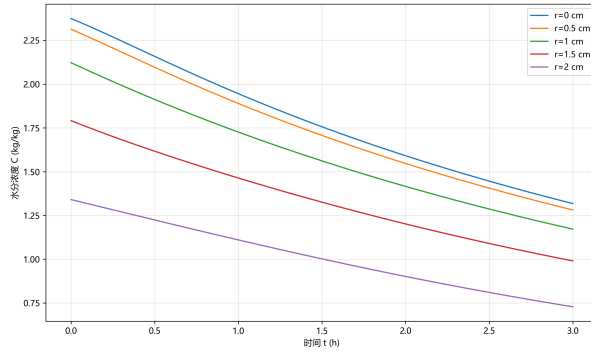


图 7 问题二水分浓度随时间的变化

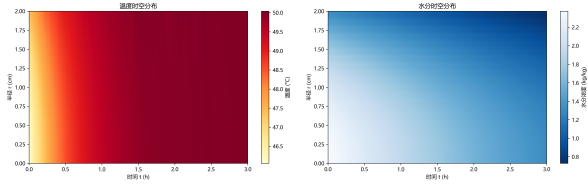


图 8 问题二温度和水分浓度热力图

六、 问题三：全过程固定半径模型的建立与求解

6.1 模型建立

问题三沿用问题二的固定半径假设与附录 3 的变物性关系，即

$$R(t) \equiv R_0 = 0.02 \text{ m}, \quad (49)$$

且物性参数 $\rho(C)$, $c_p(C)$, $k(C)$ 与扩散系数 $D(C, T)$ 的表达式与问题二完全一致。所不同的是，问题三不再将 5500 s 之后的阶段作为新的初值问题处理，而是从 $t = 0$ 的原始均匀状态出发，直接计算至烘干结束。因此，环境边界使用统一建模框架中定义的拼接边

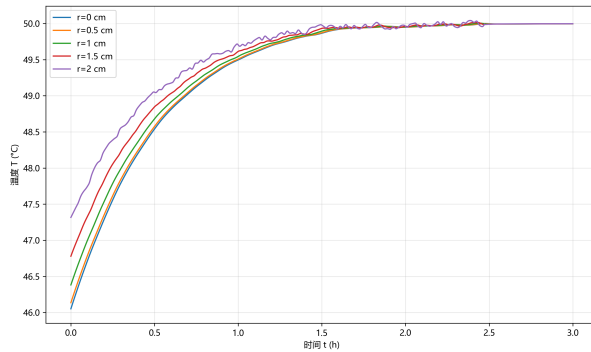


图 9 问题二温度随时间的变化

图 10 问题二的时空演化图

界 $B(t)$ ，即

$$B(t) = \begin{cases} (T_1(t), C_1(t)), & 0 \leq t \leq t_s, \\ (T_2(t), C_2(t)), & t > t_s, \end{cases} \quad (50)$$

其中 $t_s = 5500 \text{ s}$ ， T_1, C_1 为附件 1 的变化边界， T_2, C_2 为稳定阶段边界。控制方程、中心对称条件与表面 Robin 条件均与问题二相同，此处不再重复。

6.2 全过程求解

问题三采用与问题二相同的系数滞后后向 Euler 半隐式控制体格式，但时间步长取 $\Delta t = 60 \text{ s}$ ，空间步长仍为 $\Delta r = 0.001 \text{ m}$ 。最大绝对计算时间设为

$$t_{\max} = 259200 \text{ s} = 72 \text{ h}. \quad (51)$$

每一步使用绝对时间查询拼接后的环境边界值 T_{∞}^{n+1} 与 C_{∞}^{n+1} ，初值为

$$T_i^0 = 28^\circ\text{C}, \quad C_i^0 = 2.55 \text{ kg/kg}, \quad i = 0, 1, \dots, N_r - 1. \quad (52)$$

温度与水分的三对角方程组形式与问题二完全一致，此处直接引用。

6.3 烘干结束判据

根据题意，药材内部各处的水分浓度均应低于 0.15 kg/kg 。为判断烘干是否结束，定义各时刻药材内部水分浓度的最大值为

$$C_{\max}(t_n) = \max_{0 \leq i \leq N_r - 1} C_i^n. \quad (53)$$

取首次满足 $C_{\max}(t_j) \leq 0.15$ 的网格时刻作为阈值跨越点。若上一时刻仍有 $C_{\max}(t_{j-1}) > 0.15$ ，则在区间 $[t_{j-1}, t_j]$ 上对 $C_{\max}$ 进行线性插值，得到精确的烘干结束时间

$$t_d = t_{j-1} + \frac{0.15 - C_{\max}(t_{j-1})}{C_{\max}(t_j) - C_{\max}(t_{j-1})}(t_j - t_{j-1}). \quad (54)$$

程序按 60 s 保存全过程结果，并在末尾追加 t_d 对应的线性插值行；论文用表按每 6 h 及结束时刻提取。本次计算首次达到阈值的时间为

$$t_d = 210634.7398 \text{ s} = 58.50965 \text{ h}. \quad (55)$$

表 5 药材烘干过程的水分浓度

时间/h	到药材中心距离/cm				
	0	0.5	1	1.5	2
6	1.0186	0.9897	0.903	0.7563	0.5342
12	0.4568	0.4437	0.4036	0.3302	0.1651
18	0.2992	0.2919	0.2692	0.226	0.0843
...					
烘干结束时间	0.15	0.1478	0.1406	0.126	0.0525

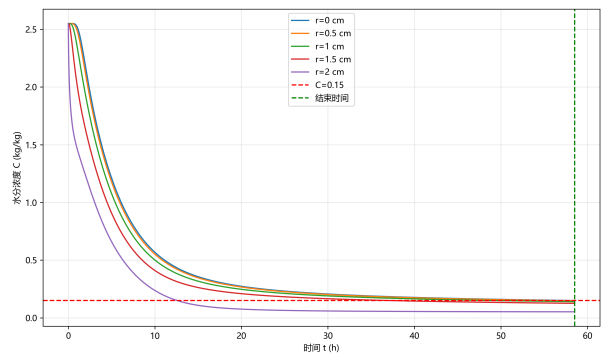


图 11 问题三水分浓度随时间的变化

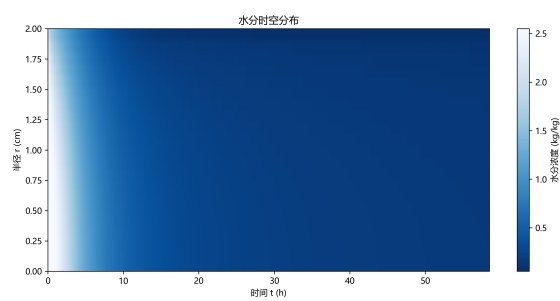


图 12 问题三水分浓度热力图

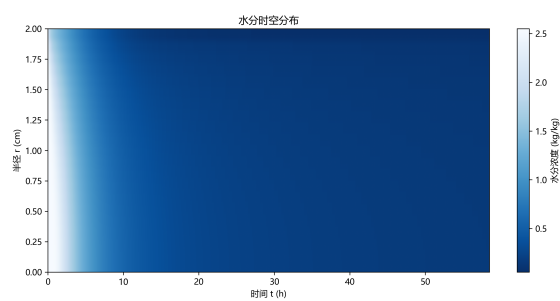


图 13 问题三水分浓度热力图（重复绘制）

图 14 问题三达到干燥标准前后的径向含水率

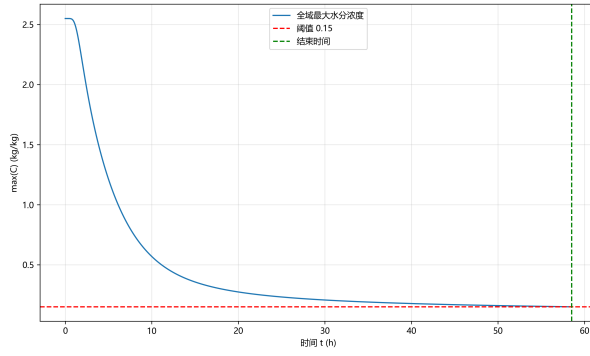


图 15 问题三干燥阈值判定

6.4 求解结果

按照上述模型与判据，程序输出 $0 \sim t_d$ 内每隔 60 s、径向每隔 0.1 cm 的完整水分浓度结果，并按表 5 的格式给出每隔 6 h、距药材中心每隔 0.5 cm 的水分浓度。由计算结果可知，药材各处水分浓度均低于 0.15 kg/kg 所需的时间约为 58.51 h，即约 2.44 天，符合题目中烘干过程持续 2 ~ 3 天的实际情形。

七、问题四：变半径材料坐标模型

7.1 移动边界与材料坐标

问题一至问题三均假设药材半径 R 为常数。然而在实际烘干过程中，药材因水分流失会发生明显的体积收缩。此时计算区域不再是固定的 $0 \leq r \leq R$ ，而是随时间变化的移动边界区域 $0 \leq r \leq R(t)$ 。问题四使用附件 2 给出的半径数据 $R(t)$ ，并假设药材沿径向均匀收缩。为将移动边界问题转化为固定区域上的问题，定义材料坐标

$$\xi = \frac{r}{R(t)}, \quad 0 \leq \xi \leq 1. \quad (56)$$

固定 ξ 表示随骨架运动的同一材料位置。对任意场变量 $u(r, t) = \tilde{u}(\xi, t)$ ，由链式法则有

$$\left. \frac{\partial u}{\partial r} \right|_t = \frac{1}{R(t)} \frac{\partial \tilde{u}}{\partial \xi}. \quad (57)$$

在该材料坐标描述下，沿材料点的时间变化率即为 $\partial \tilde{u} / \partial t|_{\xi}$ 。

附件 2 的半径数据先由 PCHIP 插值到计算网格 $t_n = n\Delta t$ ，得到 $R_n = R(t_n)$ 。程序再作累计最小值修正，仅消除浮点误差造成的局部极小上升，从而保证 $R_{n+1} \leq R_n$ 。

7.2 变半径控制方程与边界条件

问题四使用附录 4 给出的物性关系：

$$\rho(C) = 760 + 90C, \quad (58)$$

$$c_p(C) = 1850 + 2150 \frac{C}{C+1}, \quad (59)$$

$$k(C) = 0.12 + 0.20 \frac{C}{C+1}, \quad (60)$$

$$D(C, T) = 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C}\right) \exp\left(-\frac{3850}{T_K}\right), \quad (61)$$

其中 $T_K = T + 273.15$ 。

将材料坐标变换代入统一建模框架中的守恒方程，可得固定材料坐标下的控制方程组：

$$\rho(\tilde{C})c_p(\tilde{C})\frac{\partial \tilde{T}}{\partial t} = \frac{1}{R(t)^2\xi}\frac{\partial}{\partial \xi}\left[\xi k(\tilde{C})\frac{\partial \tilde{T}}{\partial \xi}\right], \quad (62)$$

$$\frac{\partial \tilde{C}}{\partial t} = \frac{1}{R(t)^2\xi}\frac{\partial}{\partial \xi}\left[\xi D(\tilde{C}, \tilde{T})\frac{\partial \tilde{C}}{\partial \xi}\right]. \quad (63)$$

初始条件为

$$\tilde{T}(\xi, 0) = 28^\circ\text{C}, \quad \tilde{C}(\xi, 0) = 2.55 \text{ kg/kg}, \quad 0 \leq \xi \leq 1. \quad (64)$$

中心对称条件为

$$\frac{\partial \tilde{T}}{\partial \xi}(0, t) = 0, \quad \frac{\partial \tilde{C}}{\partial \xi}(0, t) = 0. \quad (65)$$

在当前表面 $\xi = 1$ 处，Robin 边界条件为

$$-\frac{k(\tilde{C}_s)}{R(t)}\frac{\partial \tilde{T}}{\partial \xi}(1, t) = h\left[\tilde{T}_s - T_\infty(t)\right], \quad (66)$$

$$-\frac{D(\tilde{C}_s, \tilde{T}_s)}{R(t)}\frac{\partial \tilde{C}}{\partial \xi}(1, t) = h_m\left[\tilde{C}_s - C_\infty(t)\right], \quad (67)$$

其中 $\tilde{T}_s = \tilde{T}(1, t)$, $\tilde{C}_s = \tilde{C}(1, t)$ 。

7.3 固定材料坐标控制体离散

空间网格取 $\xi_i = i\Delta\xi$, $\Delta\xi = 0.05$, $N_r = 21$ ；时间步长 $\Delta t = 60 \text{ s}$ 。单位化后的控制体权重为

$$w_0 = \left(\frac{\Delta\xi}{2}\right)^2, \quad w_i = 2\xi_i\Delta\xi \quad (1 \leq i \leq N_r - 2), \quad w_{N_r-1} = 1 - \left(1 - \frac{\Delta\xi}{2}\right)^2. \quad (68)$$

为统一表示温度 and 水分，令 u 分别取 $\tilde{T}$ 或 $\tilde{C}$ ， H_i^n 分别取 $\rho(C_i^n)c_p(C_i^n)$ 或 1， $q_{i+1/2}^n$ 分别取 k 或 D 。界面位置与界面系数为

$$\xi_{i+1/2} = \left(i + \frac{1}{2}\right) \Delta\xi, \quad (69)$$

$$q_{i+1/2}^n = \begin{cases} k \left(\frac{C_i^n + C_{i+1}^n}{2} \right), & u = \tilde{T}, \\ D \left(\frac{C_i^n + C_{i+1}^n}{2}, \frac{T_i^n + T_{i+1}^n}{2} \right), & u = \tilde{C}. \end{cases} \quad (70)$$

每个时间步使用 R_{n+1} 进入尺度因子与表面系数。对含有左右界面的控制体，后向 Euler 离散为

$$\begin{aligned} \frac{H_i^n w_i}{\Delta t} (u_i^{n+1} - u_i^n) = \frac{2}{R_{n+1}^2} & \left[\xi_{i+1/2} q_{i+1/2}^n \frac{u_{i+1}^{n+1} - u_i^{n+1}}{\Delta \xi} \right. \\ & \left. - \xi_{i-1/2} q_{i-1/2}^n \frac{u_i^{n+1} - u_{i-1}^{n+1}}{\Delta \xi} \right]. \end{aligned} \quad (71)$$

中心控制体没有左界面通量，表面控制体没有右界面通量，且表面控制体额外增加对流项：

$$\begin{aligned} \frac{H_s^n w_s}{\Delta t} (u_s^{n+1} - u_s^n) = & - \frac{2}{R_{n+1}} \xi_{s-1/2} q_{s-1/2}^n \frac{u_s^{n+1} - u_{s-1}^{n+1}}{\Delta \xi} \\ & + \frac{2h_u}{R_{n+1}} (u_{\text{air}}^{n+1} - u_s^{n+1}), \end{aligned} \quad (72)$$

其中温度方程取 $(h_u, u_{\text{air}}) = (h, T_\infty)$ ，水分方程取 $(h_u, u_{\text{air}}) = (h_m, C_\infty)$ 。

上述方程逐控制体组装为三对角方程组，并使用追赶法求解。

7.4 求解顺序

问题四采用系数滞后的后向 Euler 半隐式格式，每个时间步的计算流程如图 16 所示。流程呈顺序结构，每步的输入均来自上一时刻的已知量，时间步内不进行非线性迭代。

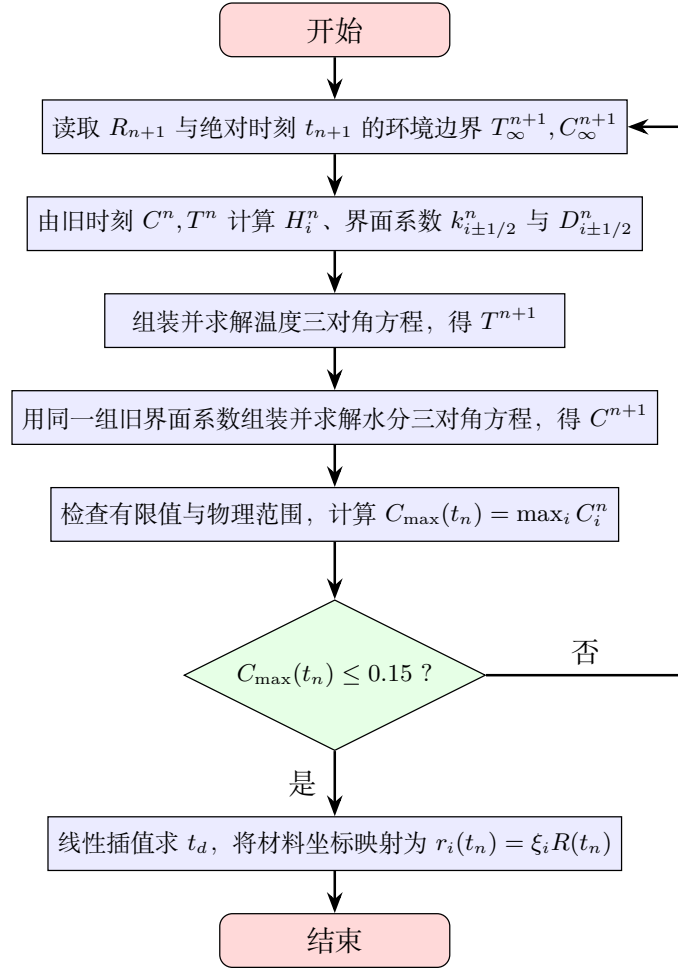


图 16 算法求解顺序图

补充说明三点：其一，半径数据经 PCHIP 插值后作累计最小值修正，以保证 $R_{n+1} \leq R_n$ ；其二，物性参数与界面系数均取旧时刻值，温度与水分方程共用，且步内不迭代；其三，输出位置按 $r_i(t_n) = \xi_i R(t_n)$ 映射，超出当前半径则留空，并保存无量纲网格与半径插值表。

整个时间推进过程中，各步均为显式物性更新与线性方程组求解，不涉及迭代与反解，因此计算效率较高、实现简洁。

7.5 求解结果

按全域水分阈值判定，本问得到烘干结束时间

$$t_d = 184815.7394 \text{ s} = 51.33771 \text{ h}, \quad (73)$$

此时药材半径由初始的 $R(0) = 2.000 \text{ cm}$ 收缩至

$$R(t_d) = 1.198 \text{ cm}. \quad (74)$$

表 6 药材烘干过程的水分浓度

时间/h	到药材中心的距离/cm			
	0	0.5	1	药材表面
6	1.7182	1.536	1.0211	0.4205
12	0.7377	0.6545	0.4079	0.1671
18	0.4088	0.3686	0.2394	0.0891
...				
烘干结束时间	0.15	0.1413	0.1085	0.0526

与问题三在固定半径假设下得到的 58.51 h 相比，考虑尺寸收缩后所需的烘干时间缩短至约 51.34 h，减小了约 12%。这一差异表明，药材收缩会缩短水分迁移路径并增大表面对流传质面积，从而加速干燥过程。因此，在实际烘干工艺中若忽略尺寸变化，会高估所需烘干时长，影响生产效率与能耗评估的准确性。最终结果按照表 6 的格式，给出每隔 6 h、距药材中心每隔 0.5 cm 的水分浓度，以及药材内部水分浓度每隔 60 s、距药材中心每隔 0.1 cm 的完整结果。

八、模型的分析与检验

8.1 结果检验

8.2 数值算法的守恒性验证

为验证上述显式有限差分算法求解结果的可靠性，本文分别从全局能量守恒和质量守恒两个维度，对数值解进行守恒性检验。

8.2.1 能量守恒检验

在连续域中，考虑半径 R 、单位长度的圆柱状物料，其温度场满足柱坐标热传导方程：

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(kr \frac{\partial T}{\partial r} \right), \quad 0 < r < R \quad (75)$$

表面 $r = R$ 处为第三类对流换热边界：

$$-k \frac{\partial T}{\partial r} \Big|_{r=R} = h (T_\infty(t) - T(R, t)) \quad (76)$$

将控制方程在 $[0, R]$ 上积分并代入边界条件, 可得全局能量平衡方程。在 $[0, t]$ 上进一步积分, 可推导得总能量变化等于边界热流累积:

$$\underbrace{Q(t) - Q(0)}_{\text{总能量变化}} = \underbrace{\int_0^t 2\pi R h (T_\infty(\tau) - T(R, \tau)) d\tau}_{\text{边界热流累积}} \quad (77)$$

其中, 总能量定义为 $Q(t) = \int_0^R \rho c_p T(r, t) \cdot 2\pi r dr$ 。

在离散域中, 取节点 $r_i = i \cdot \Delta r$, 对应控制体体积 (单位长度) 分别为 $V_0 = \pi(\Delta r/2)^2$, $V_i = 2\pi r_i \Delta r$ ($1 \leq i \leq N_r - 2$), $V_{N_r-1} = \pi[R^2 - (R - \Delta r/2)^2]$ 。离散总能量计算式为:

$$Q^n = \sum_{i=0}^{N_r-1} \rho c_p T_i^n V_i \quad (78)$$

边界热流累积为:

$$Q_{\text{flux}}^n = \sum_{m=0}^{n-1} h \cdot 2\pi R (T_\infty^m - T_{N_r-1}^m) \Delta t \quad (79)$$

定义能量守恒相对误差为:

$$\varepsilon_Q^n = \frac{|(Q^n - Q^0) - Q_{\text{flux}}^n|}{\max(|Q^n - Q^0|, \epsilon)} \quad (80)$$

其中, ϵ 为防止分母为零的小量, 本文取 $\epsilon = 10^{-12}$ 。

8.2.2 水分质量守恒检验

在连续域中, 水分浓度场满足柱坐标变系数扩散方程:

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C) \frac{\partial C}{\partial r} \right), \quad 0 < r < R \quad (81)$$

其中 $D(C) = 7 \times 10^{-9} \exp(-\frac{0.89}{C})$ 为随浓度变化的扩散系数。表面 $r = R$ 处为第三类对流传质边界:

$$-D(C) \frac{\partial C}{\partial r} \Big|_{r=R} = h_m (C_\infty(t) - C(R, t)) \quad (82)$$

同理, 将控制方程积分并代入边界条件, 可得总水分变化等于边界质流累积:

$$\underbrace{M(t) - M(0)}_{\text{总水分变化}} = \underbrace{\int_0^t 2\pi R h_m (C_\infty(\tau) - C(R, \tau)) d\tau}_{\text{边界质流累积}} \quad (83)$$

其中, 总水分定义为 $M(t) = \int_0^R C(r, t) \cdot 2\pi r dr$ 。

在离散域中, 离散总水分计算式为 $M^n = \sum_{i=0}^{N_r-1} C_i^n V_i$, 边界质流累积为 $M_{\text{flux}}^n = \sum_{m=0}^{n-1} m^m \Delta t$, 其中表面质流 $m^n = h_m \cdot 2\pi R (C_\infty^n - C_{N_r-1}^n)$ 。同理可定义水分守恒相对误差 ε_M^n 。

8.2.3 变系数扩散的守恒性说明

水分方程求解的关键难点在于 $D(C)$ 是变量。若直接将 D 取节点值 $D(C_i)$ 进行常规差分, 则会漏掉 $\partial D / \partial r \cdot \partial C / \partial r$ 项, 导致相邻控制体界面通量不相等, 从而破坏总水分守恒。

为克服这一问题, 本文采用有限体积法思想, 界面扩散系数取左右节点算术平均:

$$D_{i+1/2} = D \left(\frac{C_i + C_{i+1}}{2} \right) \quad (84)$$

界面通量表示为:

$$F_{i+1/2} = D_{i+1/2} \frac{C_{i+1} - C_i}{\Delta r} \quad (85)$$

此时, 相邻控制体界面通量大小相等、方向相反, 求和时自动抵消, 从而在数学上保证了总水分严格守恒。

8.3 误差分析

九、模型的评价

9.1 模型的优点

- 优点 1
- 优点 2
- 优点 3

9.2 模型的推广与改进

9.3 模型的缺点

- 缺点 1
- 缺点 2

附录 A 文件列表

文件名	功能描述
q1_1.py, q1_2.py, q1_3.py	问题一：边界插值、显式差分与结果整理
q2_1.py, q2_2.py, q2_3.py	问题二：恒温段边界、求解器与检验
q3.py	问题三：全过程固定半径求解
q4.py	问题四：变半径材料坐标求解
validate_all.py	全部问题的交叉验证

附录 B 代码

2.1 问题一程序

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from scipy.interpolate import PchipInterpolator
5 from pathlib import Path
6
7 # 文件路径
8 BASE_DIR = Path(__file__).resolve().parent
9 input_file = BASE_DIR / 'input_excel' / '附件1.xlsx'
10 output_file = BASE_DIR / 'output_excel' / 'output1.xlsx'
11 output_file.parent.mkdir(parents=True, exist_ok=True)
12
13 # 读取Excel (前三列: 时间, 温度, 水分浓度)
14 df = pd.read_excel(input_file, header=0)
15 t_orig = df.iloc[:, 0].values.astype(float)
16 temp_orig = df.iloc[:, 1].values.astype(float)
17 moist_orig = df.iloc[:, 2].values.astype(float)
18
19 # 生成步长为1s的时间序列
20 t_new = np.arange(t_orig.min(), t_orig.max() + 0.1, 1.0)
21
22 # 保形插值, 避免过冲
23 temp_new = PchipInterpolator(t_orig, temp_orig)(t_new)
24 moist_new = PchipInterpolator(t_orig, moist_orig)(t_new)
25
26 # 保存结果
27 pd.DataFrame({
28     '时间': t_new,
29     '温度': temp_new,
30     '水分浓度': moist_new
31 }).to_excel(output_file, index=False)
32
33 print(f"完成, 已保存至 {output_file}")
34
35 # 画温度图
36 plt.figure()
37 plt.plot(t_new, temp_new, label='插值数据 (1s)')
38 plt.plot(t_orig, temp_orig, 'o', label='原始数据 (60s)')
39 plt.xlabel('时间 (s)')
40 plt.ylabel('温度')
41 plt.legend()
42 plt.show()
43
44 # 画水分浓度图
45 plt.figure()
46 plt.plot(t_new, moist_new, label='插值数据 (1s)')
47 plt.plot(t_orig, moist_orig, 'o', label='原始数据 (60s)')
48 plt.xlabel('时间 (s)')
49 plt.ylabel('水分浓度')
50 plt.legend()
51 plt.show()
```

Listing 1: 问题一边界插值程序

```

1 import pandas as pd
2 import numpy as np
3 import os
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from pathlib import Path
7
8 BASE_DIR = Path(__file__).resolve().parent
9 OUTPUT_DIR = BASE_DIR / 'output_excel'
10 PIC_DIR = BASE_DIR / 'pic'
11
12 # 读取输入数据: 第一列时间, 第二列环境温度T_air, 第三列环境湿度C_air
13 file_name = OUTPUT_DIR / 'output1.xlsx'
14 if not file_name.exists():
15     raise FileNotFoundError(f"未找到边界文件: {file_name}, 请先运行 q1_1.py")
16 df_input = pd.read_excel(file_name)
17 t_array = df_input.iloc[:, 0].values.astype(float)
18 T_air = df_input.iloc[:, 1].values.astype(float)
19 C_air = df_input.iloc[:, 2].values.astype(float)
20 if len(t_array) < 1801:
21     raise ValueError("边界文件至少需要覆盖 0~1800 s")
22
23 # 物理参数
24 rho, Cp, k = 820.0, 2600.0, 0.36
25 h, hm = 25.0, 8e-7
26 alpha = k / (rho * Cp)
27 Nt, Nr = 1801, 21
28 dt, dr = 1.0, 0.001
29 r_grid = np.linspace(0, 2, Nr)
30
31 T_matrix, C_matrix = np.zeros((Nt, Nr)), np.zeros((Nt, Nr))
32 T_matrix[0, :] = 28.0
33 C_matrix[0, :] = 2.55
34
35 # 水分扩散系数函数
36 def D_of_C(C_val):
37     C_val = np.clip(C_val, 1e-5, 10.0)
38     return 7e-9 * np.exp(-0.89 / C_val)
39
40 # 界面扩散系数: 用左右节点 C 的算术平均求 D
41 def D_face(C_left, C_right):
42     return D_of_C(0.5 * (C_left + C_right))
43
44 print("开始求解问题...")
45
46 for n in range(0, Nt - 1):
47     T_air_next, C_air_next = T_air[n+1], C_air[n+1]
48
49     for i in range(1, Nr - 1):
50         r_val = i * dr
51         T_matrix[n+1, i] = T_matrix[n, i] + alpha * dt / (dr**2) * (
52             (1 + dr/(2*r_val)) * T_matrix[n, i+1]
53             - 2 * T_matrix[n, i]
54             + (1 - dr/(2*r_val)) * T_matrix[n, i-1])
55
56     T_matrix[n+1, 0] = T_matrix[n, 0] + 4 * alpha * dt / (dr**2) * (T_matrix[n, 1] - T_matrix[n, 0])
57
58 # 表面温度: 控制体内导热与外部对流
59 i_s = Nr - 1
60 V_factor = i_s - 0.25
61 T_matrix[n+1, i_s] = T_matrix[n, i_s] + dt * (
62     2 * (i_s - 0.5) * k * (T_matrix[n, i_s - 1] - T_matrix[n, i_s])
63     / (rho * Cp * V_factor * dr**2)
64     + 2 * i_s * h * (T_air_next - T_matrix[n, i_s])
65     / (rho * Cp * V_factor * dr)
66 )
67
68 # 内部节点 i = 1 ... Nr-2
69 for i in range(1, Nr - 1):
70     r_val = i * dr
71     # 左界面 D_{i-1/2}, 右界面 D_{i+1/2}
72     D_L = D_face(C_matrix[n, i-1], C_matrix[n, i])
73     D_R = D_face(C_matrix[n, i], C_matrix[n, i+1])
74     # 通量形式: dC/dt = [ (i+0.5)/i * D_R*(C_{i+1}-C_i) + (i-0.5)/i * D_L*(C_{i-1}-C_i) ] / dr^2
75     C_matrix[n+1, i] = C_matrix[n, i] + dt / (dr**2) * (
76         (1 + dr/(2*r_val)) * D_R * (C_matrix[n, i+1] - C_matrix[n, i])
77         + (1 - dr/(2*r_val)) * D_L * (C_matrix[n, i-1] - C_matrix[n, i])
78     )
79
80 # 中心节点 i = 0: D 用界面 D_{1/2}
81 D_01 = D_face(C_matrix[n, 0], C_matrix[n, 1])
82 C_matrix[n+1, 0] = C_matrix[n, 0] + 4 * D_01 * dt / (dr**2) * (C_matrix[n, 1] - C_matrix[n, 0])
83
84 # 表面节点: 有限体积
85 i_s = Nr - 1
86 D_sm = D_face(C_matrix[n, i_s-1], C_matrix[n, i_s]) # 界面 D_{N-3/2}

```

```

86     V_factor = i_s - 0.25 # 表面控制体积 / (π dr²)
87     C_matrix[n+1, i_s] = C_matrix[n, i_s] + dt * (
88         2 * (i_s - 0.5) * D_sm * (C_matrix[n, i_s-1] - C_matrix[n, i_s]) / (V_factor * dr**2)
89         + 2 * i_s * hm * (C_air_next - C_matrix[n, i_s]) / (V_factor * dr)
90     )
91
92     print("问题一求解完成")
93
94     # 构建完整输出DataFrame
95     col_names = [f'{x:.1f}' for x in r_grid]
96     time_col = np.arange(0, 1801, 1)
97
98     df_q1_1 = pd.DataFrame(np.round(T_matrix, 4), columns=col_names)
99     df_q1_1.insert(0, '时间', time_col)
100
101     df_q1_2 = pd.DataFrame(np.round(C_matrix, 4), columns=col_names)
102     df_q1_2.insert(0, '时间', time_col)
103
104     # 提取特定时间及半径切片数据
105     target_times = [100, 300, 600, 900, 1200, 1500, 1800]
106     target_radii = ['0.0', '0.5', '1.0', '1.5', '2.0']
107
108     df_q1_3 = df_q1_1[df_q1_1['时间'].isin(target_times)][['时间'] + target_radii].reset_index(drop=True)
109     df_q1_4 = df_q1_2[df_q1_2['时间'].isin(target_times)][['时间'] + target_radii].reset_index(drop=True)
110
111     # 覆盖并保存四个Excel文件
112     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
113     for name, frame in {
114         'q1_1.xlsx': df_q1_1,
115         'q1_2.xlsx': df_q1_2,
116         'q1_3.xlsx': df_q1_3,
117         'q1_4.xlsx': df_q1_4,
118     }.items():
119         try:
120             frame.to_excel(OUTPUT_DIR / name, index=False)
121         except PermissionError:
122             pending = OUTPUT_DIR / name.replace('.xlsx', '_pending.xlsx')
123             frame.to_excel(pending, index=False)
124             print(f'文件被占用, 已写入临时结果: {pending}')
125
126     # 按题目格式保存双工作表文件
127     result1_file = OUTPUT_DIR / 'result1.xlsx'
128     try:
129         with pd.ExcelWriter(result1_file) as writer:
130             df_q1_1.to_excel(writer, sheet_name='温度', index=False)
131             df_q1_2.to_excel(writer, sheet_name='水分浓度', index=False)
132         print(f'题目正式结果已保存: {result1_file}')
133     except PermissionError:
134         pending = OUTPUT_DIR / 'result1_pending.xlsx'
135         with pd.ExcelWriter(pending) as writer:
136             df_q1_1.to_excel(writer, sheet_name='温度', index=False)
137             df_q1_2.to_excel(writer, sheet_name='水分浓度', index=False)
138         print(f'文件被占用, 已写入临时结果: {pending}')
139
140     print("处理完成! 已覆盖并生成 q1_1.xlsx, q1_2.xlsx, q1_3.xlsx, q1_4.xlsx")
141
142     plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'Arial Unicode MS', 'PingFang SC']
143     plt.rcParams['axes.unicode_minus'] = False # 解决负号显示为方块的问题
144
145     # 读取数据
146     df_T = pd.read_excel(OUTPUT_DIR / 'q1_1.xlsx')
147     df_C = pd.read_excel(OUTPUT_DIR / 'q1_2.xlsx')
148
149     time = df_T['时间'].values
150     radii = df_T.columns[1:].astype(float).values
151
152     # 空间分布图 (水分浓度)
153     plt.figure(figsize=(10, 6))
154     target_radii = [0.0, 0.5, 1.0, 1.5, 2.0]
155     for r in target_radii:
156         plt.plot(time, df_C[f'{r:.1f}'].values, label=f'r = {r} cm')
157     plt.xlabel('时间 t (s)', fontsize=12)
158     plt.ylabel('水分浓度 C (kg/kg)', fontsize=12)
159     plt.legend()
160     plt.grid(True, alpha=0.3)
161     PIC_DIR.mkdir(parents=True, exist_ok=True)
162     plt.savefig(PIC_DIR / '1_水分时间演化图.png', dpi=300, bbox_inches='tight')
163     plt.show()
164     plt.close()
165
166     # 时间演化图 (温度)
167     plt.figure(figsize=(10, 6))
168     target_radii = [0.0, 0.5, 1.0, 1.5, 2.0]
169     for r in target_radii:
170         plt.plot(time, df_T[f'{r:.1f}'].values, label=f'r = {r} cm')

```

```

171 plt.xlabel('时间 t (s)', fontsize=12)
172 plt.ylabel('温度 T (°C)', fontsize=12)
173 # plt.title('不同半径位置药材温度随烘干时间的变化', fontsize=14)
174 plt.legend()
175 plt.grid(True, alpha=0.3)
176 plt.savefig(PIC_DIR / '2_温度时间演化图.png', dpi=300, bbox_inches='tight')
177 plt.show()
178 plt.close()
179
180 # 二维热力图 (温度与水分)
181 fig, axes = plt.subplots(1, 2, figsize=(16, 5))
182
183 # 温度热力图 温度随时间与空间的二维分布
184 sns.heatmap(df.T.iloc[:, 1:].T, cmap='YlOrRd', cbar_kws={'label': '温度 (°C)'}, ax=axes[0],
185             yticklabels=np.round(radrii, 1))
186 axes[0].set_xlabel('时间 t (s)')
187 axes[0].set_ylabel('半径 r (cm)')
188 axes[0].set_xticks(np.linspace(0, 1800, 7))
189 axes[0].set_xticklabels([int(x) for x in np.linspace(0, 1800, 7)])
190
191 # 水分热力图
192 sns.heatmap(df.C.iloc[:, 1:].T, cmap='Blues_r', cbar_kws={'label': '水分浓度 (kg/kg)'}, ax=axes[1],
193             yticklabels=np.round(radrii, 1))
194 axes[1].set_xlabel('时间 t (s)')
195 axes[1].set_ylabel('半径 r (cm)')
196 axes[1].set_xticks(np.linspace(0, 1800, 7))
197 axes[1].set_xticklabels([int(x) for x in np.linspace(0, 1800, 7)])
198
199 plt.tight_layout()
200 plt.savefig(PIC_DIR / '3_温度和水分热力图.png', dpi=300, bbox_inches='tight')
201 plt.show()
202 plt.close()
203
204 print("图表已生成并保存到当前文件夹!")

```

Listing 2: 问题一显式差分求解程序

```

1 # 1. 时间步收敛 2. 空间步长收敛 3. 守恒检查
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from scipy.interpolate import PchipInterpolator
6 from pathlib import Path
7
8 BASE_DIR = Path(__file__).resolve().parent
9 OUTPUT_DIR = BASE_DIR / 'output_excel'
10 PIC_DIR = BASE_DIR / 'pic'
11
12 plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei',
13                                   'Arial Unicode MS', 'PingFang SC']
14 plt.rcParams['axes.unicode_minus'] = False
15
16
17 # 读取边界条件, 返回 T_air(t), C_air(t)
18 def get_boundary_funcs():
19     path = OUTPUT_DIR / 'output1.xlsx'
20     df = pd.read_excel(path)
21     t_orig = df.iloc[:, 0].values.astype(float)
22     T_orig = df.iloc[:, 1].values.astype(float)
23     C_orig = df.iloc[:, 2].values.astype(float)
24     if np.any(np.diff(t_orig) <= 0):
25         raise ValueError('output1.xlsx 时间必须严格递增')
26     T_func = PchipInterpolator(t_orig, T_orig, extrapolate=False)
27     C_func = PchipInterpolator(t_orig, C_orig, extrapolate=False)
28     print(f'已读取 {path} 作为边界条件')
29     return T_func, C_func
30
31 # 求解 PDE, 返回 t, r, T, C
32 def solve_pde(dt, dr, r_max=0.02, t_end=1800.0,
33              T_air_func=None, C_air_func=None):
34     if T_air_func is None or C_air_func is None:
35         T_air_func, C_air_func = get_boundary_funcs()
36
37     rho, Cp, k = 820.0, 2600.0, 0.36
38     h, hm = 25.0, 8e-7
39     alpha = k / (rho * Cp)
40
41     Nr = int(round(r_max / dr)) + 1
42     Nt = int(round(t_end / dt)) + 1
43
44     r = np.linspace(0, r_max, Nr)
45     t = np.linspace(0, t_end, Nt)
46
47     T = np.zeros((Nt, Nr))

```

```

48 C = np.zeros((Nt, Nr))
49 T[0, :] = 28.0
50 C[0, :] = 2.55
51
52 r_inner = r[1:-1]
53 coef_p = 1.0 + dr / (2.0 * r_inner) # r_{i+1/2}/r_i
54 coef_m = 1.0 - dr / (2.0 * r_inner) # r_{i-1/2}/r_i
55
56 # 水分扩散系数
57 def D_of_C(Cval):
58     return 7e-9 * np.exp(-0.89 / np.clip(Cval, 1e-5, 10.0))
59
60 stab_T = 4.0 * alpha * dt / dr**2
61 if stab_T >= 1.0:
62     print(f' [!] dt={dt}, dr={dr} 时  $4\alpha \cdot dt/dr^2 = \{stab\_T:.3f\} \geq 1$ , 可能不稳定')
63
64 for n in range(Nt - 1):
65     T_air_next = float(T_air_func(t[n + 1]))
66     C_air_next = float(C_air_func(t[n + 1]))
67
68     # 温度: 内部节点 (不变)
69     T[n + 1, 1:-1] = T[n, 1:-1] + alpha * dt / dr**2 * (
70         coef_p * T[n, 2:] - 2.0 * T[n, 1:-1] + coef_m * T[n, :-2]
71     )
72
73     # 水分: 内部节点 (有限体积, D 用界面值)
74     C_inner = C[n, 1:-1]
75     C_left = C[n, :-2]
76     C_right = C[n, 2:]
77     D_L = D_of_C(0.5 * (C_left + C_inner)) # D_{i-1/2}
78     D_R = D_of_C(0.5 * (C_inner + C_right)) # D_{i+1/2}
79     C[n + 1, 1:-1] = C_inner + dt / dr**2 * (
80         coef_p * D_R * (C_right - C_inner)
81         + coef_m * D_L * (C_left - C_inner)
82     )
83
84     # 温度: 中心节点 (不变)
85     T[n + 1, 0] = T[n, 0] + 4.0 * alpha * dt / dr**2 * (T[n, 1] - T[n, 0])
86
87     # 水分: 中心节点 (D 用界面 D_{1/2})
88     D_01 = D_of_C(0.5 * (C[n, 0] + C[n, 1]))
89     C[n + 1, 0] = C[n, 0] + 4.0 * D_01 * dt / dr**2 * (C[n, 1] - C[n, 0])
90
91     # 温度: 表面控制体
92     i_s = Nr - 1
93     V_factor = i_s - 0.25
94     T[n + 1, -1] = T[n, -1] + dt * (
95         2.0 * (i_s - 0.5) * k * (T[n, i_s - 1] - T[n, i_s])
96         / (rho * Cp * V_factor * dr**2)
97         + 2.0 * i_s * h * (T_air_next - T[n, i_s])
98         / (rho * Cp * V_factor * dr)
99     )
100
101     # 水分: 表面节点 (有限体积, 严格守恒)
102     i_s = Nr - 1
103     D_sm = D_of_C(0.5 * (C[n, i_s - 1] + C[n, i_s])) # D_{N-3/2}
104     V_factor = i_s - 0.25 # 表面控制体体积 / ( $\pi dr^2$ )
105     C[n + 1, -1] = C[n, -1] + dt * (
106         2.0 * (i_s - 0.5) * D_sm * (C[n, i_s - 1] - C[n, -1]) / (V_factor * dr**2)
107         + 2.0 * i_s * hm * (C_air_next - C[n, -1]) / (V_factor * dr)
108     )
109
110     return t, r, T, C
111
112 # 验证1: 时间步收敛
113 def verify_time_step():
114     print('\n时间步收敛验证')
115
116     T_func, C_func = get_boundary_funcs()
117     dr = 0.001
118     dt_list = [1.0, 0.5, 0.25]
119     results = {}
120
121     for dt in dt_list:
122         print(f' 求解 dt = {dt} s ...')
123         t, r, T, C = solve_pde(dt, dr, T_air_func=T_func, C_air_func=C_func)
124         results[dt] = (t, r, T, C)
125
126     # 共同时间点比较
127     common_t = np.arange(0, 1801, 100)
128     T_common, C_common = {}, {}
129     for dt in dt_list:
130         t, r, T, C = results[dt]
131         idx = np.round(common_t / dt).astype(int)
132         T_common[dt] = T[idx, :]

```

```

133     C_common[dt] = C[idx, :]
134
135     err_T_1_05 = np.max(np.abs(T_common[1.0] - T_common[0.5]))
136     err_T_05_025 = np.max(np.abs(T_common[0.5] - T_common[0.25]))
137     err_C_1_05 = np.max(np.abs(C_common[1.0] - C_common[0.5]))
138     err_C_05_025 = np.max(np.abs(C_common[0.5] - C_common[0.25]))
139
140     print('\n误差结果: ')
141     print(f' 温度 dt=1.0 vs 0.5 : {err_T_1_05:.6f} °C')
142     print(f' 温度 dt=0.5 vs 0.25 : {err_T_05_025:.6f} °C')
143     print(f' 水分 dt=1.0 vs 0.5 : {err_C_1_05:.6f}')
144     print(f' 水分 dt=0.5 vs 0.25 : {err_C_05_025:.6f}')
145     print('\n判断标准: ')
146     print(' 1) 第二次误差应小于第一次')
147     print(' 2) 收敛比应接近 0.5 (一阶)')
148     print(f' 温度收敛比 = {err_T_05_025/err_T_1_05:.3f}')
149     print(f' 水分收敛比 = {err_C_05_025/err_C_1_05:.3f}')
150     print(' 3) 温度误差 < 0.1°C, 水分误差 < 0.01')
151     ratio_T = err_T_05_025 / err_T_1_05
152     ratio_C = err_C_05_025 / err_C_1_05
153     if not (err_T_05_025 < err_T_1_05 and err_C_05_025 < err_C_1_05
154            and 0.35 < ratio_T < 0.65 and 0.35 < ratio_C < 0.65
155            and err_T_1_05 < 0.1 and err_C_1_05 < 0.01):
156         raise AssertionError('问题一时间步收敛未通过')
157
158     fig, axes = plt.subplots(1, 3, figsize=(18, 5))
159
160     for dt in dt_list:
161         t, r, T, C = results[dt]
162         axes[0].plot(t, T[:, 0], label=f'dt={dt}s')
163         axes[1].plot(t, C[:, -1], label=f'dt={dt}s')
164
165     axes[0].set_xlabel('时间 t (s)')
166     axes[0].set_ylabel('中心温度 T (°C)')
167     axes[0].set_title('中心温度随时间变化 (不同 dt)')
168     axes[0].legend()
169     axes[0].grid(alpha=0.3)
170
171     axes[1].set_xlabel('时间 t (s)')
172     axes[1].set_ylabel('表面水分 C')
173     axes[1].set_title('表面水分随时间变化 (不同 dt)')
174     axes[1].legend()
175     axes[1].grid(alpha=0.3)
176
177     dt_err = np.array([0.5, 0.25])
178     err_T = np.array([err_T_1_05, err_T_05_025])
179     err_C = np.array([err_C_1_05, err_C_05_025])
180     axes[2].loglog(dt_err, err_T, 'o-', label='温度误差')
181     axes[2].loglog(dt_err, err_C, 's-', label='水分误差')
182     axes[2].set_xlabel('dt (s)')
183     axes[2].set_ylabel('最大绝对误差')
184     axes[2].set_title('时间步收敛误差 (log-log)')
185     axes[2].legend()
186     axes[2].grid(alpha=0.3, which='both')
187
188     plt.tight_layout()
189     plt.savefig(PIC_DIR / '验证_时间步收敛.png', dpi=300, bbox_inches='tight')
190     plt.show()
191     plt.close()
192
193
194 # 验证2: 空间步收敛
195 def verify_space_step():
196     print('\n空间步收敛验证')
197
198     T_func, C_func = get_boundary_funcs()
199     dt = 0.05
200     dr_list = [0.001, 0.0005, 0.00025]
201     results = {}
202
203     for dr in dr_list:
204         print(f' 求解 dr = {dr} m ...')
205         t, r, T, C = solve_pde(dt, dr, T_func=T_func, C_func=C_func)
206         results[dr] = (t, r, T, C)
207
208     # 共同半径点比较
209     common_r = np.array([0.0, 0.005, 0.01, 0.015, 0.02])
210     T_final, C_final = {}, {}
211     for dr in dr_list:
212         t, r, T, C = results[dr]
213         T_final[dr] = np.interp(common_r, r, T[-1, :])
214         C_final[dr] = np.interp(common_r, r, C[-1, :])
215
216     err_T_1_05 = np.max(np.abs(T_final[0.001] - T_final[0.0005]))
217     err_T_05_025 = np.max(np.abs(T_final[0.0005] - T_final[0.00025]))

```

```

218 err_C_1_05 = np.max(np.abs(C_final[0.001] - C_final[0.0005]))
219 err_C_05_025 = np.max(np.abs(C_final[0.0005] - C_final[0.00025]))
220
221 print('\n误差结果: ')
222 print(f' 温度 dr=1mm vs 0.5mm : {err_T_1_05:.6f} °C')
223 print(f' 温度 dr=0.5mm vs 0.25mm : {err_T_05_025:.6f} °C')
224 print(f' 水分 dr=1mm vs 0.5mm : {err_C_1_05:.6f}')
225 print(f' 水分 dr=0.5mm vs 0.25mm : {err_C_05_025:.6f}')
226 print('\n判断标准: ')
227 print(' 1) 第二次误差应小于第一次')
228 print(' 2) 收敛比应接近 0.25 (二阶)')
229 print(f' 温度收敛比 = {err_T_05_025/err_T_1_05:.3f}')
230 print(f' 水分收敛比 = {err_C_05_025/err_C_1_05:.3f}')
231 print(' 3) 温度差异 < 0.05°C, 水分差异 < 0.005')
232 ratio_T = err_T_05_025 / err_T_1_05
233 ratio_C = err_C_05_025 / err_C_1_05
234 if not (err_T_05_025 < err_T_1_05 and err_C_05_025 < err_C_1_05
235         and 0.15 < ratio_T < 0.35 and 0.15 < ratio_C < 0.35
236         and err_T_1_05 < 0.05 and err_C_1_05 < 0.005):
237     raise AssertionError('问题-空间步收敛未通过')
238
239 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
240
241 for dr in dr_list:
242     t, r, T, C = results[dr]
243     axes[0].plot(r * 100, T[-1, :], 'o-', label=f'dr={dr*1000:.2f}mm')
244     axes[1].plot(r * 100, C[-1, :], 'o-', label=f'dr={dr*1000:.2f}mm')
245
246 axes[0].set_xlabel('半径 r (cm)')
247 axes[0].set_ylabel('最终温度 T (°C)')
248 axes[0].set_title('最终温度沿半径分布 (不同 dr)')
249 axes[0].legend()
250 axes[0].grid(alpha=0.3)
251
252 axes[1].set_xlabel('半径 r (cm)')
253 axes[1].set_ylabel('最终水分 C')
254 axes[1].set_title('最终水分沿半径分布 (不同 dr)')
255 axes[1].legend()
256 axes[1].grid(alpha=0.3)
257
258 dr_err = np.array([0.0005, 0.00025])
259 err_T = np.array([err_T_1_05, err_T_05_025])
260 err_C = np.array([err_C_1_05, err_C_05_025])
261 axes[2].loglog(dr_err, err_T, 'o-', label='温度误差')
262 axes[2].loglog(dr_err, err_C, 's-', label='水分误差')
263 axes[2].set_xlabel('dr (m)')
264 axes[2].set_ylabel('最大绝对误差')
265 axes[2].set_title('空间步长收敛误差 (log-log)')
266 axes[2].legend()
267 axes[2].grid(alpha=0.3, which='both')
268
269 plt.tight_layout()
270 plt.savefig(PIC_DIR / '验证_空间步长收敛.png', dpi=300, bbox_inches='tight')
271 plt.show()
272 plt.close()
273
274
275 # 验证3: 守恒检查
276 def verify_conservation():
277     print('\n守恒检查')
278
279     T_func, C_func = get_boundary_funcs()
280     dt = 1.0
281     dr = 0.001
282     r_max = 0.02
283     t_end = 1800.0
284
285     t, r, T, C = solve_pde(dt, dr, r_max, t_end, T_func, C_func)
286     Nt, Nr = T.shape
287
288     rho, Cp = 820.0, 2600.0
289     h, hm = 25.0, 8e-7
290
291     # 控制体体积
292     V = np.zeros(Nr)
293     V[0] = np.pi * (dr / 2.0)**2
294     for i in range(1, Nr - 1):
295         V[i] = 2.0 * np.pi * (i * dr) * dr
296     r_surf = (Nr - 1) * dr
297     V[-1] = np.pi * (r_surf**2 - (r_surf - dr / 2.0)**2)
298
299     # 总能量、总水分
300     Q = np.sum(rho * Cp * T * V, axis=1)
301     M = np.sum(C * V, axis=1)
302

```

```

303 A_surf = 2.0 * np.pi * r_surf
304 T_air_arr = np.array([float(T_func(tt)) for tt in t])
305 C_air_arr = np.array([float(C_func(tt)) for tt in t])
306
307 # 表面通量取 n+1 时刻环境值和 n 时刻表面值
308 q_step = h * A_surf * (T_air_arr[1:] - T[:-1, -1])
309 m_step = hm * A_surf * (C_air_arr[1:] - C[:-1, -1])
310 Q_flux_int = np.concatenate([0.0, np.cumsum(q_step * dt)])
311 M_flux_int = np.concatenate([0.0, np.cumsum(m_step * dt)])
312
313 Q_change = Q - Q[0]
314 M_change = M - M[0]
315
316 eps = 1e-12
317 rel_err_Q = np.abs(Q_change - Q_flux_int) / np.maximum(np.abs(Q_change), eps)
318 rel_err_M = np.abs(M_change - M_flux_int) / np.maximum(np.abs(M_change), eps)
319
320 print('\n误差结果:')
321 print(f' 最终能量守恒相对误差 : {rel_err_Q[-1]:.4%}')
322 print(f' 最终水分守恒相对误差 : {rel_err_M[-1]:.4%}')
323 print('\n判断标准:')
324 print(' 1) 两条累积曲线应基本重合')
325 print(' 2) 相对误差应 < 1%~5%')
326 print(' 3) 若误差快速累积, 检查边界离散或体积权重')
327 if rel_err_Q[-1] >= 0.01 or rel_err_M[-1] >= 0.01:
328     raise AssertionError('问题一守恒检验未通过')
329
330 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
331
332 axes[0].plot(t, Q_change, label='总能量变化')
333 axes[0].plot(t, Q_flux_int, '--', label='边界热流积分')
334 axes[0].set_xlabel('时间 t (s)')
335 axes[0].set_ylabel('能量变化 (J/m)')
336 axes[0].set_title('能量守恒检查')
337 axes[0].legend()
338 axes[0].grid(alpha=0.3)
339
340 axes[1].plot(t, M_change, label='总水分变化')
341 axes[1].plot(t, M_flux_int, '--', label='边界质流积分')
342 axes[1].set_xlabel('时间 t (s)')
343 axes[1].set_ylabel('水分变化')
344 axes[1].set_title('水分守恒检查')
345 axes[1].legend()
346 axes[1].grid(alpha=0.3)
347
348 axes[2].plot(t, rel_err_Q, label='能量相对误差')
349 axes[2].plot(t, rel_err_M, label='水分相对误差')
350 axes[2].set_xlabel('时间 t (s)')
351 axes[2].set_ylabel('相对误差')
352 axes[2].set_title('守恒相对误差')
353 axes[2].legend()
354 axes[2].grid(alpha=0.3)
355
356 plt.tight_layout()
357 plt.savefig(PIC_DIR / '验证_守恒检查.png', dpi=300, bbox_inches='tight')
358 plt.show()
359 plt.close()
360
361
362 if __name__ == '__main__':
363     verify_time_step()
364     verify_space_step()
365     verify_conservation()
366     print('\n全部验证完成, 已生成 3 张图。')

```

Listing 3: 问题一结果整理与绘图程序

2.2 问题二程序

```

1 # 问题二生成数据以及画图
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from pathlib import Path
6
7 plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'Arial Unicode MS']
8 plt.rcParams['axes.unicode_minus'] = False
9
10 BASE_DIR = Path(__file__).resolve().parent
11 input_path = BASE_DIR / 'output_excel' / 'output1.xlsx'

```

```

12 output_path = BASE_DIR / 'output_excel' / 'output2.xlsx'
13 plot_path = BASE_DIR / 'pic' / '验证_稳定时间.png'
14
15 # 读取数据
16 try:
17     df = pd.read_excel(input_path)
18 except FileNotFoundError:
19     print(f"未找到文件: {input_path}")
20     exit()
21
22 t = df.iloc[:, 0].to_numpy()
23 T = df.iloc[:, 1].to_numpy()
24 C = df.iloc[:, 2].to_numpy()
25
26 # 计算末尾稳定值 (取最后1000个点或后10%, 取较小值)
27 tail_n = min(1000, len(T) // 10)
28 if tail_n < 10:
29     tail_n = 10
30 T_stable = np.mean(T[-tail_n:])
31 C_stable = np.mean(C[-tail_n:])
32 print(f"计算得到稳定值: T = {T_stable:.4f}, C = {C_stable:.4f}")
33
34 # 划定5%误差带
35 tol_T = 0.05 * abs(T_stable)
36 tol_C = 0.05 * abs(C_stable)
37
38 # 判断每个时刻是否同时满足 (两者必须都在误差带内)
39 cond_T = np.abs(T - T_stable) <= tol_T
40 cond_C = np.abs(C - C_stable) <= tol_C
41 cond_both = cond_T & cond_C
42
43 # 从后往前找, 找到第一个不满足条件的位置, 其下一个位置即为稳定起点
44 idx_start = 0
45 for i in range(len(cond_both) - 1, -1, -1):
46     if not cond_both[i]:
47         idx_start = i + 1
48         break
49
50 # 统计稳定阶段数据
51 if idx_start < len(t):
52     t_stable_point = t[idx_start]
53     print(f"\n温度和水分从 t = {t_stable_point:.2f} 开始基本不变 (同时满足5%误差带)。")
54
55     T_stable_phase = T[idx_start:]
56     C_stable_phase = C[idx_start:]
57
58     print("\n--- 稳定阶段统计信息 ---")
59     print(f"温度 T: 最大值={np.max(T_stable_phase):.4f}, 最小值={np.min(T_stable_phase):.4f}, 平均值={np.mean(T_stable_phase):.4f}")
60     print(f"水分 C: 最大值={np.max(C_stable_phase):.4f}, 最小值={np.min(C_stable_phase):.4f}, 平均值={np.mean(C_stable_phase):.4f}")
61
62     df_out = df.iloc[idx_start:].copy()
63 else:
64     print("未找到稳定时间段, 数据始终在变化。")
65     df_out = df.copy()
66     t_stable_point = None
67
68 output_path.parent.mkdir(parents=True, exist_ok=True)
69 plot_path.parent.mkdir(parents=True, exist_ok=True)
70 df_out.to_excel(output_path, index=False)
71 print(f"\n稳定阶段数据已保存至 '{output_path}'")
72
73 # 绘图
74 fig, ax1 = plt.subplots(figsize=(10, 6))
75
76 # 温度曲线 (左轴)
77 color_T = 'tab:red'
78 ax1.set_xlabel('时间 t (s)')
79 ax1.set_ylabel('温度 T (°C)', color=color_T)
80 ax1.plot(t, T, color=color_T, label='温度')
81 ax1.tick_params(axis='y', labelcolor=color_T)
82 ax1.grid(alpha=0.3)
83
84 # 水分曲线 (右轴)
85 ax2 = ax1.twinx()
86 color_C = 'tab:blue'
87 ax2.set_ylabel('水分浓度 C', color=color_C)
88 ax2.plot(t, C, color=color_C, label='水分浓度')
89 ax2.tick_params(axis='y', labelcolor=color_C)
90 ax2.set_ylim(0, 0.2) # 修改右轴范围为 0 到 1
91
92 # 绘制5%误差带 (水平虚线)
93 ax1.axhline(y=T_stable + tol_T, color='gray', linestyle=':', alpha=0.6)
94 ax1.axhline(y=T_stable - tol_T, color='gray', linestyle=':', alpha=0.6)
95 ax2.axhline(y=C_stable + tol_C, color='gray', linestyle=':', alpha=0.6)
96 ax2.axhline(y=C_stable - tol_C, color='gray', linestyle=':', alpha=0.6)

```

```

97
98 # 标注稳定起始点（无箭头）
99 if t_stable_point is not None:
100     ax1.axvline(x=t_stable_point, color='green', linestyle='--', linewidth=2)
101     # 标注文字
102     ax1.text(t_stable_point + (t[-1]-t[0])*0.02, np.max(T)*0.95,
103             f'同时稳定起点\nt={t_stable_point:.1f}s',
104             color='green', fontsize=12, verticalalignment='top')
105
106 # 合并图例
107 lines_1, labels_1 = ax1.get_legend_handles_labels()
108 lines_2, labels_2 = ax2.get_legend_handles_labels()
109 ax1.legend(lines_1 + lines_2, labels_1 + labels_2, loc='center right')
110
111 plt.tight_layout()
112 plt.savefig(plot_path, dpi=300, bbox_inches='tight')
113 plt.show()
114 plt.close()

```

Listing 4: 问题二恒温边界生成程序

```

1 """生成问题二的 5500 s 初始剖面。"""
2
3 from pathlib import Path
4
5 import numpy as np
6 import pandas as pd
7
8 from q2_3 import OUTPUT_DIR, get_boundary_funcs, solve_pde
9
10
11 BASE_DIR = Path(__file__).resolve().parent
12 INITIAL_TIME = 0.0
13 TARGET_TIME = 5500.0
14 DT = 1.0
15 DR = 0.001
16 NR = 21
17 R_MAX = 0.02
18
19
20 def main():
21     """用附录 3 变物性模型计算 5500 s 状态。"""
22     boundary_file = OUTPUT_DIR / "output1.xlsx"
23     if not boundary_file.exists():
24         raise FileNotFoundError(f"未找到边界文件: {boundary_file}, 请先运行 q1_1.py")
25
26     T_func, C_func = get_boundary_funcs(path=boundary_file, t_start=INITIAL_TIME)
27     t, r, T, C = solve_pde(
28         dt=DT,
29         dr=DR,
30         r_max=R_MAX,
31         t_end=TARGET_TIME,
32         t_start=INITIAL_TIME,
33         initial_file=None,
34         initial_T=28.0,
35         initial_C=2.55,
36         T_air_func=T_func,
37         C_air_func=C_func,
38     )
39
40     result = pd.DataFrame({
41         "半径 r (cm)": r * 100.0,
42         "温度 T (°C)": np.round(T[-1], 4),
43         "水分浓度 C (kg/kg)": np.round(C[-1], 4),
44     })
45     out_file = OUTPUT_DIR / "output3.xlsx"
46     result.to_excel(out_file, index=False)
47     print(f"5500 s 初始剖面已保存: {out_file}")
48     print(result.to_string(index=False))
49     return out_file, t, r, T, C
50
51
52 if __name__ == "__main__":
53     main()

```

Listing 5: 问题二初始剖面生成程序

```

1 """问题二：恒温段求解、输出与检验。"""
2
3 from pathlib import Path
4
5 import matplotlib.pyplot as plt

```

```

6 import numpy as np
7 import pandas as pd
8
9
10 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Arial Unicode MS"]
11 plt.rcParams["axes.unicode_minus"] = False
12
13 BASE_DIR = Path(__file__).resolve().parent
14 OUTPUT_DIR = BASE_DIR / "output_excel"
15 PIC_DIR = BASE_DIR / "pic"
16 DEFAULT_INITIAL_FILE = OUTPUT_DIR / "output3.xlsx"
17 DEFAULT_BOUNDARY_FILE = OUTPUT_DIR / "output2.xlsx"
18 INITIAL_TIME = 5500.0
19 DEFAULT_SIMULATION_TIME = 10800.0 # 5500 s 后计算3小时
20
21
22 def _as_scalar_or_array(value):
23     arr = np.asarray(value, dtype=float)
24     return float(arr) if arr.ndim == 0 else arr
25
26
27 def get_boundary_funcs(path=DEFAULT_BOUNDARY_FILE, t_start=INITIAL_TIME):
28     """读取边界，不向前外推。"""
29     path = Path(path)
30     if not path.is_absolute():
31         path = BASE_DIR / path
32     if not path.exists():
33         raise FileNotFoundError(f"Boundary file not found: {path}")
34
35     df = pd.read_excel(path)
36     if df.shape[1] < 3 or len(df) < 2:
37         raise ValueError("Boundary file must contain at least 2 rows and 3 columns")
38     t_data = df.iloc[:, 0].to_numpy(dtype=float)
39     T_data = df.iloc[:, 1].to_numpy(dtype=float)
40     C_data = df.iloc[:, 2].to_numpy(dtype=float)
41     if not (np.all(np.isfinite(t_data)) and np.all(np.isfinite(T_data))
42             and np.all(np.isfinite(C_data))):
43         raise ValueError("Boundary data contains NaN or infinite values")
44     if np.any(np.diff(t_data) <= 0):
45         raise ValueError("Boundary time must be strictly increasing")
46     if t_start < t_data[0] - 1e-9:
47         raise ValueError(
48             f"Simulation starts at {t_start:g} s, but boundary data starts at "
49             f"{t_data[0]:g} s. Do not extrapolate backward."
50         )
51
52     tail_n = min(1000, len(t_data))
53     T_tail = float(np.mean(T_data[-tail_n:]))
54     C_tail = float(np.mean(C_data[-tail_n:]))
55
56     def make_func(values, tail_value):
57         def func(query):
58             q = np.asarray(query, dtype=float)
59             if np.any(q < t_data[0] - 1e-9):
60                 raise ValueError(
61                     f"Boundary queried before available data: min={np.min(q):g} s, "
62                     f"first={t_data[0]:g} s"
63                 )
64             result = np.interp(q, t_data, values)
65             result = np.where(q > t_data[-1], tail_value, result)
66             return _as_scalar_or_array(result)
67
68         return func
69
70     T_func = make_func(T_data, T_tail)
71     C_func = make_func(C_data, C_tail)
72     print(
73         f"Loaded boundary: {path.name}, range={t_data[0]:g}..{t_data[-1]:g} s, "
74         f"tail=({T_tail:.4f} C, {C_tail:.6f})"
75     )
76     return T_func, C_func
77
78
79 def load_initial_profile(path=DEFAULT_INITIAL_FILE, r_max=0.02, Nr=21):
80     """读取 5500 s 剖面并插值。"""
81     path = Path(path)
82     if not path.is_absolute():
83         path = BASE_DIR / path
84
85     if not path.exists():
86         raise FileNotFoundError(f"Initial profile file not found: {path}")
87
88     df = pd.read_excel(path)
89
90     if df.shape[1] < 3:

```

```

91         raise ValueError("output3.xlsx必须包含三列: r,T,C")
92
93     radius_cm = df.iloc[:, 0].to_numpy(dtype=float)
94     T0_data = df.iloc[:, 1].to_numpy(dtype=float)
95     C0_data = df.iloc[:, 2].to_numpy(dtype=float)
96
97     # 删除空行
98     mask = np.isfinite(radius_cm) & np.isfinite(T0_data) & np.isfinite(C0_data)
99     radius_cm = radius_cm[mask]
100    T0_data = T0_data[mask]
101    C0_data = C0_data[mask]
102
103    if len(radius_cm) < 2 or np.any(np.diff(radius_cm) <= 0):
104        raise ValueError("output3.xlsx半径必须严格递增")
105
106    # 检查半径范围
107    if radius_cm[0] > 1e-12:
108        raise ValueError("output3.xlsx第一行半径必须为0cm")
109
110    # 缺少 2.0 cm 时补节点
111    if radius_cm[-1] < r_max * 100 - 1e-12:
112        if abs(radius_cm[-1] - (r_max*100-0.1)) < 1e-8:
113            radius_cm = np.append(radius_cm, r_max*100)
114            T0_data = np.append(T0_data, T0_data[-1])
115            C0_data = np.append(C0_data, C0_data[-1])
116        else:
117            raise ValueError(
118                f"output3.xlsx半径范围不足: {radius_cm[0]}~{radius_cm[-1]} cm"
119            )
120
121    radius_m = radius_cm / 100.0
122
123    r = np.linspace(0.0, r_max, Nr)
124
125    T0 = np.interp(r, radius_m, T0_data)
126    C0 = np.interp(r, radius_m, C0_data)
127
128    print(
129        f"Loaded initial profile: {path.name}"
130    )
131    print(
132        f"radius: {radius_cm[0]:.1f}~{radius_cm[-1]:.1f} cm, "
133        f"nodes={len(radius_cm)}"
134    )
135    print(
136        f"T range={T0.min():.4f}~{T0.max():.4f} C, "
137        f"C range={C0.min():.6f}~{C0.max():.6f} kg/kg"
138    )
139
140    return r, T0, C0
141
142    # 附录 3 物性公式
143    def rho_of_C(C_val):
144        return 650.0 + 128.0 * C_val
145
146
147    def cp_of_C(C_val):
148        return 1450.0 + 2736.0 * (C_val / (C_val + 1.0))
149
150
151    def k_of_C(C_val):
152        return 0.21 + 0.38 * (C_val / (C_val + 1.0))
153
154
155    def D_of_CT(C_val, T_val):
156        C_safe = np.clip(C_val, 0.05, 10.0)
157        T_K = np.clip(T_val + 273.15, 273.15, 373.15)
158        D = 2.4e-3 * np.exp(-0.45 / C_safe) * np.exp(-3850.0 / T_K)
159        return np.clip(D, 1e-14, 1e-3)
160
161
162    def solve_tridiag(a, b, c, d):
163        """追赶法求解三对角方程。"""
164        n = len(b)
165        cp = np.zeros(n)
166        dp = np.zeros(n)
167        if abs(b[0]) < 1e-14:
168            raise ZeroDivisionError("Singular tridiagonal system at first row")
169        cp[0] = c[0] / b[0]
170        dp[0] = d[0] / b[0]
171        for i in range(1, n):
172            m = b[i] - a[i] * cp[i - 1]
173            if abs(m) < 1e-14:
174                raise ZeroDivisionError(f"Singular tridiagonal system at row {i}")

```

```

176     if i < n - 1:
177         cp[i] = c[i] / m
178         dp[i] = (d[i] - a[i] * dp[i - 1]) / m
179     x = np.zeros(n)
180     x[-1] = dp[-1]
181     for i in range(n - 2, -1, -1):
182         x[i] = dp[i] - cp[i] * x[i + 1]
183     return x
184
185
186 def solve_pde(
187     dt,
188     dr,
189     r_max=0.02,
190     t_end=DEFAULT_SIMULATION_TIME,
191     t_start=INITIAL_TIME,
192     initial_file=DEFAULT_INITIAL_FILE,
193     initial_T=None,
194     initial_C=None,
195     T_air_func=None,
196     C_air_func=None,
197 ):
198     """求解恒温段, t 为经过时间, 边界用绝对时间。"""
199     if dt <= 0 or dr <= 0 or r_max <= 0 or t_end < 0:
200         raise ValueError("dt, dr and r_max must be positive; t_end cannot be negative")
201     if (T_air_func is None) != (C_air_func is None):
202         raise ValueError("Provide both T_air_func and C_air_func, or neither")
203     if T_air_func is None:
204         T_air_func, C_air_func = get_boundary_funcs(t_start=t_start)
205
206     Nr = int(round(r_max / dr)) + 1
207     Nt = int(round(t_end / dt)) + 1
208     r = np.linspace(0.0, r_max, Nr)
209     t = np.linspace(0.0, t_end, Nt)
210     if initial_T is None and initial_C is None:
211         r0, T0, C0 = load_initial_profile(initial_file, r_max=r_max, Nr=Nr)
212         if not np.allclose(r0, r):
213             raise ValueError("Initial profile grid does not match solver grid")
214     elif initial_T is None or initial_C is None:
215         raise ValueError("initial_T 和 initial_C 必须同时提供")
216     else:
217         T0 = np.asarray(initial_T, dtype=float)
218         C0 = np.asarray(initial_C, dtype=float)
219         if T0.ndim == 0:
220             T0 = np.full(Nr, float(T0))
221         if C0.ndim == 0:
222             C0 = np.full(Nr, float(C0))
223         if T0.shape != (Nr,) or C0.shape != (Nr,):
224             raise ValueError(f"初始数组长度必须为 {Nr}")
225         if not (np.all(np.isfinite(T0)) and np.all(np.isfinite(C0))):
226             raise ValueError("初始温度或水分含有 NaN/无穷值")
227
228     T = np.zeros((Nt, Nr))
229     C = np.zeros((Nt, Nr))
230     T[0, :] = T0
231     C[0, :] = C0
232
233     h, hm = 25.0, 8e-7
234     i_s = Nr - 1
235     V_factor = i_s - 0.25
236     r_innner = r[1:-1]
237     coef_p = 1.0 + dr / (2.0 * r_innner)
238     coef_m = 1.0 - dr / (2.0 * r_innner)
239
240     for n in range(Nt - 1):
241         T_air_next = float(T_air_func(t_start + t[n + 1]))
242         C_air_next = float(C_air_func(t_start + t[n + 1]))
243
244         C_mid_L = 0.5 * (C[n, :-2] + C[n, 1:-1])
245         C_mid_R = 0.5 * (C[n, 1:-1] + C[n, 2:])
246         T_mid_L = 0.5 * (T[n, :-2] + T[n, 1:-1])
247         T_mid_R = 0.5 * (T[n, 1:-1] + T[n, 2:])
248         k_L = k_of_C(C_mid_L)
249         k_R = k_of_C(C_mid_R)
250         D_L = D_of_CT(C_mid_L, T_mid_L)
251         D_R = D_of_CT(C_mid_R, T_mid_R)
252         rho_innner = rho_of_C(C[n, 1:-1])
253         cp_innner = cp_of_C(C[n, 1:-1])
254         alpha_T_innner = dt / (rho_innner * cp_innner * dr**2)
255
256         a_T = np.zeros(Nr)
257         b_T = np.zeros(Nr)
258         c_T = np.zeros(Nr)
259         d_T = np.zeros(Nr)
260         for idx in range(Nr - 2):

```

```

261         i = idx + 1
262         ai = alpha_T_inner[idx]
263         a_T[i] = -ai * coef_m[idx] * k_L[idx]
264         b_T[i] = 1.0 + ai * (coef_p[idx] * k_R[idx] + coef_m[idx] * k_L[idx])
265         c_T[i] = -ai * coef_p[idx] * k_R[idx]
266         d_T[i] = T[n, i]
267
268         k_01 = k_of_C(0.5 * (C[n, 0] + C[n, 1]))
269         rho_0 = rho_of_C(C[n, 0])
270         cp_0 = cp_of_C(C[n, 0])
271         alpha_0 = dt / (rho_0 * cp_0 * dr**2)
272         b_T[0] = 1.0 + 4.0 * alpha_0 * k_01
273         c_T[0] = -4.0 * alpha_0 * k_01
274         d_T[0] = T[n, 0]
275
276         C_s_T = C[n, i_s]
277         rho_s_T = rho_of_C(C_s_T)
278         cp_s_T = cp_of_C(C_s_T)
279         # 表面界面取两节点平均物性
280         k_s_T = k_of_C(0.5 * (C[n, i_s - 1] + C[n, i_s]))
281         beta_T = dt / (rho_s_T * cp_s_T * V_factor * dr**2)
282         coef_s_T_diff = 2.0 * (i_s - 0.5) * k_s_T
283         coef_s_T_conv = 2.0 * i_s * h * dr
284         a_T[i_s] = -beta_T * coef_s_T_diff
285         b_T[i_s] = 1.0 + beta_T * (coef_s_T_diff + coef_s_T_conv)
286         d_T[i_s] = T[n, i_s] + beta_T * coef_s_T_conv * T_air_next
287         T[n + 1, :] = solve_tridiag(a_T, b_T, c_T, d_T)
288
289         alpha_C_inner = dt / dr**2
290         a_C = np.zeros(Nr)
291         b_C = np.zeros(Nr)
292         c_C = np.zeros(Nr)
293         d_C = np.zeros(Nr)
294         for idx in range(Nr - 2):
295             i = idx + 1
296             a_C[i] = -alpha_C_inner * coef_m[idx] * D_L[idx]
297             b_C[i] = 1.0 + alpha_C_inner * (coef_p[idx] * D_R[idx] + coef_m[idx] * D_L[idx])
298             c_C[i] = -alpha_C_inner * coef_p[idx] * D_R[idx]
299             d_C[i] = C[n, i]
300
301         D_01 = D_of_CT(0.5 * (C[n, 0] + C[n, 1]), 0.5 * (T[n, 0] + T[n, 1]))
302         b_C[0] = 1.0 + 4.0 * alpha_C_inner * D_01
303         c_C[0] = -4.0 * alpha_C_inner * D_01
304         d_C[0] = C[n, 0]
305
306         # 共用表面界面扩散系数
307         D_s_C = D_of_CT(
308             0.5 * (C[n, i_s - 1] + C[n, i_s]),
309             0.5 * (T[n, i_s - 1] + T[n, i_s]),
310         )
311         g1 = dt * 2.0 * (i_s - 0.5) * D_s_C / (V_factor * dr**2)
312         g2 = dt * 2.0 * i_s * hm / (V_factor * dr)
313         a_C[i_s] = -g1
314         b_C[i_s] = 1.0 + g1 + g2
315         d_C[i_s] = C[n, i_s] + g2 * C_air_next
316         C[n + 1, :] = solve_tridiag(a_C, b_C, c_C, d_C)
317
318         C[n + 1, :] = np.clip(C[n + 1, :], 0.0, 10.0)
319         T[n + 1, :] = np.clip(T[n + 1, :], -50.0, 250.0)
320
321     return t, r, T, C
322
323
324 def _solve_for_validation(dt, dr, t_end=1800.0):
325     T_func, C_func = get_boundary_funcs(t_start=INITIAL_TIME)
326     return solve_pde(
327         dt, dr, t_end=t_end, t_start=INITIAL_TIME,
328         initial_file=DEFAULT_INITIAL_FILE,
329         T_air_func=T_func, C_air_func=C_func,
330     )
331
332
333 def verify_initial_profile():
334     """检查初值是否等于 output3.xlsx."""
335     T0, C0 = load_initial_profile(DEFAULT_INITIAL_FILE, r_max=0.02, Nr=21)
336     t, r, T, C = _solve_for_validation(1.0, 0.001, t_end=0.0)
337     err_T = float(np.max(np.abs(T[0] - T0)))
338     err_C = float(np.max(np.abs(C[0] - C0)))
339     print("\n初始剖面检查")
340     print(f"max |T[0]-T_output3| = {err_T:.3e} C")
341     print(f"max |C[0]-C_output3| = {err_C:.3e} kg/kg")
342     print(f"initial radius grid = {r[0] * 100:g}...{r[-1] * 100:g} cm")
343     if err_T > 1e-10 or err_C > 1e-10:
344         raise AssertionError("Initial row does not match output3.xlsx")
345     return t, r, T, C

```

```

346
347
348 def verify_time_step(t_end=1800.0):
349     print("\n时间步收敛")
350     dt_list = [1.0, 0.5, 0.25]
351     results = {dt: _solve_for_validation(dt, 0.001, t_end=t_end) for dt in dt_list}
352     common_t = np.arange(0.0, t_end + 1e-9, 100.0)
353     T_common, C_common = {}, {}
354     for dt, result in results.items():
355         _, _, T, C = result
356         idx = np rint(common_t / dt).astype(int)
357         T_common[dt] = T[idx, :]
358         C_common[dt] = C[idx, :]
359     err_T_1_05 = np.max(np.abs(T_common[1.0] - T_common[0.5]))
360     err_T_05_025 = np.max(np.abs(T_common[0.5] - T_common[0.25]))
361     err_C_1_05 = np.max(np.abs(C_common[1.0] - C_common[0.5]))
362     err_C_05_025 = np.max(np.abs(C_common[0.5] - C_common[0.25]))
363     print(f" T: dt 1.0 vs 0.5 = {err_T_1_05:.6g} C")
364     print(f" T: dt 0.5 vs 0.25 = {err_T_05_025:.6g} C")
365     print(f" C: dt 1.0 vs 0.5 = {err_C_1_05:.6g}")
366     print(f" C: dt 0.5 vs 0.25 = {err_C_05_025:.6g}")
367     print(f" 温度收敛比 = {err_T_05_025 / err_T_1_05:.4f}")
368     print(f" 水分收敛比 = {err_C_05_025 / err_C_1_05:.4f}")
369     print(" 标准: 第二次误差更小, 收敛比接近 0.5")
370     ratio_T = err_T_05_025 / err_T_1_05
371     ratio_C = err_C_05_025 / err_C_1_05
372     if not (err_T_05_025 < err_T_1_05 and err_C_05_025 < err_C_1_05
373             and 0.35 < ratio_T < 0.65 and 0.35 < ratio_C < 0.65
374             and err_T_1_05 < 0.01 and err_C_1_05 < 0.01):
375         raise AssertionError("问题二时间步收敛未通过")
376     return results
377
378
379 def verify_space_step(t_end=1800.0):
380     print("\n空间步收敛")
381     dt = 1.0
382     dr_list = [0.001, 0.0005, 0.00025]
383     results = {dr: _solve_for_validation(dt, dr, t_end=t_end) for dr in dr_list}
384     common_r = np.array([0.0, 0.005, 0.01, 0.015, 0.02])
385     T_final, C_final = {}, {}
386     for dr, result in results.items():
387         _, r, T, C = result
388         T_final[dr] = np.interp(common_r, r, T[-1, :])
389         C_final[dr] = np.interp(common_r, r, C[-1, :])
390     err_T_1_05 = np.max(np.abs(T_final[0.001] - T_final[0.0005]))
391     err_T_05_025 = np.max(np.abs(T_final[0.0005] - T_final[0.00025]))
392     err_C_1_05 = np.max(np.abs(C_final[0.001] - C_final[0.0005]))
393     err_C_05_025 = np.max(np.abs(C_final[0.0005] - C_final[0.00025]))
394     print(f" T: dr 1.0 vs 0.5 mm = {err_T_1_05:.6g} C")
395     print(f" T: dr 0.5 vs 0.25 mm = {err_T_05_025:.6g} C")
396     print(f" C: dr 1.0 vs 0.5 mm = {err_C_1_05:.6g}")
397     print(f" C: dr 0.5 vs 0.25 mm = {err_C_05_025:.6g}")
398     print(f" 温度收敛比 = {err_T_05_025 / err_T_1_05:.4f}")
399     print(f" 水分收敛比 = {err_C_05_025 / err_C_1_05:.4f}")
400     print(" 标准: 第二次误差更小, 收敛比接近 0.25")
401     ratio_T = err_T_05_025 / err_T_1_05
402     ratio_C = err_C_05_025 / err_C_1_05
403     if not (err_T_05_025 < err_T_1_05 and err_C_05_025 < err_C_1_05
404             and 0.15 < ratio_T < 0.35 and 0.15 < ratio_C < 0.35
405             and err_T_1_05 < 0.01 and err_C_1_05 < 0.01):
406         raise AssertionError("问题二空间步收敛未通过")
407     return results
408
409
410 def _control_volumes(dr, Nr):
411     V = np.zeros(Nr)
412     V[0] = np.pi * (dr / 2.0) ** 2
413     for i in range(1, Nr - 1):
414         V[i] = 2.0 * np.pi * (i * dr) * dr
415     r_surf = (Nr - 1) * dr
416     V[-1] = np.pi * (r_surf**2 - (r_surf - dr / 2.0) ** 2)
417     return V, r_surf
418
419
420 def verify_conservation(t_end=DEFAULT_SIMULATION_TIME):
421     print("\n守恒检查")
422     dt, dr = 1.0, 0.001
423     T_func, C_func = get_boundary_funcs(t_start=INITIAL_TIME)
424     t, r, T, C = solve_pde(
425         dt, dr, t_end=t_end, t_start=INITIAL_TIME,
426         initial_file=DEFAULT_INITIAL_FILE,
427         T_air_func=T_func, C_air_func=C_func,
428     )
429     _, Nr = T.shape
430     V, r_surf = _control_volumes(dr, Nr)

```

```

431 h, hm = 25.0, 8e-7
432 A_surf = 2.0 * np.pi * r_surf
433
434 M = np.sum(C * V, axis=1)
435 # 使用与表面方程相同的离散通量
436 C_air_next = np.asarray(C_func(INITIAL_TIME + t[1:]), dtype=float)
437 m_flux_step = hm * A_surf * (C_air_next - C[1:, -1])
438 M_flux_int = np.concatenate([[0.0], np.cumsum(m_flux_step * dt)])
439 M_change = M - M[0]
440 moisture_rel_err = abs(M_change[-1] - M_flux_int[-1]) / max(abs(M_change[-1]), 1e-12)
441
442 # 按离散方程计算蓄热量
443 thermal_storage = np.sum(
444     rho_of_C(C[:-1]) * cp_of_C(C[:-1]) * (T[1:] - T[:-1]) * V,
445     axis=1,
446 )
447 T_air_next = np.asarray(T_func(INITIAL_TIME + t[1:]), dtype=float)
448 q_flux_step = h * A_surf * (T_air_next - T[1:, -1])
449 energy_residual = abs(np.sum(thermal_storage) - np.sum(q_flux_step * dt))
450 energy_scale = max(abs(np.sum(thermal_storage)), 1e-12)
451 energy_rel_err = energy_residual / energy_scale
452
453 print(f" 最终水分守恒残差 = {moisture_rel_err:.4%}")
454 print(f" 最终热量诊断残差 = {energy_rel_err:.4%}")
455 print(" 标准: 水分和热量残差均小于 1%")
456 if moisture_rel_err >= 0.01 or energy_rel_err >= 0.01:
457     raise AssertionError("问题二守恒检验未通过")
458 return moisture_rel_err, energy_rel_err
459
460
461 def verify_physical_bounds(t_end=DEFAULT_SIMULATION_TIME):
462     print("\n物理范围检查")
463     t, r, T, C = _solve_for_validation(1.0, 0.001, t_end=t_end)
464     finite = bool(np.all(np.isfinite(T)) and np.all(np.isfinite(C)))
465     clipped_T = bool(np.any((T <= -50.0) | (T >= 250.0)))
466     clipped_C = bool(np.any((C <= 0.0) | (C >= 10.0)))
467     print(f" 数组有限: {finite}")
468     print(f" T 范围: {T.min():.6f}..{T.max():.6f} C; 触及截断: {clipped_T}")
469     print(f" C 范围: {C.min():.6f}..{C.max():.6f} kg/kg; 触及截断: {clipped_C}")
470     print(" 标准: 有限值、C>0, 且计算未触及截断")
471     if not finite:
472         raise AssertionError("NaN or infinite value detected")
473     if np.any(C < -1e-12) or clipped_T or clipped_C:
474         raise AssertionError("问题二物理范围检验未通过")
475     return t, r, T, C
476
477
478 def save_problem2_results(
479     dt=1.0,
480     dr=0.001,
481     t_end=DEFAULT_SIMULATION_TIME,
482 ):
483     """保存问题二四个结果表, 时间从 0 开始。"""
484     T_func, C_func = get_boundary_funcs(t_start=INITIAL_TIME)
485     t, r, T, C = solve_pde(
486         dt, dr, t_end=t_end, t_start=INITIAL_TIME,
487         initial_file=DEFAULT_INITIAL_FILE,
488         T_func=T_func, C_func=C_func,
489     )
490     radii_cm = [f"{value * 100.0:.1f}" for value in r]
491     time_s = np rint(t).astype(int)
492     df_T = pd.DataFrame(np.round(T, 4), columns=radii_cm)
493     df_C = pd.DataFrame(np.round(C, 4), columns=radii_cm)
494     df_T.insert(0, "时间", time_s)
495     df_C.insert(0, "时间", time_s)
496
497     target_hours = np.arange(0.5, 3.0 + 1e-12, 0.5)
498     target_seconds = np rint(target_hours * 3600.0).astype(int)
499     target_radii = ["0.0", "0.5", "1.0", "1.5", "2.0"]
500     # 按目标秒数取样, 避免浮点时间比较误差
501     sample_idx = [int(np.argmax(np.abs(time_s - value))) for value in target_seconds]
502     df_T_sample = df_T.iloc[sample_idx][["时间"] + target_radii].copy()
503     df_C_sample = df_C.iloc[sample_idx][["时间"] + target_radii].copy()
504     # 摘要表改用小时
505     df_T_sample["时间"] = df_T_sample["时间"] / 3600.0
506     df_C_sample["时间"] = df_C_sample["时间"] / 3600.0
507
508     if not np.allclose(df_T_sample["时间"].to_numpy(), target_hours):
509         raise AssertionError("温度摘要表时间点错误")
510     if not np.allclose(df_C_sample["时间"].to_numpy(), target_hours):
511         raise AssertionError("水分摘要表时间点错误")
512
513     outputs = {
514         "q2_1.xlsx": df_T,
515         "q2_2.xlsx": df_C,
516     }

```

```

516     "q2_3.xlsx": df_T_sample,
517     "q2_4.xlsx": df_C_sample,
518 }
519 OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
520 for name, frame in outputs.items():
521     target = OUTPUT_DIR / name
522     try:
523         frame.to_excel(target, index=False)
524         print(f"已保存: {target}")
525     except PermissionError:
526         pending = OUTPUT_DIR / name.replace(".xlsx", "_pending.xlsx")
527         frame.to_excel(pending, index=False)
528         print(f"文件被占用, 已写入临时结果: {pending}")
529
530 # 按题目格式保存双工作表文件
531 result2_file = OUTPUT_DIR / "result2.xlsx"
532 try:
533     with pd.ExcelWriter(result2_file) as writer:
534         df_T.to_excel(writer, sheet_name="温度", index=False)
535         df_C.to_excel(writer, sheet_name="水分浓度", index=False)
536         print(f"题目正式结果已保存: {result2_file}")
537 except PermissionError:
538     pending = OUTPUT_DIR / "result2_pending.xlsx"
539     with pd.ExcelWriter(pending) as writer:
540         df_T.to_excel(writer, sheet_name="温度", index=False)
541         df_C.to_excel(writer, sheet_name="水分浓度", index=False)
542         print(f"文件被占用, 已写入临时结果: {pending}")
543
544 print(f"完整表时间: 0~{int(round(t_end))} s, 共 {len(t)} 行")
545 print(f"摘要表时间: 0.5、1.0、1.5、2.0、2.5、3.0 h")
546 print(f"径向范围: 0~{r[-1] * 100:g} cm, 共 {len(r)} 个节点")
547 return outputs, t, r, T, C
548
549
550 def plot_problem2_results(t, r, T, C):
551     """绘制问题二温度、水分和热力图。"""
552     PIC_DIR.mkdir(parents=True, exist_ok=True)
553     time_h = t / 3600.0
554     target_r = [0.0, 0.5, 1.0, 1.5, 2.0]
555     target_idx = [int(np.argmax(np.abs(r * 100.0 - value))) for value in target_r]
556
557     plt.figure(figsize=(10, 6))
558     for radius, idx in zip(target_r, target_idx):
559         plt.plot(time_h, C[:, idx], label=f"r={radius:g} cm")
560     plt.xlabel("时间 t (h)")
561     plt.ylabel("水分浓度 C (kg/kg)")
562     plt.grid(alpha=0.3)
563     plt.legend()
564     plt.tight_layout()
565     plt.savefig(PIC_DIR / "q2_水分时间演化图.png", dpi=300)
566     plt.close()
567
568     plt.figure(figsize=(10, 6))
569     for radius, idx in zip(target_r, target_idx):
570         plt.plot(time_h, T[:, idx], label=f"r={radius:g} cm")
571     plt.xlabel("时间 t (h)")
572     plt.ylabel("温度 T (°C)")
573     plt.grid(alpha=0.3)
574     plt.legend()
575     plt.tight_layout()
576     plt.savefig(PIC_DIR / "q2_温度时间演化图.png", dpi=300)
577     plt.close()
578
579     fig, axes = plt.subplots(1, 2, figsize=(16, 5))
580     extent = [time_h[0], time_h[-1], r[0] * 100.0, r[-1] * 100.0]
581     im0 = axes[0].imshow(T.T, origin="lower", aspect="auto", extent=extent, cmap="YlOrRd")
582     axes[0].set_xlabel("时间 t (h)")
583     axes[0].set_ylabel("半径 r (cm)")
584     axes[0].set_title("温度时空分布")
585     fig.colorbar(im0, ax=axes[0], label="温度 (°C)")
586     im1 = axes[1].imshow(C.T, origin="lower", aspect="auto", extent=extent, cmap="Blues_r")
587     axes[1].set_xlabel("时间 t (h)")
588     axes[1].set_ylabel("半径 r (cm)")
589     axes[1].set_title("水分时空分布")
590     fig.colorbar(im1, ax=axes[1], label="水分浓度 (kg/kg)")
591     fig.tight_layout()
592     fig.savefig(PIC_DIR / "q2_温度和水分热力图.png", dpi=300)
593     plt.close(fig)
594
595
596 if __name__ == "__main__":
597     _, t_result, r_result, T_result, C_result = save_problem2_results()
598     plot_problem2_results(t_result, r_result, T_result, C_result)
599     verify_initial_profile()
600     verify_time_step()

```

```

601     verify_space_step()
602     verify_conservation()
603     verify_physical_bounds()
604     print("\n问题二全部数值检验完成。")

```

Listing 6: 问题二求解、检验与输出程序

2.3 问题三程序

```

1  """问题三：计算达到含水率阈值的时间。"""
2
3  from pathlib import Path
4
5  import numpy as np
6  import pandas as pd
7
8  from q2_3 import (
9      BASE_DIR,
10     DEFAULT_INITIAL_FILE,
11     OUTPUT_DIR,
12     INITIAL_TIME,
13     _control_volumes,
14     get_boundary_funcs,
15     load_initial_profile,
16     solve_pde,
17 )
18
19
20 PIC_DIR = BASE_DIR / "pic"
21 MOISTURE_LIMIT = 0.15
22 MAX_ABSOLUTE_TIME = 259200.0 # 最长3天
23 DT = 60.0
24 DR = 0.001
25 R_MAX = 0.02
26
27
28 def get_full_boundary_funcs():
29     """拼接两段环境边界。"""
30     T_pre, C_pre = get_boundary_funcs(
31         path=OUTPUT_DIR / "output1.xlsx", t_start=0.0
32     )
33     stable_file = OUTPUT_DIR / "output2.xlsx"
34     if not stable_file.exists():
35         raise FileNotFoundError(f"未找到恒温边界: {stable_file}, 请先运行 q2_1.py")
36     T_stable, C_stable = get_boundary_funcs(
37         path=stable_file, t_start=INITIAL_TIME
38     )
39
40 def join(pre_func, stable_func):
41     def func(query):
42         q = np.asarray(query, dtype=float)
43         scalar = q.ndim == 0
44         q1 = np.atleast_1d(q)
45         if np.any(q1 < -1e-9):
46             raise ValueError("边界查询时间不能为负")
47         out = np.empty_like(q1)
48         pre_mask = q1 <= INITIAL_TIME
49         if np.any(pre_mask):
50             out[pre_mask] = np.asarray(pre_func(q1[pre_mask]), dtype=float)
51         if np.any(~pre_mask):
52             out[~pre_mask] = np.asarray(stable_func(q1[~pre_mask]), dtype=float)
53         return float(out[0]) if scalar else out
54
55     return func
56
57 return join(T_pre, T_stable), join(C_pre, C_stable)
58
59
60 def get_simulation_end(max_absolute_time=MAX_ABSOLUTE_TIME, dt=DT):
61     """返回最大时间内的整步终点。"""
62     if max_absolute_time < 0 or dt <= 0:
63         raise ValueError("时间参数必须为正")
64     return float(np.floor(max_absolute_time / dt) * dt)
65
66
67 def solve_problem3(dt=DT, dr=DR, t_end=None):
68     """从原始初态计算到阈值或最大时长。"""
69     if t_end is None:
70         t_end = get_simulation_end(dt=dt)
71

```

```

72 # 第三问按全过程计时, 5500 s 后切换恒温边界。
73 T_func, C_func = get_full_boundary_funcs()
74 t, r, T, C = solve_pde(
75     dt=dt,
76     dr=dr,
77     r_max=R_MAX,
78     t_end=t_end,
79     t_start=0.0,
80     initial_file=None,
81     initial_T=28.0,
82     initial_C=2.55,
83     T_air_func=T_func,
84     C_air_func=C_func,
85 )
86
87 max_C = np.max(C, axis=1)
88 hit = np.flatnonzero(max_C <= MOISTURE_LIMIT)
89 if len(hit) == 0:
90     return t, r, T, C, None, max_C
91
92 j = int(hit[0])
93 if j == 0:
94     drying_time = 0.0
95 else:
96     # 相邻时间点线性插值
97     t0, t1 = t[j - 1], t[j]
98     c0, c1 = max_C[j - 1], max_C[j]
99     drying_time = float(
100         t0 + (MOISTURE_LIMIT - c0) * (t1 - t0) / (c1 - c0)
101     )
102 return t, r, T, C, drying_time, max_C
103
104
105 def _select_rows(t, times):
106     """按目标时间取最近行。"""
107     rows = []
108     for target in times:
109         if target < t[0] - 1e-9 or target > t[-1] + 1e-9:
110             continue
111         rows.append(int(np.argmax(np.abs(t - target))))
112     return np.asarray(rows, dtype=int)
113
114
115 def verify_q2_checkpoint():
116     """核对 5500 s 剖面。"""
117     if not DEFAULT_INITIAL_FILE.exists():
118         raise FileNotFoundError(f"未找到问题二初始文件: {DEFAULT_INITIAL_FILE}")
119     T_func, C_func = get_boundary_funcs(
120         path=OUTPUT_DIR / "output1.xlsx", t_start=0.0
121     )
122     _, r, T, C = solve_pde(
123         1.0,
124         DR,
125         r_max=R_MAX,
126         t_end=5500.0,
127         t_start=0.0,
128         initial_file=None,
129         initial_T=28.0,
130         initial_C=2.55,
131         T_air_func=T_func,
132         C_air_func=C_func,
133     )
134     _, T_ref, C_ref = load_initial_profile(
135         DEFAULT_INITIAL_FILE, r_max=R_MAX, Nr=len(r)
136     )
137     err_T = float(np.max(np.abs(T[-1] - T_ref)))
138     err_C = float(np.max(np.abs(C[-1] - C_ref)))
139     print(f"5500 s 剖面校验: T={err_T:.3e}, C={err_C:.3e}")
140     if err_T > 1e-4 or err_C > 1e-4:
141         raise AssertionError("第三问与问题二的 5500 s 剖面不一致")
142     return err_T, err_C
143
144
145 def save_problem3_results(
146     output_file=OUTPUT_DIR / "result3.xlsx",
147     summary_file=OUTPUT_DIR / "result3_summary.xlsx",
148 ):
149     """保存 60 s 完整表和 6 h 摘要。"""
150     t_raw, r, T_raw, C_raw, drying_time, max_C_raw = solve_problem3()
151
152     if drying_time is None:
153         t, T, C, max_C = t_raw, T_raw, C_raw, max_C_raw
154         end_time = float(t[-1])
155         end_label = f"最大计算时长内未达到 {MOISTURE_LIMIT:g}"
156     else:

```

```

157     end_idx = int(np.flatnonzero(max_C_raw <= MOISTURE_LIMIT)[0])
158     # 保留结束前的整步，并追加插值结束行
159     t = t_raw[:end_idx]
160     T = T_raw[:end_idx]
161     C = C_raw[:end_idx]
162     max_C = max_C_raw[:end_idx]
163     if end_idx == 0:
164         frac = 0.0
165     else:
166         t_prev = t_raw[end_idx - 1]
167         frac = (drying_time - t_prev) / (t_raw[end_idx] - t_prev)
168     t = np.append(t, drying_time)
169     T_end = T_raw[max(end_idx - 1, 0)] + frac * (
170         T_raw[end_idx] - T_raw[max(end_idx - 1, 0)]
171     )
172     C_end = C_raw[max(end_idx - 1, 0)] + frac * (
173         C_raw[end_idx] - C_raw[max(end_idx - 1, 0)]
174     )
175     T = np.vstack([T, T_end])
176     C = np.vstack([C, C_end])
177     max_C = np.append(max_C, np.max(C_end))
178     end_time = drying_time
179     end_label = f"烘干结束: {drying_time / 3600.0:.4f} h (自 0 s) "
180
181     radii_cm = [f"{value * 100.0:.1f}" for value in r]
182     time_s = np.round(t, 2)
183     df_full = pd.DataFrame(np.round(C, 4), columns=radii_cm)
184     df_full.insert(0, "时间", time_s)
185
186     # 规则时间点为 6 h 的整数倍
187     regular_hours = np.arange(
188         6.0, np.floor(end_time / 21600.0) * 6.0 + 1e-12, 6.0
189     )
190     summary_idx = _select_rows(t, regular_hours * 3600.0)
191     target_radii = ["0.0", "0.5", "1.0", "1.5", "2.0"]
192     summary_rows = []
193     for idx in summary_idx:
194         row = {"时间/h": round(float(t[idx] / 3600.0), 4)}
195         for radius in target_radii:
196             radius_idx = int(round(float(radius) / (r[1] * 100.0)))
197             row[radius] = round(float(C[idx, radius_idx]), 4)
198         summary_rows.append(row)
199
200     if drying_time is not None:
201         end_idx = int(np.flatnonzero(max_C_raw <= MOISTURE_LIMIT)[0])
202         if end_idx == 0:
203             C_end = C_raw[0]
204         else:
205             t0, t1 = t_raw[end_idx - 1], t_raw[end_idx]
206             frac = (drying_time - t0) / (t1 - t0)
207             C_end = C_raw[end_idx - 1] + frac * (C_raw[end_idx] - C_raw[end_idx - 1])
208         row = {"时间/h": round(float(drying_time / 3600.0), 4)}
209         for radius in target_radii:
210             radius_idx = int(round(float(radius) / (r[1] * 100.0)))
211             row[radius] = float(round(C_end[radius_idx], 4))
212         summary_rows.append(row)
213
214     df_summary = pd.DataFrame(
215         summary_rows, columns=["时间/h"] + target_radii
216     )
217
218     output_file = Path(output_file)
219     summary_file = Path(summary_file)
220     if not output_file.is_absolute():
221         output_file = BASE_DIR / output_file
222     if not summary_file.is_absolute():
223         summary_file = BASE_DIR / summary_file
224     output_file.parent.mkdir(parents=True, exist_ok=True)
225     summary_file.parent.mkdir(parents=True, exist_ok=True)
226     df_full.to_excel(output_file, index=False)
227     df_summary.to_excel(summary_file, index=False)
228
229     print(f"问题三完整结果: {output_file}")
230     print(f"问题三摘要结果: {summary_file}")
231     print(end_label)
232     return output_file, summary_file, t, r, T, C, drying_time, max_C
233
234
235 def verify_problem3():
236     """检查初值、阈值、单调性和范围。"""
237     print("\n问题三数值检验")
238     verify_q2_checkpoint()
239     _, t, r, T, C, drying_time, max_C = save_problem3_results()
240
241     err_T = float(np.max(np.abs(T[0] - 28.0)))

```

```

242 err_C = float(np.max(np.abs(C[0] - 2.55)))
243 print(f"初值最大误差: T={err_T:.3e}, C={err_C:.3e}")
244
245 if drying_time is None:
246     print(f"最大计算时长内未达阈值, 末端 max(C)={max_C[-1]:.6f}")
247 else:
248     cross = int(np.searchsorted(t, drying_time))
249     before = max_C[max(cross - 1, 0)]
250     after = max_C[min(cross, len(max_C) - 1)]
251     print(f"烘干时间: {drying_time:.2f} s = {drying_time / 3600.0:.4f} h")
252     print(f"阈值前 max(C)={before:.6f}")
253     print(f"阈值后 max(C)={after:.6f}")
254     if not (before > MOISTURE_LIMIT and after <= MOISTURE_LIMIT):
255         raise AssertionError("阈值跨越判定错误")
256
257 print(f"max(C) 是否单调不减: {bool(np.all(np.diff(max_C) <= 1e-10))}")
258 print(f"温度范围: {T.min():.6f}~{T.max():.6f} °C")
259 print(f"水分范围: {C.min():.6f}~{C.max():.6f} kg/kg")
260 print(f"标准: 初值误差 < 1e-10; max(C) 单调不减; 阈值正确跨越; 无 NaN 和负值。")
261
262 if err_T > 1e-10 or err_C > 1e-10:
263     raise AssertionError("问题三初值不一致")
264 if not np.all(np.isfinite(T)) or not np.all(np.isfinite(C)):
265     raise AssertionError("问题三出现 NaN 或无穷值")
266 if np.any(C < -1e-12):
267     raise AssertionError("问题三出现负水分浓度")
268 if not np.all(np.diff(max_C) <= 1e-10):
269     raise AssertionError("全域最大含水率不是单调不减")
270
271 return drying_time
272
273
274 def verify_time_step(t_end=21600.0):
275     """比较时间步误差。"""
276     print("\n问题三时间步收敛")
277     dt_list = [60.0, 30.0, 15.0]
278     results = {dt: solve_problem3(dt=dt, dr=DR, t_end=t_end) for dt in dt_list}
279     common_t = np.arange(0.0, t_end + 1e-9, 3600.0)
280     max_c = {}
281     for dt, (t, _, _, C, _, _) in results.items():
282         idx = np rint(common_t / dt).astype(int)
283         max_c[dt] = np.max(C[idx, :], axis=1)
284     e1 = float(np.max(np.abs(max_c[60.0] - max_c[30.0])))
285     e2 = float(np.max(np.abs(max_c[30.0] - max_c[15.0])))
286     print(f"误差: 60/30 s={e1:.6g}, 30/15 s={e2:.6g}, 收敛比={e2 / e1:.4f}")
287     print(f"标准: 第二次误差更小, 收敛比接近 0.5")
288     return e1, e2
289
290
291 def verify_space_step(t_end=21600.0):
292     """比较空间步误差。"""
293     print("\n问题三空间步收敛")
294     dr_list = [0.001, 0.0005, 0.00025]
295     results = {dr: solve_problem3(dt=DT, dr=dr, t_end=t_end) for dr in dr_list}
296     common_r = np.array([0.0, 0.005, 0.01, 0.015, 0.02])
297     final = {}
298     for dr, (_, r, _, C, _, _) in results.items():
299         final[dr] = np.interp(common_r, r, C[-1, :])
300     e1 = float(np.max(np.abs(final[0.001] - final[0.0005])))
301     e2 = float(np.max(np.abs(final[0.0005] - final[0.00025])))
302     print(f"误差: 1/0.5 mm={e1:.6g}, 0.5/0.25 mm={e2:.6g}, 收敛比={e2 / e1:.4f}")
303     print(f"标准: 第二次误差更小, 收敛比接近 0.25")
304     return e1, e2
305
306
307 def verify_conservation(t_end=21600.0):
308     """检查水分守恒。"""
309     print("\n问题三水分守恒")
310     T_func, C_func = get_full_boundary_funcs()
311     t, r, _, C = solve_pde(
312         DT,
313         DR,
314         r_max=R_MAX,
315         t_end=t_end,
316         t_start=0.0,
317         initial_file=None,
318         initial_T=28.0,
319         initial_C=2.55,
320         T_air_func=T_func,
321         C_air_func=C_func,
322     )
323     V, r_surf = _control_volumes(DR, len(r))
324     hm = 8e-7
325     area = 2.0 * np.pi * r_surf
326     M = np.sum(C * V, axis=1)

```

```

327 c_air_next = np.asarray(C_func(t[1:]), dtype=float)
328 flux = hm * area * (c_air_next - C[1:, -1])
329 flux_integral = np.concatenate([[0.0], np.cumsum(flux * DT)])
330 residual = abs((M[-1] - M[0]) - flux_integral[-1]) / max(
331     abs(M[-1] - M[0]), 1e-12
332 )
333 print(f"最终相对残差: {residual:.4%}")
334 print("标准: 水分守恒残差小于 1%")
335 if residual >= 0.01:
336     raise AssertionError("问题三水分守恒未通过")
337 return residual
338
339
340 def plot_problem3_results(t, r, C, drying_time):
341     """绘制含水率曲线和热力图。"""
342     import matplotlib.pyplot as plt
343
344     PIC_DIR.mkdir(parents=True, exist_ok=True)
345     time_h = t / 3600.0
346     target_r = [0.0, 0.5, 1.0, 1.5, 2.0]
347     target_idx = [int(round(x / (r[1] * 100.0))) for x in target_r]
348
349     plt.figure(figsize=(10, 6))
350     for radius, idx in zip(target_r, target_idx):
351         plt.plot(time_h, C[:, idx], label=f"r={radius:g} cm")
352     plt.axhline(MOISTURE_LIMIT, color="red", linestyle="--", label="C=0.15")
353     if drying_time is not None:
354         plt.axvline(drying_time / 3600.0, color="green", linestyle="--", label="结束时间")
355     plt.xlabel("时间 t (h)")
356     plt.ylabel("水分浓度 C (kg/kg)")
357     plt.grid(alpha=0.3)
358     plt.legend()
359     plt.tight_layout()
360     plt.savefig(PIC_DIR / "q3_水分时间演化图.png", dpi=300)
361     plt.close()
362
363     plt.figure(figsize=(10, 6))
364     plt.plot(time_h, np.max(C, axis=1), label="全域最大水分浓度")
365     plt.axhline(MOISTURE_LIMIT, color="red", linestyle="--", label="阈值 0.15")
366     if drying_time is not None:
367         plt.axvline(drying_time / 3600.0, color="green", linestyle="--", label="结束时间")
368     plt.xlabel("时间 t (h)")
369     plt.ylabel("max(C) (kg/kg)")
370     plt.grid(alpha=0.3)
371     plt.legend()
372     plt.tight_layout()
373     plt.savefig(PIC_DIR / "q3_阈值判定图.png", dpi=300)
374     plt.close()
375
376     fig, ax = plt.subplots(figsize=(10, 5))
377     im = ax.imshow(
378         C.T,
379         origin="lower",
380         aspect="auto",
381         extent=[time_h[0], time_h[-1], r[0] * 100.0, r[-1] * 100.0],
382         cmap="Blues_r",
383     )
384     ax.set_xlabel("时间 t (h)")
385     ax.set_ylabel("半径 r (cm)")
386     ax.set_title("水分时空分布")
387     fig.colorbar(im, ax=ax, label="水分浓度 (kg/kg)")
388     fig.tight_layout()
389     fig.savefig(PIC_DIR / "q3_水分热力图.png", dpi=300)
390     plt.close(fig)
391
392
393 if __name__ == "__main__":
394     result = save_problem3_results()
395     plot_problem3_results(result[2], result[3], result[5], result[6])
396     verify_problem3()
397     verify_time_step()
398     verify_space_step()
399     verify_conservation()

```

Listing 7: 问题三全过程固定半径求解程序

2.4 问题四程序

```

1 """问题四: 变半径、变物性热质耦合模型。"""
2

```

```

3 from pathlib import Path
4
5 import matplotlib.pyplot as plt
6 import numpy as np
7 import pandas as pd
8 from scipy.interpolate import PchipInterpolator
9
10 from q3 import get_full_boundary_funcs
11
12
13 BASE_DIR = Path(__file__).resolve().parent
14 INPUT_DIR = BASE_DIR / "input_excel"
15 OUTPUT_DIR = BASE_DIR / "output_excel"
16 PIC_DIR = BASE_DIR / "pic"
17 RADIUS_FILE = INPUT_DIR / "附件2.xlsx"
18 if not RADIUS_FILE.exists():
19     RADIUS_FILE = BASE_DIR / "附件" / "附件2.xlsx"
20
21 MOISTURE_LIMIT = 0.15
22 DT = 60.0
23 DX = 0.05
24 R0 = 0.02
25 MAX_TIME = 259200.0
26 R_MIN = 0.05
27 R_MAX = 10.0
28 T_MIN = -50.0
29 T_MAX = 250.0
30
31
32 def load_radius_data(path=RADIUS_FILE):
33     """读取附件2半径数据。"""
34     path = Path(path)
35     if not path.exists():
36         raise FileNotFoundError(f"未找到半径文件: {path}")
37     df = pd.read_excel(path)
38     if df.shape[1] < 2:
39         raise ValueError("附件2至少需要时间和半径两列")
40     t = df.iloc[:, 0].to_numpy(dtype=float)
41     radius_cm = df.iloc[:, 1].to_numpy(dtype=float)
42     if not (np.all(np.isfinite(t)) and np.all(np.isfinite(radius_cm))):
43         raise ValueError("附件2含有NaN或无穷值")
44     if np.any(np.diff(t) <= 0) or np.any(radius_cm <= 0):
45         raise ValueError("附件2时间须递增且半径须为正")
46     if t[0] > 1e-12 or t[-1] < MAX_TIME - 1e-12:
47         raise ValueError("附件2必须覆盖0~259200 s")
48     if np.any(np.diff(radius_cm) > 1e-10):
49         raise ValueError("附件2半径应单调不减")
50     return t, radius_cm / 100.0
51
52
53 def interpolate_radius(dt=DT, t_end=MAX_TIME, path=RADIUS_FILE):
54     """PCHIP插值到计算时间网格。"""
55     t_raw, r_raw = load_radius_data(path)
56     if dt <= 0 or t_end < 0:
57         raise ValueError("时间参数必须为正")
58     n = int(round(t_end / dt))
59     t = np.arange(n + 1, dtype=float) * dt
60     if t[-1] > t_raw[-1] + 1e-9:
61         raise ValueError("插值终点超过附件2范围")
62     pchip = PchipInterpolator(t_raw, r_raw)
63     r = np.asarray(pchip(t), dtype=float)
64     # PCHIP理论上保持单调, 浮点误差只做微小修正。
65     r = np.minimum.accumulate(r)
66     if np.any(r <= 0):
67         raise ValueError("插值后出现非正半径")
68     return t, r, t_raw, r_raw
69
70
71 def save_radius_interpolation():
72     """保存插值表和插值图。"""
73     t, r, t_raw, r_raw = interpolate_radius()
74     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
75     pd.DataFrame({"时间/s": t.astype(int), "半径/m": r, "半径/cm": r * 100.0}).to_excel(
76         OUTPUT_DIR / "q4_radius_interp.xlsx", index=False
77     )
78     PIC_DIR.mkdir(parents=True, exist_ok=True)
79     fig, ax = plt.subplots(figsize=(9, 5))
80     ax.plot(t / 3600.0, r * 100.0, label="PCHIP插值")
81     ax.scatter(t_raw / 3600.0, r_raw * 100.0, s=16, label="附件2观测")
82     ax.set_xlabel("时间/h")
83     ax.set_ylabel("半径/cm")
84     ax.set_title("附件2半径数据的PCHIP插值")
85     ax.grid(alpha=0.3)
86     ax.legend()
87     fig.tight_layout()

```

```

88     fig.savefig(PIC_DIR / "q4_半径插值图.png", dpi=300)
89     plt.close(fig)
90     return t, r
91
92
93 # 附录4物性公式
94 def rho_of_C(C):
95     return 760.0 + 90.0 * C
96
97
98 def cp_of_C(C):
99     C_safe = np.maximum(C, 1e-8)
100     return 1850.0 + 2150.0 * C_safe / (C_safe + 1.0)
101
102
103 def k_of_C(C):
104     C_safe = np.maximum(C, 1e-8)
105     return 0.12 + 0.20 * C_safe / (C_safe + 1.0)
106
107
108 def D_of_CT(C, T):
109     C_safe = np.clip(C, 0.05, 10.0)
110     T_K = np.clip(T + 273.15, 273.15, 373.15)
111     D = 4.2e-4 * np.exp(-0.30 / C_safe) * np.exp(-3850.0 / T_K)
112     return np.clip(D, 1e-14, 1e-3)
113
114
115 def solve_tridiag(a, b, c, d):
116     """追赶法。"""
117     n = len(b)
118     if n == 0:
119         return np.empty(0)
120     cp = np.zeros(n)
121     dp = np.zeros(n)
122     if abs(b[0]) < 1e-14:
123         raise ZeroDivisionError("三对角矩阵首行奇异")
124     cp[0] = c[0] / b[0]
125     dp[0] = d[0] / b[0]
126     for i in range(1, n):
127         den = b[i] - a[i] * cp[i - 1]
128         if abs(den) < 1e-14:
129             raise ZeroDivisionError(f"三对角矩阵第{i}行奇异")
130         if i < n - 1:
131             cp[i] = c[i] / den
132             dp[i] = (d[i] - a[i] * dp[i - 1]) / den
133     x = np.zeros(n)
134     x[-1] = dp[-1]
135     for i in range(n - 2, -1, -1):
136         x[i] = dp[i] - cp[i] * x[i + 1]
137     return x
138
139
140 def _assemble_scalar(old, coeff, storage, radius, dt, dx,
141                      boundary_value, boundary_coeff):
142     """组装固定ε控制体隐式方程。"""
143     nr = len(old)
144     x = np.linspace(0.0, 1.0, nr)
145     w = np.zeros(nr)
146     w[0] = (dx / 2.0) ** 2
147     w[1:-1] = 2.0 * x[1:-1] * dx
148     w[-1] = 1.0 - (1.0 - dx / 2.0) ** 2
149     a = np.zeros(nr)
150     b = np.ones(nr)
151     c = np.zeros(nr)
152     d = old.copy()
153     # 内部扩散通量, 方程已变换到ε坐标。
154     face_x = (np.arange(nr - 1) + 0.5) * dx
155     for f in range(nr - 1):
156         xi_face = face_x[f]
157         left, right = f, f + 1
158         # F=ε q (u_right-u_left)/dx; 两侧控制体共享同一F。
159         g_left = 2.0 * dt * xi_face * coeff[f] / (radius**2 * storage[left] * w[left] * dx)
160         g_right = 2.0 * dt * xi_face * coeff[f] / (radius**2 * storage[right] * w[right] * dx)
161         c[left] -= g_left
162         b[left] += g_left
163         a[right] -= g_right
164         b[right] += g_right
165     # 表面对流项: -q/R= h(u_s-u_air)。
166     g_conv = 2.0 * dt * boundary_coeff / (radius * storage[-1] * w[-1])
167     b[-1] += g_conv
168     d[-1] += g_conv * boundary_value
169     return a, b, c, d
170
171
172 def solve_problem4(dt=DT, dx=DX, t_end=MAX_TIME, stop_at_threshold=True):

```

```

173     """求解变半径全过程，返回无量纲径向网格上的场。"""
174     if dt <= 0 or dx <= 0 or t_end < 0:
175         raise ValueError("时间和空间步长必须为正")
176     n_interval = int(round(1.0 / dx))
177     if not np.isclose(n_interval * dx, 1.0, atol=1e-12, rtol=0.0):
178         raise ValueError("dx 必须能整除无量纲区间 [0,1]")
179     nr = n_interval + 1
180     x = np.linspace(0.0, 1.0, nr)
181     t, radius, _, _ = interpolate_radius(dt=dt, t_end=t_end)
182     T_air_func, C_air_func = get_full_boundary_funcs()
183     T = np.zeros((len(t), nr))
184     C = np.zeros((len(t), nr))
185     T[0, :] = 28.0
186     C[0, :] = 2.55
187     max_C = np.zeros(len(t))
188     max_C[0] = np.max(C[0])
189     hit = None
190
191     for n in range(len(t) - 1):
192         Rnext = radius[n + 1]
193         T_air = float(T_air_func(t[n + 1]))
194         C_air = float(C_air_func(t[n + 1]))
195         C_mid = 0.5 * (C[n, :-1] + C[n, 1:])
196         T_mid = 0.5 * (T[n, :-1] + T[n, 1:])
197         k_face = k_of_C(C_mid)
198         D_face = D_of_CT(C_mid, T_mid)
199         H = rho_of_C(C[n]) * cp_of_C(C[n])
200
201         aT, bT, cT, dT = _assemble_scalar(
202             T[n], k_face, H, Rnext, dt, dx, T_air, 25.0,
203         )
204         # 温度存储量为 rho*cp。
205         T[n + 1] = solve_tridiag(aT, bT, cT, dT)
206
207         aC, bC, cC, dC = _assemble_scalar(
208             C[n], D_face, np.ones(nr), Rnext, dt, dx, C_air, 8.0e-7,
209         )
210         C[n + 1] = solve_tridiag(aC, bC, cC, dC)
211         T[n + 1] = np.clip(T[n + 1], T_MIN, T_MAX)
212         C[n + 1] = np.clip(C[n + 1], 0.0, R_MAX)
213         max_C[n + 1] = np.max(C[n + 1])
214         if hit is None and max_C[n + 1] <= MOISTURE_LIMIT:
215             hit = n + 1
216             if stop_at_threshold:
217                 break
218
219     n_used = len(t) if (hit is None or not stop_at_threshold) else hit + 1
220     t = t[:n_used]
221     radius = radius[:n_used]
222     T = T[:n_used]
223     C = C[:n_used]
224     max_C = max_C[:n_used]
225     drying_time = None
226     if hit is not None:
227         if hit == 0:
228             drying_time = 0.0
229         else:
230             drying_time = float(
231                 t[hit - 1] + (MOISTURE_LIMIT - max_C[hit - 1])
232                 * (t[hit] - t[hit - 1]) / (max_C[hit] - max_C[hit - 1])
233             )
234     return t, radius, x, T, C, max_C, drying_time
235
236
237 def _append_interpolated_end(t, radius, T, C, max_C, drying_time):
238     """追加阈值线性插值。"""
239     if drying_time is None or drying_time >= t[-1] - 1e-12:
240         return t, radius, T, C, max_C
241     j = len(t) - 1
242     t0, t1 = t[j - 1], t[j]
243     f = (drying_time - t0) / (t1 - t0)
244     t = np.append(t[:-1], drying_time)
245     radius = np.append(radius[:-1], radius[j - 1] + f * (radius[j] - radius[j - 1]))
246     T = np.vstack([T[:-1], T[j - 1] + f * (T[j] - T[j - 1])])
247     C = np.vstack([C[:-1], C[j - 1] + f * (C[j] - C[j - 1])])
248     max_C = np.append(max_C[:-1], np.max(C[-1]))
249     return t, radius, T, C, max_C
250
251
252 def _physical_table(t, radius, x, C):
253     """按物理距离输出，域外位置留空。"""
254     target_cm = np.round(np.arange(0.0, 2.0 + 1e-12, 0.1), 1)
255     rows = []
256     for n, time in enumerate(t):
257         row = {"时间/s": round(float(time), 2)}

```

```

258     r_cm = radius[n] * 100.0
259     r_nodes = x * radius[n] * 100.0
260     for p in target_cm:
261         key = f"{p:.1f}"
262         row[key] = round(float(np.interp(p, r_nodes, C[n])), 4) if p <= r_cm + 1e-10 else None
263     row["药材表面"] = round(float(C[n, -1]), 4)
264     row["半径/cm"] = round(float(r_cm), 4)
265     rows.append(row)
266     return pd.DataFrame(rows, columns=["时间/s"] + [f"{p:.1f}" for p in target_cm] + ["药材表面", "半径/cm"])
267
268
269 def _summary_table(t, radius, x, C, drying_time):
270     """生成表6摘要。"""
271     regular = np.arange(6.0, np.floor(t[-1] / 21600.0) * 6.0 + 1e-12, 6.0) * 3600.0
272     targets = list(regular)
273     if drying_time is not None and (not targets or drying_time > targets[-1] + 1e-8):
274         targets.append(drying_time)
275     target_cm = [round(v, 1) for v in np.arange(0.0, 2.0 + 1e-12, 0.5)]
276     rows = []
277     for target in targets:
278         idx = int(np.argmin(np.abs(t - target)))
279         r_cm = radius[idx] * 100.0
280         r_nodes = x * radius[idx] * 100.0
281         row = {"时间/h": round(float(t[idx] / 3600.0), 4), "半径/cm": round(float(r_cm), 4)}
282         for p in target_cm:
283             row[f"{p:.1f}"] = round(float(np.interp(p, r_nodes, C[idx])), 4) if p <= r_cm + 1e-10 else None
284         row["药材表面"] = round(float(C[idx, -1]), 4)
285         rows.append(row)
286     cols = ["时间/h"] + [f"{p:.1f}" for p in target_cm] + ["药材表面", "半径/cm"]
287     return pd.DataFrame(rows, columns=cols)
288
289
290 def save_results():
291     """求解并保存第四问结果。"""
292     t, radius, x, T, C, max_C, drying_time = solve_problem4()
293     t, radius, T, C, max_C = _append_interpolated_end(t, radius, T, C, max_C, drying_time)
294     full = _physical_table(t, radius, x, C)
295     summary = _summary_table(t, radius, x, C, drying_time)
296     normalized = pd.DataFrame(np.round(C, 4), columns=[f"xi={v:.2f}" for v in x])
297     normalized.insert(0, "时间/s", np.round(t, 2))
298     normalized["半径/cm"] = np.round(radius * 100.0, 4)
299     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
300     with pd.ExcelWriter(OUTPUT_DIR / "result4.xlsx") as writer:
301         full.to_excel(writer, sheet_name="水分浓度", index=False)
302         summary.to_excel(writer, sheet_name="表6摘要", index=False)
303         normalized.to_excel(writer, sheet_name="无量纲网格", index=False)
304         pd.DataFrame({"时间/s": np.round(t, 2), "半径/cm": np.round(radius * 100.0, 6)}).to_excel(
305             writer, sheet_name="半径插值", index=False)
306     )
307     _plot_results(t, radius, x, C, max_C, drying_time)
308     print(f"第四问结果已保存: {OUTPUT_DIR / 'result4.xlsx'}")
309     if drying_time is None:
310         print("最大计算时长内未达到水分阈值")
311     else:
312         print(f"第四问烘干时间: {drying_time:.4f} s = {drying_time / 3600.0:.5f} h")
313     return t, radius, x, T, C, max_C, drying_time
314
315
316 def _plot_results(t, radius, x, C, max_C, drying_time):
317     """绘制第四问结果图。"""
318     PIC_DIR.mkdir(parents=True, exist_ok=True)
319     fig, ax = plt.subplots(figsize=(9, 5))
320     ax.plot(t / 3600.0, max_C, label="全域最大值")
321     ax.plot(t / 3600.0, C[:, 0], label="中心")
322     ax.plot(t / 3600.0, C[:, -1], label="表面")
323     ax.axhline(MOISTURE_LIMIT, color="black", linestyle="--", label="阈值0.15")
324     if drying_time is not None:
325         ax.axvline(drying_time / 3600.0, color="tab:red", linestyle=":", label=f"结束({drying_time / 3600.0:.3f} h)")
326     ax.set_xlabel("时间/h")
327     ax.set_ylabel("水分浓度")
328     ax.set_title("问题四水分浓度与烘干阈值")
329     ax.grid(alpha=0.3)
330     ax.legend()
331     fig.tight_layout()
332     fig.savefig(PIC_DIR / "q4_阈值判定图.png", dpi=300)
333     plt.close(fig)
334
335     fig, ax = plt.subplots(figsize=(9, 5))
336     mesh_t, mesh_x = np.meshgrid(t / 3600.0, x, indexing="ij")
337     im = ax.pcolormesh(mesh_t, mesh_x, C, shading="auto", cmap="viridis")
338     fig.colorbar(im, ax=ax, label="水分浓度")
339     ax.set_xlabel("时间/h")
340     ax.set_ylabel("无量纲半径  $\xi=r/R(t)$ ")
341     ax.set_title("问题四水分浓度时空分布")
342     fig.tight_layout()

```

```

343 fig.savefig(PIC_DIR / "q4_水分热力图.png", dpi=300)
344 plt.close(fig)
345
346
347 def validate_q4():
348     """第四问基本校验。"""
349     result = solve_problem4(stop_at_threshold=False)
350     t, radius, x, T, C, max_C, drying_time = result
351     if not (np.all(np.isfinite(T)) and np.all(np.isfinite(C)) and np.all(C >= -1e-12)):
352         raise AssertionError("第四问出现非有限值或负水分")
353     if np.any(np.diff(radius) > 1e-12):
354         raise AssertionError("第四问半径未保持单调不减")
355     # 收缩初期允许表面压缩造成的微小回升, 恒温段应单调下降。
356     stable_idx = np.searchsorted(t, 5500.0)
357     if np.any(np.diff(max_C[stable_idx:]) > 1e-7):
358         raise AssertionError("第四问恒温段全域最大水分不单调")
359     if drying_time is None:
360         hit = np.flatnonzero(max_C <= MOISTURE_LIMIT)
361         if len(hit):
362             raise AssertionError("第四问阈值时间记录异常")
363     # 检查ε坐标下的水分守恒。
364     dt = float(t[1] - t[0]) if len(t) > 1 else DT
365     w = np.zeros(len(x))
366     w[0] = (DX / 2.0) ** 2
367     w[1:-1] = 2.0 * x[1:-1] * DX
368     w[-1] = 1.0 - (1.0 - DX / 2.0) ** 2
369     _, C_air = get_full_boundary_funcs()
370     air = np.asarray(C_air(t[1:]), dtype=float)
371     flux = 2.0 * dt * 8.0e-7 / radius[1:] * (air - C[1:, -1])
372     mass_res = float(np.max(np.abs(np.sum((C[1:] - C[:-1]) * w, axis=1) - flux)))
373     if mass_res > 1e-10:
374         raise AssertionError(f"第四问水分守恒残差过大: {mass_res:.3e}")
375     return {"time_end_s": float(t[-1]), "radius_start_cm": float(radius[0] * 100),
376           "radius_end_cm": float(radius[-1] * 100), "C_min": float(C.min()),
377           "C_max": float(C.max()), "drying_time_s": drying_time,
378           "moisture_balance_abs": mass_res}
379
380
381 if __name__ == "__main__":
382     save_radius_interpolation()
383     save_results()

```

Listing 8: 问题四变半径求解程序

2.5 统一验证程序

```

1 """四问结果的独立重算、收敛、守恒和物理范围交叉验证。"""
2
3 from __future__ import annotations
4
5 import json
6 from pathlib import Path
7
8 import matplotlib.pyplot as plt
9 import numpy as np
10 import pandas as pd
11
12 from q1_3 import get_boundary_funcs as get_q1_boundary
13 from q1_3 import solve_pde as solve_q1
14 from q2_3 import (
15     DEFAULT_INITIAL_FILE,
16     INITIAL_TIME,
17     OUTPUT_DIR,
18     get_boundary_funcs as get_q2_boundary,
19     load_initial_profile,
20     solve_pde as solve_q2,
21 )
22 from q3 import MOISTURE_LIMIT, solve_problem3
23 import q4
24
25
26 BASE_DIR = Path(__file__).resolve().parent
27 PIC_DIR = BASE_DIR / "pic"
28 REPORT_FILE = OUTPUT_DIR / "validation_report.json"
29 R_MAX = 0.02
30
31 plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Arial Unicode MS"]
32 plt.rcParams["axes.unicode_minus"] = False
33
34

```

```

35 def control_volumes(dr: float, nr: int):
36     """返回单位长度控制体体积和表面积。"""
37     v = np.zeros(nr)
38     v[0] = np.pi * (dr / 2.0) ** 2
39     for i in range(1, nr - 1):
40         v[i] = 2.0 * np.pi * (i * dr) * dr
41     r_surface = (nr - 1) * dr
42     v[-1] = np.pi * (r_surface**2 - (r_surface - dr / 2.0) ** 2)
43     return v, 2.0 * np.pi * r_surface
44
45
46 def read_sheet(path: Path, sheet_index: int = 0) -> pd.DataFrame:
47     book = pd.ExcelFile(path)
48     return pd.read_excel(path, sheet_name=book.sheet_names[sheet_index])
49
50
51 def add_check(report: dict, name: str, passed: bool, value=None, criterion: str = ""):
52     item = {"passed": bool(passed), "criterion": criterion}
53     if value is not None:
54         item["value"] = value
55     report["checks"][name] = item
56     if not passed:
57         raise AssertionError(f"验证未通过: {name}; value={value}; criterion={criterion}")
58
59
60 def validate_file_structure(report: dict):
61     """检查结果文件的工作表、网格、时间和有限值。"""
62     expected = {
63         "result1.xlsx": (2, 1801, 0.0, 1800.0, 1.0),
64         "result2.xlsx": (2, 10801, 0.0, 10800.0, 1.0),
65     }
66     for name, (n_sheet, n_rows, t0, t1, step) in expected.items():
67         path = OUTPUT_DIR / name
68         book = pd.ExcelFile(path)
69         ok_sheets = len(book.sheet_names) == n_sheet
70         add_check(report, f"{name}_工作表", ok_sheets, book.sheet_names, f"工作表数={n_sheet}")
71         for j in range(n_sheet):
72             df = pd.read_excel(path, sheet_name=book.sheet_names[j])
73             times = df.iloc[:, 0].to_numpy(dtype=float)
74             values = df.iloc[:, 1:].to_numpy(dtype=float)
75             radii = list(df.columns[1:])
76             ok = (
77                 df.shape == (n_rows, 22)
78                 and np.isclose(times[0], t0)
79                 and np.isclose(times[-1], t1)
80                 and np.allclose(np.diff(times), step)
81                 and radii == [f"{i / 10:.1f}" for i in range(21)]
82                 and np.all(np.isfinite(values))
83                 and np.allclose(values, np.round(values, 4))
84             )
85             add_check(
86                 report,
87                 f"{name}_表{j + 1}_结构",
88                 ok,
89                 {"shape": list(df.shape), "time": [float(times[0]), float(times[-1])]},
90                 "22列; 时间连续; 半径0.0~2.0 cm; 四位小数; 无NaN",
91             )
92
93     path = OUTPUT_DIR / "result3.xlsx"
94     df = read_sheet(path)
95     times = df.iloc[:, 0].to_numpy(dtype=float)
96     values = df.iloc[:, 1:].to_numpy(dtype=float)
97     ok = (
98         df.shape == (3512, 22)
99         and np.isclose(times[0], 0.0)
100         and np.isclose(times[-1], 210634.74, atol=0.02)
101         and np.all(np.diff(times[:-1]) == 60.0)
102         and np.all(np.isfinite(values))
103         and np.allclose(values, np.round(values, 4))
104     )
105     add_check(
106         report,
107         "result3_结构",
108         ok,
109         {"shape": list(df.shape), "first_time_s": float(times[0]), "last_time_s": float(times[-1])},
110         "0~烘干结束; 60 s网格并追加插值结束行; 无NaN; 四位小数",
111     )
112
113     summary = read_sheet(OUTPUT_DIR / "result3_summary.xlsx")
114     summary_t = summary.iloc[:, 0].to_numpy(dtype=float)
115     ok = (
116         summary.shape == (10, 6)
117         and np.allclose(summary_t[:-1], np.arange(6.0, 55.0, 6.0))
118         and np.isclose(summary_t[-1], 210634.74 / 3600.0, atol=1e-4)
119         and np.all(np.isfinite(summary.iloc[:, 1:].to_numpy(dtype=float)))

```

```

120 )
121 add_check(report, "result3_摘要结构", ok, summary_t.tolist(), "每6 h及结束时刻")
122
123 profile = read_sheet(OUTPUT_DIR / "output3.xlsx")
124 ok = profile.shape == (21, 3) and np.all(np.isfinite(profile.iloc[:, 1:].to_numpy(dtype=float)));
125 add_check(report, "output3_结构", ok, list(profile.shape), "21个径向节点, 温度和水分均有限")
126
127 path = OUTPUT_DIR / "result4.xlsx"
128 book = pd.ExcelFile(path)
129 full = pd.read_excel(path, sheet_name=book.sheet_names[0])
130 normalized = pd.read_excel(path, sheet_name=book.sheet_names[2])
131 radius = pd.read_excel(path, sheet_name=book.sheet_names[3])
132 times = full.iloc[:, 0].to_numpy(dtype=float)
133 radii = full.iloc[:, -1].to_numpy(dtype=float)
134 values = full.iloc[:, 1:22].to_numpy(dtype=float)
135 physical_ok = True
136 target_cm = np.arange(0.0, 2.0 + 1e-12, 0.1)
137 for i in range(len(full)):
138     inside = target_cm <= radii[i] + 1e-8
139     physical_ok &= np.all(np.isfinite(values[i, inside])) and np.all(~np.isfinite(values[i, ~inside]))
140 ok = (
141     len(book.sheet_names) == 4
142     and full.shape[1] == 24
143     and normalized.shape[1] == 23
144     and radius.shape[1] == 2
145     and np.isclose(times[0], 0.0)
146     and np.isclose(times[-1], 184815.74, atol=0.02)
147     and np.all(np.diff(times) > 0)
148     and physical_ok
149     and np.all(np.isfinite(normalized.iloc[:, 1:].to_numpy(dtype=float)))
150     and np.all(np.isfinite(radius.iloc[:, 1:].to_numpy(dtype=float)))
151 )
152 add_check(report, "result4_结构", ok,
153     {"sheets": book.sheet_names, "shape": list(full.shape),
154      "first_time_s": float(times[0]), "last_time_s": float(times[-1])},
155     "变半径水分表、表6摘要、无量纲网格和半径插值均存在; 域外物理位置留空")
156
157
158 def q1_recompute_and_balance(report: dict):
159     """独立重算问题一, 并检查全局能量和水分守恒。"""
160     t_func, c_func = get_q1_boundary()
161     t, r, temp, moist = solve_q1(1.0, 0.001, t_end=1800.0, T_air_func=t_func, C_air_func=c_func)
162     book = pd.ExcelFile(OUTPUT_DIR / "result1.xlsx")
163     stored_t = read_sheet(OUTPUT_DIR / "result1.xlsx", 0).iloc[:, 1:].to_numpy(dtype=float)
164     stored_c = read_sheet(OUTPUT_DIR / "result1.xlsx", 1).iloc[:, 1:].to_numpy(dtype=float)
165     err_t = float(np.max(np.abs(np.round(temp, 4) - stored_t)))
166     err_c = float(np.max(np.abs(np.round(moist, 4) - stored_c)))
167     add_check(report, "问题一_独立重算", err_t <= 1e-12 and err_c <= 1e-12,
168         {"T_max_abs": err_t, "C_max_abs": err_c}, "重算结果与Excel四位小数结果一致")
169
170     v, area = control_volumes(0.001, len(r))
171     rho, cp, h, hm = 820.0, 2600.0, 25.0, 8e-7
172     air_t = np.asarray(t_func(t), dtype=float)
173     air_c = np.asarray(c_func(t), dtype=float)
174     q_step = h * area * (air_t[1:] - temp[:-1, -1])
175     m_step = hm * area * (air_c[1:] - moist[:-1, -1])
176     storage_q = np.sum(rho * cp * (temp[1:] - temp[:-1]) * v, axis=1)
177     storage_m = np.sum(moist[1:] - moist[:-1]) * v, axis=1)
178     q_res = abs(np.sum(storage_q) - np.sum(q_step)) / max(abs(np.sum(storage_q)), 1e-12)
179     m_res = abs(np.sum(storage_m) - np.sum(m_step)) / max(abs(np.sum(storage_m)), 1e-12)
180     add_check(report, "问题一_守恒", q_res < 0.01 and m_res < 0.01,
181         {"energy_relative": q_res, "moisture_relative": m_res}, "两项相对残差<1%")
182     return {"t": t, "r": r, "T": temp, "C": moist, "q_res": q_res, "m_res": m_res}
183
184
185 def q2_recompute_and_balance(report: dict):
186     """独立重算问题二, 并检查5500 s断点、守恒和物理范围。"""
187     t_func, c_func = get_q2_boundary(t_start=INITIAL_TIME)
188     t, r, temp, moist = solve_q2(
189         1.0, 0.001, t_end=10800.0, t_start=INITIAL_TIME,
190         initial_file=DEFAULT_INITIAL_FILE, T_air_func=t_func, C_air_func=c_func,
191     )
192     stored_t = read_sheet(OUTPUT_DIR / "result2.xlsx", 0).iloc[:, 1:].to_numpy(dtype=float)
193     stored_c = read_sheet(OUTPUT_DIR / "result2.xlsx", 1).iloc[:, 1:].to_numpy(dtype=float)
194     err_t = float(np.max(np.abs(np.round(temp, 4) - stored_t)))
195     err_c = float(np.max(np.abs(np.round(moist, 4) - stored_c)))
196     add_check(report, "问题二_独立重算", err_t <= 1e-12 and err_c <= 1e-12,
197         {"T_max_abs": err_t, "C_max_abs": err_c}, "重算结果与Excel四位小数结果一致")
198
199     _, t0, c0 = load_initial_profile(DEFAULT_INITIAL_FILE, r_max=R_MAX, Nr=len(r))
200     add_check(report, "问题二_初始剖面", np.max(abs(temp[0] - t0)) <= 1e-12 and np.max(abs(moist[0] - c0)) <= 1e-12,
201         {"T_max_abs": float(np.max(abs(temp[0] - t0))), "C_max_abs": float(np.max(abs(moist[0] - c0)))},
202         "t=0行等于output3.xlsx")
203
204     v, area = control_volumes(0.001, len(r))

```

```

205 h, hm = 25.0, 8e-7
206 air_t = np.asarray(t_func(INITIAL_TIME + t), dtype=float)
207 air_c = np.asarray(c_func(INITIAL_TIME + t), dtype=float)
208 rho = 650.0 + 128.0 * moist[:-1]
209 cp = 1450.0 + 2736.0 * (moist[:-1] / (moist[:-1] + 1.0))
210 q_step = h * area * (air_t[1:] - temp[1:, -1])
211 m_step = hm * area * (air_c[1:] - moist[1:, -1])
212 storage_q = np.sum(rho * cp * (temp[1:] - temp[:-1]) * v, axis=1)
213 storage_m = np.sum((moist[1:] - moist[:-1]) * v, axis=1)
214 q_res = abs(np.sum(storage_q) - np.sum(q_step)) / max(abs(np.sum(storage_q)), 1e-12)
215 m_res = abs(np.sum(storage_m) - np.sum(m_step)) / max(abs(np.sum(storage_m)), 1e-12)
216 finite = bool(np.all(np.isfinite(temp)) and np.all(np.isfinite(moist)))
217 physical = finite and bool(np.all(moist >= -1e-12)) and bool(np.all(temp > -50.0)) and bool(np.all(temp < 250.0))
218 add_check(report, "问题二_守恒", q_res < 0.01 and m_res < 0.01,
219           {"energy_relative": q_res, "moisture_relative": m_res}, "两项相对残差<1%")
220 add_check(report, "问题二_物理范围", physical,
221           {"T_min": float(temp.min()), "T_max": float(temp.max()), "C_min": float(moist.min()), "C_max": float(moist.max())},
222           "有限值, C>=0且未触及截断")
223 return {"t": t, "r": r, "T": temp, "C": moist, "q_res": q_res, "m_res": m_res}
224
225
226 def convergence_q1(report: dict):
227     tf, cf = get_q1_boundary()
228     dt_list = [1.0, 0.5, 0.25]
229     sol = {dt: solve_q1(dt, 0.001, t_end=1800.0, T_air_func=tf, C_air_func=cf) for dt in dt_list}
230     common_t = np.arange(0.0, 1800.0 + 1e-9, 100.0)
231     vals = {dt: (x[2][np rint(common_t / dt).astype(int)], x[3][np rint(common_t / dt).astype(int)]) for dt, x in sol.items()}
232     e_t = [float(np.max(abs(vals[1.0][0] - vals[0.5][0])), float(np.max(abs(vals[0.5][0] - vals[0.25][0])))]
233     e_c = [float(np.max(abs(vals[1.0][1] - vals[0.5][1])), float(np.max(abs(vals[0.5][1] - vals[0.25][1])))]
234     sf = {"dt": [0.5, 0.25], "T_error": e_t, "C_error": e_c, "T_ratio": e_t[1] / e_t[0], "C_ratio": e_c[1] / e_c[0]}
235     add_check(report, "问题一_时间收敛", 0.35 < sf["T_ratio"] < 0.65 and 0.35 < sf["C_ratio"] < 0.65 and e_t[1] < e_t[0] and e_c[1] < e_c[0], sf,
236           "误差递减, 收敛比接近0.5")
237
238     dr_list = [0.001, 0.0005, 0.00025]
239     sol = {dr: solve_q1(0.05, dr, t_end=1800.0, T_air_func=tf, C_air_func=cf) for dr in dr_list}
240     common_r = np.array([0.0, 0.005, 0.01, 0.015, 0.02])
241     final = {dr: (np.interp(common_r, x[1], x[2][-1]), np.interp(common_r, x[1], x[3][-1])) for dr, x in sol.items()}
242     e_t = [float(np.max(abs(final[0.001][0] - final[0.0005][0])), float(np.max(abs(final[0.0005][0] - final[0.00025][0])))]
243     e_c = [float(np.max(abs(final[0.001][1] - final[0.0005][1])), float(np.max(abs(final[0.0005][1] - final[0.00025][1])))]
244     sf2 = {"dr": [0.0005, 0.00025], "T_error": e_t, "C_error": e_c, "T_ratio": e_t[1] / e_t[0], "C_ratio": e_c[1] / e_c[0]}
245     add_check(report, "问题一_空间收敛", 0.15 < sf2["T_ratio"] < 0.35 and 0.15 < sf2["C_ratio"] < 0.35 and e_t[1] < e_t[0] and e_c[1] < e_c[0],
246           sf2, "误差递减, 收敛比接近0.25")
247     return {"time": sf, "space": sf2}
248
249
250 def convergence_q2(report: dict):
251     tf, cf = get_q2_boundary(t_start=INITIAL_TIME)
252     dt_list = [1.0, 0.5, 0.25]
253     sol = {dt: solve_q2(dt, 0.001, t_end=1800.0, t_start=INITIAL_TIME, initial_file=DEFAULT_INITIAL_FILE, T_air_func=tf, C_air_func=cf) for dt
254             in dt_list}
255     common_t = np.arange(0.0, 1800.0 + 1e-9, 100.0)
256     vals = {dt: (x[2][np rint(common_t / dt).astype(int)], x[3][np rint(common_t / dt).astype(int)]) for dt, x in sol.items()}
257     e_t = [float(np.max(abs(vals[1.0][0] - vals[0.5][0])), float(np.max(abs(vals[0.5][0] - vals[0.25][0])))]
258     e_c = [float(np.max(abs(vals[1.0][1] - vals[0.5][1])), float(np.max(abs(vals[0.5][1] - vals[0.25][1])))]
259     sf = {"dt": [0.5, 0.25], "T_error": e_t, "C_error": e_c, "T_ratio": e_t[1] / e_t[0], "C_ratio": e_c[1] / e_c[0]}
260     add_check(report, "问题二_空间收敛", 0.15 < sf["T_ratio"] < 0.35 and 0.15 < sf["C_ratio"] < 0.35 and e_t[1] < e_t[0] and e_c[1] < e_c[0], sf,
261           "误差递减, 收敛比接近0.5")
262
263     dr_list = [0.001, 0.0005, 0.00025]
264     sol = {dr: solve_q2(1.0, dr, t_end=1800.0, t_start=INITIAL_TIME, initial_file=DEFAULT_INITIAL_FILE, T_air_func=tf, C_air_func=cf) for dr in
265             dr_list}
266     common_r = np.array([0.0, 0.005, 0.01, 0.015, 0.02])
267     final = {dr: (np.interp(common_r, x[1], x[2][-1]), np.interp(common_r, x[1], x[3][-1])) for dr, x in sol.items()}
268     e_t = [float(np.max(abs(final[0.001][0] - final[0.0005][0])), float(np.max(abs(final[0.0005][0] - final[0.00025][0])))]
269     e_c = [float(np.max(abs(final[0.001][1] - final[0.0005][1])), float(np.max(abs(final[0.0005][1] - final[0.00025][1])))]
270     sf2 = {"dr": [0.0005, 0.00025], "T_error": e_t, "C_error": e_c, "T_ratio": e_t[1] / e_t[0], "C_ratio": e_c[1] / e_c[0]}
271     add_check(report, "问题二_时间收敛", 0.15 < sf2["T_ratio"] < 0.35 and 0.15 < sf2["C_ratio"] < 0.35 and e_t[1] < e_t[0] and e_c[1] < e_c[0],
272           sf2, "误差递减, 收敛比接近0.25")
273     return {"time": sf, "space": sf2}
274
275
276 def convergence_q3(report: dict):
277     dt_list = [60.0, 30.0, 15.0]
278     sol = {dt: solve_problem3(dt=dt, dr=0.001, t_end=21600.0) for dt in dt_list}
279     common_t = np.arange(0.0, 21600.0 + 1e-9, 3600.0)
280     maxc = {dt: np.max(x[3][np rint(common_t / dt).astype(int)], axis=1) for dt, x in sol.items()}
281     e_t = [float(np.max(abs(maxc[60.0] - maxc[30.0])), float(np.max(abs(maxc[30.0] - maxc[15.0])))]
282     sf = {"dt": [30.0, 15.0], "maxC_error": e_t, "ratio": e_t[1] / e_t[0]}
283     add_check(report, "问题三_时间收敛", 0.35 < sf["ratio"] < 0.65 and e_t[1] < e_t[0], sf, "误差递减, 收敛比接近0.5")
284
285     dr_list = [0.001, 0.0005, 0.00025]
286     sol = {dr: solve_problem3(dt=60.0, dr=dr, t_end=21600.0) for dr in dr_list}
287     common_r = np.array([0.0, 0.005, 0.01, 0.015, 0.02])
288     final = {dr: np.interp(common_r, x[1], x[3][-1]) for dr, x in sol.items()}
289     e_s = [float(np.max(abs(final[0.001] - final[0.0005])), float(np.max(abs(final[0.0005] - final[0.00025])))]

```

```

284 sf2 = {"dr": [0.0005, 0.00025], "maxC_error": e_s, "ratio": e_s[1] / e_s[0]}
285 add_check(report, "问题三_空间收敛", 0.15 < sf2["ratio"] < 0.35 and e_s[1] < e_s[0], sf2, "误差递减, 收敛比接近0.25")
286 return {"time": sf, "space": sf2}
287
288
289 def q3_full(report: dict):
290     """复核问题三全过程、阈值和最终表。"""
291     t, r, temp, moist, drying_time, max_c = solve_problem3()
292     stored = read_sheet(OUTPUT_DIR / "result3.xlsx").iloc[:, 1:].to_numpy(dtype=float)
293     err = float(np.max(np.abs(np.round(moist[: stored.shape[0] - 1], 4) - stored[: -1])))
294     add_check(report, "问题三_独立重算", err <= 1e-12, err, "60 s网格重算与Excel一致")
295     j = int(np.flatnonzero(max_c <= MOISTURE_LIMIT)[0])
296     before, after = float(max_c[j - 1]), float(max_c[j])
297     frac = (drying_time - t[j - 1]) / (t[j] - t[j - 1])
298     c_end = moist[j - 1] + frac * (moist[j] - moist[j - 1])
299     monotonic = bool(np.all(np.diff(max_c) <= 1e-10))
300     physical = bool(np.all(np.isfinite(moist)) and np.all(moist >= -1e-12))
301     add_check(report, "问题三_阈值", before > MOISTURE_LIMIT and after <= MOISTURE_LIMIT and abs(np.max(c_end) - MOISTURE_LIMIT) < 1e-10,
302             {"drying_time_s": drying_time, "drying_time_h": drying_time / 3600.0, "before": before, "after": after, "interpolated_max": float(
303                 np.max(c_end))},
304             "首次跨越0.15并线性插值")
305     add_check(report, "问题三_单调与范围", monotonic and physical,
306             {"monotonic": monotonic, "T_min": float(temp.min()), "T_max": float(temp.max()), "C_min": float(moist.min()), "C_max": float(moist.
307                 max())},
308             "max(C)单调不减; 有限值; C>=0")
309
310     v, area = control_volumes(0.001, len(r))
311     hm = 8e-7
312     t_func, c_func = __import__("q3").get_full_boundary_funcs()
313     c_air = np.asarray(c_func(t[1:]), dtype=float)
314     flux = hm * area * (c_air - moist[1:, -1])
315     storage = np.sum((moist[1:] - moist[: -1]) * v, axis=1)
316     m_res = abs(np.sum(storage) - np.sum(flux * 60.0)) / max(abs(np.sum(storage)), 1e-12)
317     add_check(report, "问题三_守恒", m_res < 0.01, m_res, "水分守恒相对残差<1%")
318     return {"t": t, "r": r, "T": temp, "C": moist, "maxC": max_c, "drying_time": drying_time, "m_res": m_res}
319
320
321 def q4_recompute(report: dict):
322     """独立重算第四问并检查物理范围、半径和守恒。"""
323     q4_check = q4.validate_q4()
324     stored = read_sheet(OUTPUT_DIR / "result4.xlsx", 2).iloc[:, 1:-1].to_numpy(dtype=float)
325     t, radius, x, T, C, max_C, drying_time = q4.solve_problem4()
326     err = float(np.max(np.abs(np.round(C[: -1], 4) - stored[: -1])))
327     add_check(report, "问题四_独立重算", err <= 1e-12, err, "无量纲网格结果与Excel四位小数一致")
328     add_check(report, "问题四_物理与守恒", q4_check["moisture_balance_abs"] < 1e-10,
329             q4_check, "半径单调、恒温段最大水分单调、材料坐标积分恒等式成立")
330
331     short = {}
332     for dt in (120.0, 60.0, 30.0):
333         short[dt] = q4.solve_problem4(dt=dt, dx=0.05, t_end=18000.0, stop_at_threshold=False)
334     common_t = np.arange(0.0, 18000.0 + 1e-9, 120.0)
335     vals = {dt: s[4][np rint(common_t / dt).astype(int)] for dt, s in short.items()}
336     et = float(np.max(abs(vals[120.0] - vals[60.0])), float(np.max(abs(vals[60.0] - vals[30.0]))))
337     coarse = {}
338     for dx in (0.1, 0.05, 0.025):
339         coarse[dx] = q4.solve_problem4(dt=60.0, dx=dx, t_end=18000.0, stop_at_threshold=False)
340     ec = float(np.max(abs(coarse[0.1][4][ -1] - coarse[0.05][4][ -1][: :2])),
341             float(np.max(abs(coarse[0.05][4][ -1] - coarse[0.025][4][ -1][: :2])))
342     conv = {"time_error": et, "time_ratio": et[1] / et[0], "space_error": ec, "space_ratio": ec[1] / ec[0]}
343     add_check(report, "问题四_步长收敛", et[1] < et[0] and ec[1] < ec[0]
344             and 0.35 < conv["time_ratio"] < 0.65 and 0.15 < conv["space_ratio"] < 0.35,
345             conv, "固定ε网格误差递减; 时间比约0.5, 空间比约0.25")
346     return {"check": q4_check, "convergence": conv}
347
348
349 def plot_convergence(report: dict):
350     PIC_DIR.mkdir(parents=True, exist_ok=True)
351     colors = {"T": "tab:red", "C": "tab:blue", "maxC": "tab:green"}
352     labels = [{"问题一", report["convergence"]["q1"]}, {"问题二", report["convergence"]["q2"]}, {"问题三", report["convergence"]["q3"]}]]
353
354     fig, axes = plt.subplots(1, 3, figsize=(16, 4.8))
355     for ax, (title, data) in zip(axes, labels):
356         s = data["time"]
357         x = s["dt"]
358         for key in ["T_error", "C_error", "maxC_error"]:
359             if key in s:
360                 ax.loglog(x, s[key], "o-", color=colors["T" if key == "T_error" else "C" if key == "C_error" else "maxC"], label=key.replace(
361                     "_error", ""))
362         ax.set_title(f"{title}: 时间步长")
363         ax.set_xlabel("时间步长 Δt (s)")
364         ax.set_ylabel("最大绝对误差")
365         ax.grid(alpha=0.3, which="both")
366         ax.legend()
367     fig.suptitle("三问时间步收敛交叉验证")
368     fig.tight_layout()

```

```

366 fig.savefig(PIC_DIR / "验证_时间步收敛_三问.png", dpi=300, bbox_inches="tight")
367 plt.close(fig)
368
369 fig, axes = plt.subplots(1, 3, figsize=(16, 4.8))
370 for ax, (title, data) in zip(axes, labels):
371     s = data["space"]
372     x = s["dr"]
373     for key in ["T_error", "C_error", "maxC_error"]:
374         if key in s:
375             ax.loglog(x, s[key], "o-", color=colors["T" if key == "T_error" else "C" if key == "C_error" else "maxC"], label=key.replace(
376                 "_error", ""))
377             ax.set_title(f"{title} 空间步")
378             ax.set_xlabel("空间步长 Δr (m)")
379             ax.set_ylabel("最大绝对误差")
380             ax.grid(alpha=0.3, which="both")
381             ax.legend()
382 fig.suptitle("三问空间步收敛交叉验证")
383 fig.tight_layout()
384 fig.savefig(PIC_DIR / "验证_空间步收敛_三问.png", dpi=300, bbox_inches="tight")
385 plt.close(fig)
386
387 def plot_balance(q1: dict, q2: dict, q3: dict):
388     """绘制储量变化与边界通量积分的对照图。"""
389     fig, axes = plt.subplots(2, 2, figsize=(14, 9))
390     # 问题一-能量和水分
391     v1, area1 = control_volumes(0.001, len(q1["r"]))
392     tf1, cf1 = get_q1_boundary()
393     at1, ac1 = np.asarray(tf1(q1["t"])), np.asarray(cf1(q1["t"]))
394     dq1 = np.cumsum(25.0 * area1 * (at1[1:] - q1["T"][:-1, -1]))
395     dm1 = np.cumsum(8e-7 * area1 * (ac1[1:] - q1["C"][:-1, -1]))
396     q_storage1 = np.cumsum(np.sum(820.0 * 2600.0 * np.diff(q1["T"], axis=0) * v1, axis=1))
397     m_storage1 = np.cumsum(np.sum(np.diff(q1["C"], axis=0) * v1, axis=1))
398     axes[0, 0].plot(q1["t"][1:], q_storage1, label="储能变化")
399     axes[0, 0].plot(q1["t"][1:], dq1, "--", label="边界热流积分")
400     axes[0, 0].set_title(f"问题一-能量守恒 (残差 {q1['q_res']:.2e})")
401     axes[0, 1].plot(q1["t"][1:], m_storage1, label="水分变化")
402     axes[0, 1].plot(q1["t"][1:], dm1, "--", label="边界质流积分")
403     axes[0, 1].set_title(f"问题一-水分守恒 (残差 {q1['m_res']:.2e})")
404
405     # 问题二、三-水分
406     v2, area2 = control_volumes(0.001, len(q2["r"]))
407     tf2, cf2 = get_q2_boundary(t_start=INITIAL_TIME)
408     ac2 = np.asarray(cf2(INITIAL_TIME + q2["t"]))
409     dm2 = np.cumsum(8e-7 * area2 * (ac2[1:] - q2["C"][:-1, -1]))
410     ms2 = np.cumsum(np.sum(np.diff(q2["C"], axis=0) * v2, axis=1))
411     axes[1, 0].plot(q2["t"][1:] / 3600.0, ms2, label="水分变化")
412     axes[1, 0].plot(q2["t"][1:] / 3600.0, dm2, "--", label="边界质流积分")
413     axes[1, 0].set_title(f"问题二-水分守恒 (残差 {q2['m_res']:.2e})")
414
415     v3, area3 = control_volumes(0.001, len(q3["r"]))
416     from q3 import get_full_boundary_funcs
417
418     _, cf3 = get_full_boundary_funcs()
419     ac3 = np.asarray(cf3(q3["t"]))
420     dm3 = np.cumsum(8e-7 * area3 * (ac3[1:] - q3["C"][:-1, -1]) * 60.0)
421     ms3 = np.cumsum(np.sum(np.diff(q3["C"], axis=0) * v3, axis=1))
422     axes[1, 1].plot(q3["t"][1:] / 3600.0, ms3, label="水分变化")
423     axes[1, 1].plot(q3["t"][1:] / 3600.0, dm3, "--", label="边界质流积分")
424     axes[1, 1].set_title(f"问题三-水分守恒 (残差 {q3['m_res']:.2e})")
425
426     for ax in axes.flat:
427         ax.set_xlabel("时间 (s)" if ax in axes[0, :] else "时间 (h)")
428         ax.set_ylabel("单位长度储量变化")
429         ax.grid(alpha=0.3)
430         ax.legend()
431     fig.suptitle("守恒定律交叉验证: 储量变化与边界通量积分")
432     fig.tight_layout()
433     fig.savefig(PIC_DIR / "验证_守恒对照_三问.png", dpi=300, bbox_inches="tight")
434     plt.close(fig)
435
436
437 def plot_cross_check(report: dict):
438     names = ["Q1重算T", "Q1重算C", "Q2重算T", "Q2重算C", "5500s断点T", "5500s断点C"]
439     vals = [
440         report["checks"]["问题一_独立重算"]["value"]["T_max_abs"],
441         report["checks"]["问题一_独立重算"]["value"]["C_max_abs"],
442         report["checks"]["问题二_独立重算"]["value"]["T_max_abs"],
443         report["checks"]["问题二_独立重算"]["value"]["C_max_abs"],
444         report["checkpoint"]["T_max_abs"], report["checkpoint"]["C_max_abs"],
445     ]
446     vals_plot = np.maximum(vals, 1e-16)
447     fig, ax = plt.subplots(figsize=(10, 5))
448     bars = ax.bar(names, vals_plot, color=["tab:red", "tab:blue"] * 3)
449     ax.set_yscale("log")

```

```

450 ax.set_ylabel("最大绝对差异 (对数坐标)")
451 ax.set_title("独立重算与5500 s断点交叉校验")
452 ax.grid(axis="y", alpha=0.3, which="both")
453 for bar, value in zip(bars, vals):
454     ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height(), f"{value:.2e}", ha="center", va="bottom", fontsize=9)
455 fig.tight_layout()
456 fig.savefig(PIC_DIR / "验证_独立重算交叉校验.png", dpi=300, bbox_inches="tight")
457 plt.close(fig)
458
459
460 def plot_q4_validation(report: dict):
461     """绘制问题四步长收敛和半径范围。"""
462     conv = report["checks"]["问题四_步长收敛"]["value"]
463     fig, axes = plt.subplots(1, 2, figsize=(10, 4))
464     axes[0].loglog([120, 60], conv["time_error"], "o-")
465     axes[0].set_xlabel("时间步长/s")
466     axes[0].set_ylabel("最大误差")
467     axes[0].set_title(f"时间步收敛比 {conv['time_ratio']:.4f}")
468     axes[0].grid(alpha=0.3, which="both")
469     axes[1].loglog([0.1, 0.05], conv["space_error"], "o-", color="tab:orange")
470     axes[1].set_xlabel("无量纲空间步长")
471     axes[1].set_ylabel("最大误差")
472     axes[1].set_title(f"空间步收敛比 {conv['space_ratio']:.4f}")
473     axes[1].grid(alpha=0.3, which="both")
474     fig.suptitle("问题四移动边界步长收敛")
475     fig.tight_layout()
476     fig.savefig(PIC_DIR / "验证_问题四收敛.png", dpi=300, bbox_inches="tight")
477     plt.close(fig)
478
479
480 def main():
481     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
482     PIC_DIR.mkdir(parents=True, exist_ok=True)
483     report = {"checks": {}, "convergence": {}}
484     validate_file_structure(report)
485     q1 = q1_recompute_and_balance(report)
486     q2 = q2_recompute_and_balance(report)
487     q3 = q3_full(report)
488     q4_result = q4_recompute(report)
489
490     # 5500 s 断点: 用问题三全过程重算值与 output3.xlsx 比较
491     t_func, c_func = get_q2_boundary(path=OUTPUT_DIR / "output1.xlsx", t_start=0.0)
492     _, r_chk, t_chk, c_chk = solve_q2(
493         1.0, 0.001, t_end=5500.0, t_start=0.0, initial_file=None,
494         initial_T=28.0, initial_C=2.55, T_air_func=t_func, C_air_func=c_func,
495     )
496     _, t_ref, c_ref = load_initial_profile(DEFAULT_INITIAL_FILE, r_max=R_MAX, Nr=len(r_chk))
497     checkpoint = {"T_max_abs": float(np.max(abs(t_chk[-1] - t_ref))), "C_max_abs": float(np.max(abs(c_chk[-1] - c_ref)))}
498     add_check(report, "问题二_5500s断点", checkpoint["T_max_abs"] <= 1e-4 and checkpoint["C_max_abs"] <= 1e-4, checkpoint, "全过程重算与output3最大
        差异<=1e-4")
499     report["checkpoint"] = checkpoint
500
501     report["convergence"]["q1"] = convergence_q1(report)
502     report["convergence"]["q2"] = convergence_q2(report)
503     report["convergence"]["q3"] = convergence_q3(report)
504     report["convergence"]["q4"] = q4_result["convergence"]
505     plot_convergence(report)
506     plot_balance(q1, q2, q3)
507     plot_cross_check(report)
508     plot_q4_validation(report)
509
510     report["summary"] = {
511         "all_passed": True,
512         "q3_drying_time_s": q3["drying_time"],
513         "q3_drying_time_h": q3["drying_time"] / 3600.0,
514         "q4_drying_time_s": q4_result["check"]["drying_time_s"],
515         "q4_drying_time_h": q4_result["check"]["drying_time_s"] / 3600.0,
516         "q4_radius_start_cm": q4_result["check"]["radius_start_cm"],
517         "q4_radius_end_cm": q4_result["check"]["radius_end_cm"],
518         "figure_files": [
519             "验证_时间步收敛_三问.png",
520             "验证_空间步收敛_三问.png",
521             "验证_守恒对照_三问.png",
522             "验证_独立重算交叉校验.png",
523             "q4_半径插值图.png",
524             "q4_阈值判定图.png",
525             "q4_水分热力图.png",
526             "验证_问题四收敛.png",
527         ],
528     }
529     REPORT_FILE.write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
530     print(f"全部交叉验证通过, 报告已保存: {REPORT_FILE}")
531     print(json.dumps(report["summary"], ensure_ascii=False, indent=2))
532
533

```

```
534 if __name__ == "__main__":  
535     main()
```

Listing 9: 全部问题交叉验证程序